CSE 6213
Natural Language Processing
Lecture - 1

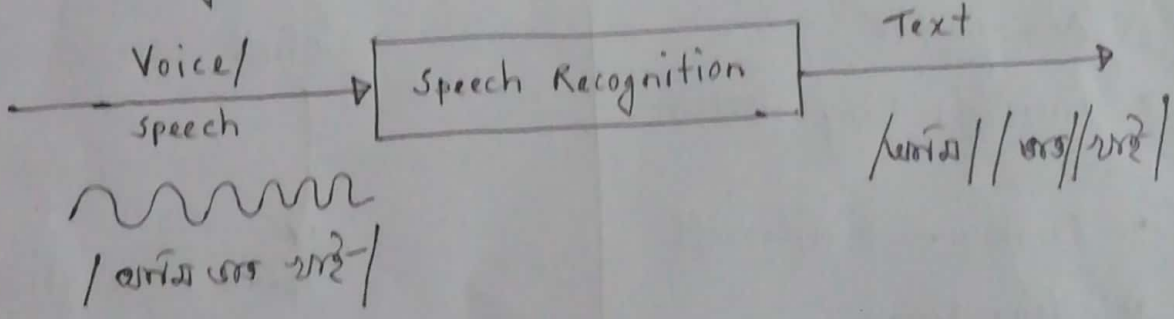
## Natural Language Processing: (NLP)

It is a subfield of linguistics, computer science, and artificial intelligence concerned with the interactions between computers and human language, in particular how to program computers to process and analyze large amounts of natural language data.

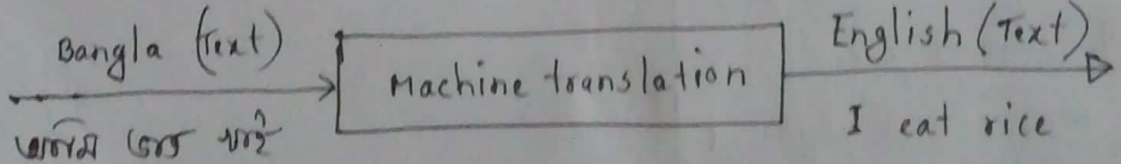
## What is natural language processing with example?

We can use NLP to create systems like speech recognition, machine translation, Speech Synthesis, document summarization, Question answering system, spam detection and so on.

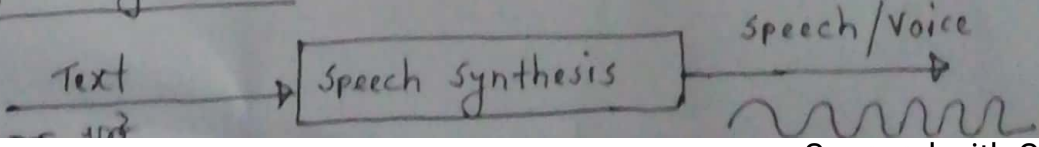
### Speech Recognition: (STT → Speech to Text)




### Machine translation: (MT → Machine Translation)



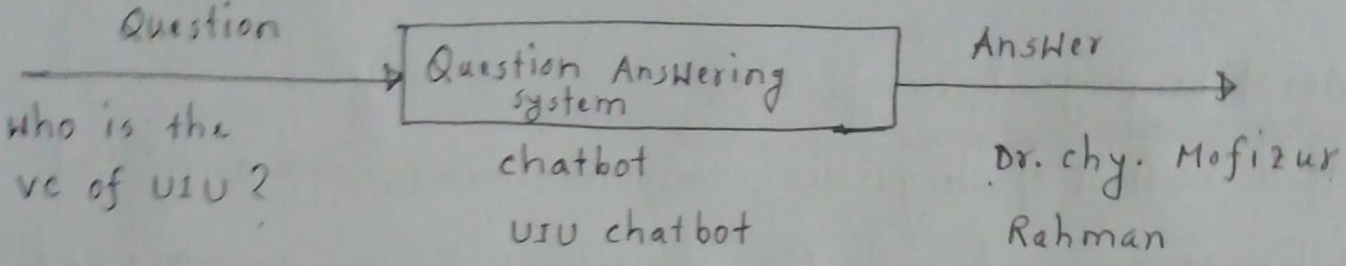

### Speech synthesis: (TTS → Text to Speech)




Scanned with CamScanner

~~Speech Type~~

## Question Answering System:



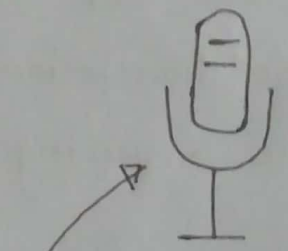

## Speech Recognizer:

Voice Recognition software:
1. Dragon Professional
2. Dragon Anywhere
3. Google Now
4. Google cloud Speech API
5. Google Docs Voice

Typing:
6. Siri → Apple
7. Amazon Lex
8. Microsoft Bing Speech

API:
9. Cortana → Microsoft
10. Voice Finger
    → Speech recognition tool that enables you to control your mouse and keyboard just using your voice (Microsoft)

1. https://speechnotes.co/
2. https://dictation.io/speech



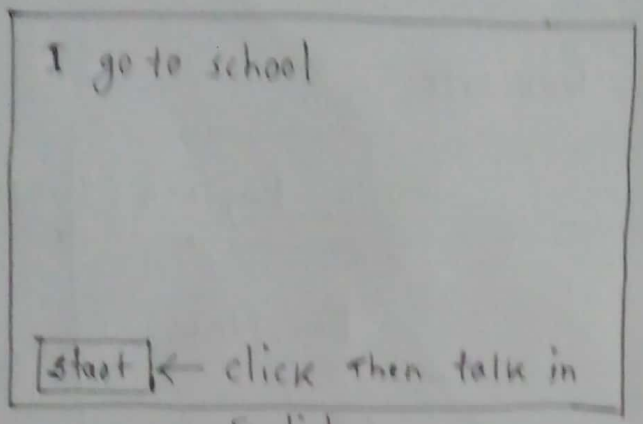
click

Speak in english now

You will see written text

Scanned with CamScanner

2. https://dictation.io/speech



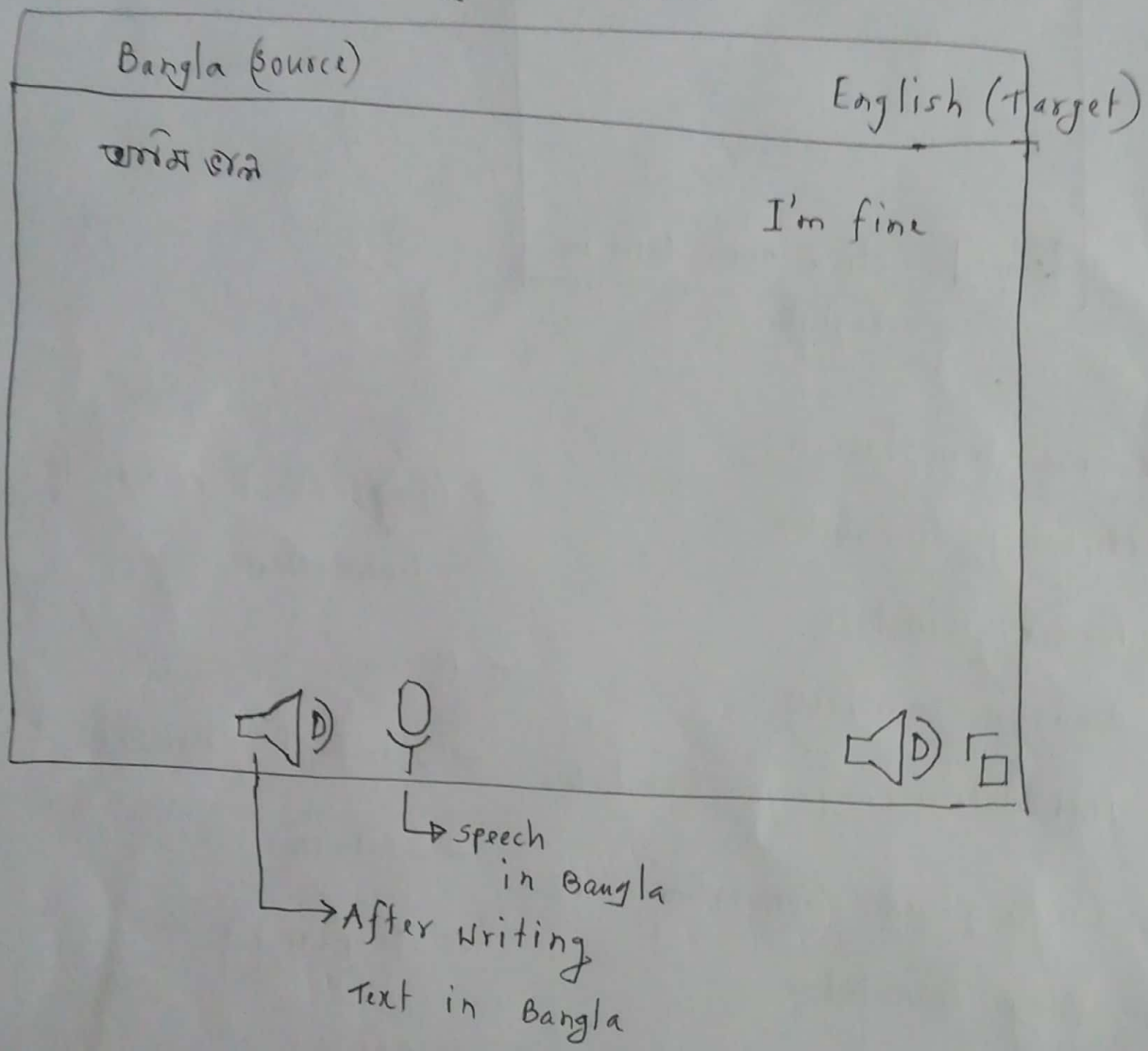

## Machine Translation:

- Microsoft Translator
- Yandex. Translate
- Amazon Translate
- IBM Watson Language Translator
- Google cloud Translation API
- Bing Translator
- Babylon
- Slate Desktop
- Azure Translator Text API
- Gram Trans
- Apertium
- OpenLogos
- Anusaaraka
- Google cloud AutoML Translation
- Moses
- Stanford Phrasal
- Systran
- Triplingo
- AppTek
- Awaken Translate
- Iconic Translation Machines
- Idiomax
- KatanMT.com
- Language studio
- Lengoo

Scanned with CamScanner

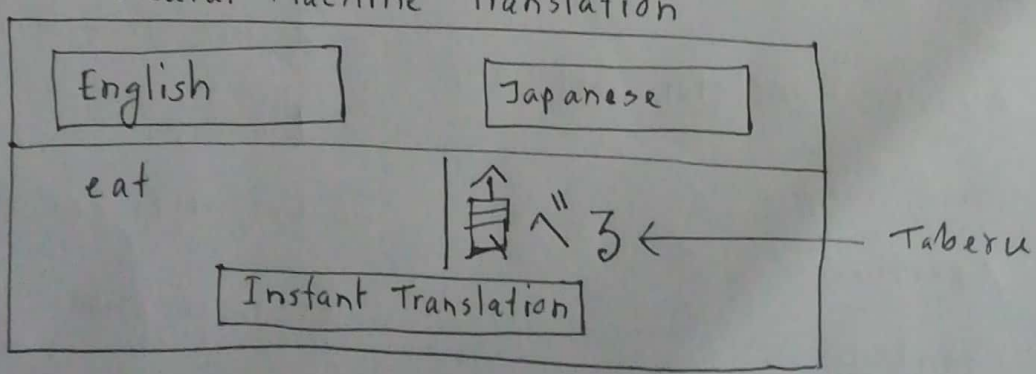
- NiuTrans
- Personal Translator
- Promt Neural Translation Server 20
- SDL Beglobal

Google Translate




Systran

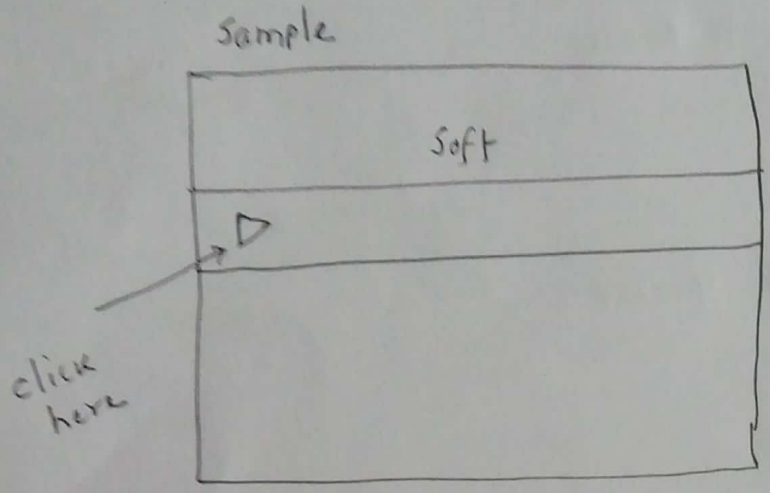
Neural Machine Translation




Scanned with CamScanner

## Speech Synthesis:

Microsoft Text-to-speech voices

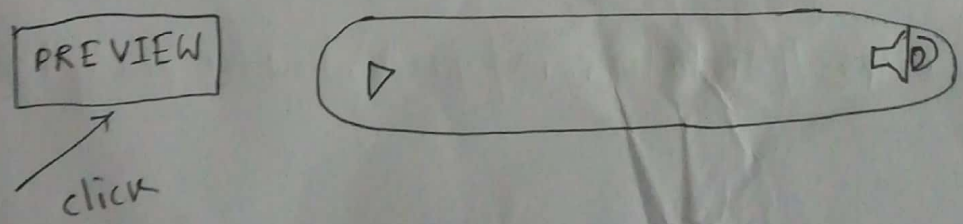



Text: The quick brown fox jumps over the lazy dog 1234567,890 times

https://wideo.co/text-to-speech/ — Wideo

Write your text here
I am fine. How Are you?

Voice
[speaker name]

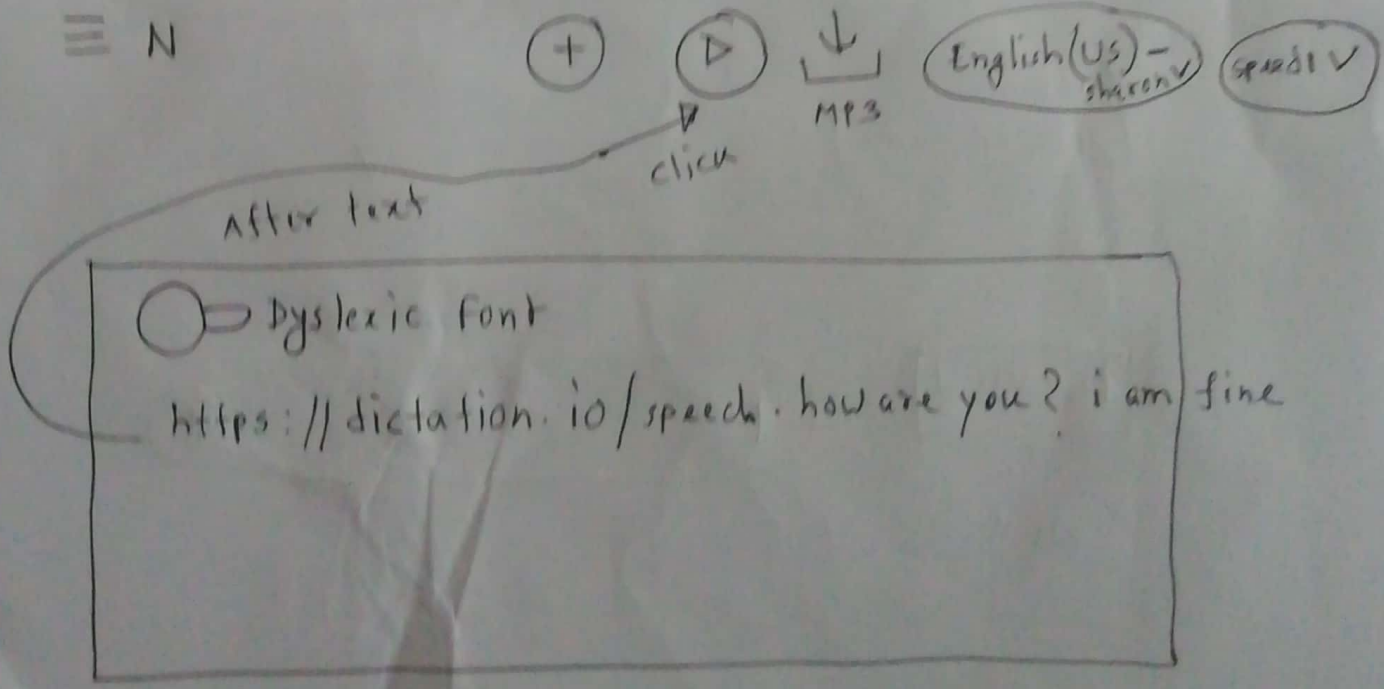



Scanned with CamScanner

1. Ivona
2. Natural Reader
3. Balabolka Text to Speech Reader
4. ispeech
5. Acapela Group Virtual Speaker
6. TextSpeech Pro
7. AudioBook Maker
8. TextAloud 3
9. Read The Words
10. Voice Reader 15

(*) https://www.naturalreaders.com/online

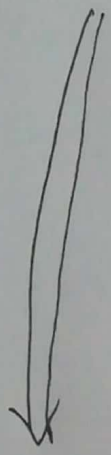
≡ N




Scanned with CamScanner

## Question Answering System:

Start Natural Language Question Answering system



how are you?

Ask Question?

⟹ how are you?

Hi I am ready now to answer your questions

start.csail.mit.edu/index.php

What is your name?

Ask Question?

⇒ What is your name?

My name is start; it stands for SynTactic Analysis using Reversible Transformation.

- Google Search
- chatbot (EBL DIA, eGenbot, Facebook Messanger)
- START
- BERT

Book:

Foundations of statistical Natural Language Processing

- Christopher Manning and Hinrich Schutze

Scanned with CamScanner

Topics

1. Basic / Introduction
2. Finite state automata
3. Regular expression
4. Context free grammar
5. Parsing
6. Morphology
7. Part-of-speech Tagging (POS tagging)
8. Speech Recognition and phonetics
9. Language Model (N-gram Model)
10. Machine Translation
11. TTS (Text to speech)
12. Semantics and Syntax
13. Information Retrieval
14. Text Categorization
15. Word Sense Disambiguation
16. clustering
17. Hidden Markov Model

Scanned with CamScanner

Topic:

Project:

Report on

i) Speech Recognizer

ii) Speech Synthesizer

iii) Machine Translation

iv) Question Answering system / chatbot

Scanned with CamScanner

CSE 6013
Computational Linguisti
Lecture-I

Computational Linguistics:

It builds on the theory and practice of multiple fields (linguistics, computer science and statistics) to design computer applications that involve the automatic processing of natural language speech or text by machines.

Course Outlines:

1. Finite state automata
2. Regular expression
3. Context free grammar
4. Parsing
5. Morphology
6. phonetics and Speech Recognition
7. N-gram
8. TTS (Text to Speech)
9. Semantics and Syntax
10. Machine Translation

## Automata theory:

**Alphabet:** An alphabet is a finite, nonempty set of symbols.
$\Sigma$ is alphabet symbol.

Example:
binary alphabet, $\Sigma = \{0, 1\}$
set of all lower-case letters, $\Sigma = \{a, b, \ldots, z\}$

**String:**
A string is a finite sequence of symbols chosen from alphabet.

Example: 01101 is a string, where $\Sigma = \{0, 1\}$
computer is a string, where $\Sigma = \{a, b \ldots z\}$

Scanned with CamScanner

## Empty string:

The empty string is the string with zero occurrences

$\epsilon$ is empty string.

## Length of a string:

The length of a string is the number of positions for symbols in the string.

string 01101, length of 01101 = $|01101| = 5$

string computer, length of computer = $|\text{computer}| = 8$

Empty string $\epsilon$, length of $\epsilon = |\epsilon| = 0$

## Powers of an Alphabet

If $\Sigma$ is an alphabet, we can express the set of all strings of a certain length from that alphabet by using an exponential notation.

$\Sigma^k$ = the set of strings of length k.

Example:

$\Sigma^0 = \{\epsilon\}$

$\Sigma^1 = \{0, 1\}$

$\Sigma^2 = \{00, 01, 10, 11\}$

$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

$\Sigma^*$ = The set of all strings

$= \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \ldots$

$= \{\epsilon\} \cup \{0,1\} \cup \{00, 01, 10, 11\} \cup \ldots$

$= \{\epsilon, 0, 1, 00, 01, 10, 11, \ldots\}$

$\Sigma^+$ = The set of all non-empty strings

$= \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \ldots$

Scanned with CamScanner

$\Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \cdots$

$= \Sigma^0 \cup \Sigma^+$

$= \{\epsilon\} \cup \Sigma^+$

$= \Sigma^+ \cup \{\epsilon\}$

## Concatenation of strings:

Let x and y be strings. Then xy denotes the concatenation of x and y, that is, the string formed by making a copy of x and following it by a copy of y.

Example: $x$ = "my"

$y$ = "computer"

$xy$ = "mycomputer"

$yx$ = "computermy"

## Languages:

A set of strings all of which are chosen from some $\Sigma^*$, where $\Sigma$ is a particular alphabet, is called a language.

If $\Sigma$ is an alphabet, and $L \subseteq \Sigma^*$, then L is a language over $\Sigma$

### Example:

alphabet, $\Sigma = \{a, b, c, \cdots, z\}$, $\Sigma^0 = \{\epsilon\}$

$\Sigma^1 = \{a\}$

$\Sigma^2 = \{am, an, to, of, \cdots\}$

$\Sigma^3 = \{boy, eat, cat, sat, mat, \cdots\}$

$\vdots$

$\therefore \Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \cdots$

$= \{\epsilon\} \cup \{a\} \cup \{am, an, to, of, \cdots\} \cup \{boy, eat, cat,$

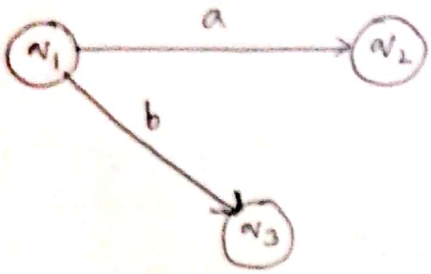
Scanned with CamScanner

$= \{ \epsilon, a, am, an, to, of, \ldots, boy, eat, cat, sat, mat, \ldots \}$

A set of strings all of which are chosen from $\Sigma^*$ is called English language.

## Deterministic Finite Automata

Deterministic refers to the fact that on each input there is one and only one state to which the automation can transition from its current state.




## Definition of DFA (Deterministic Finite Automata)

1. Finite set of states, Q
2. Input symbols $\Sigma$
3. Transition function $\delta$

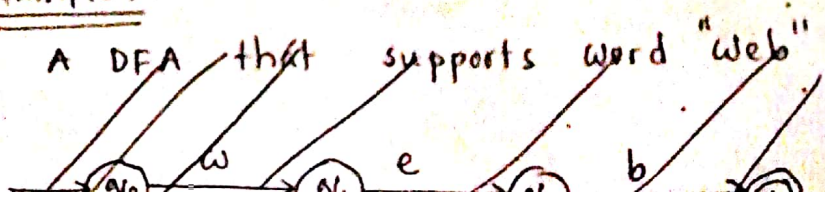
$$\delta(q_1, a) = q_2 \quad \text{[returns one state]}$$

4. starting state $q_0$.
5. A set of final or accepting states F.

$$\therefore \text{DFA} = (Q, \Sigma, \delta, q_0, F)$$

## Example:

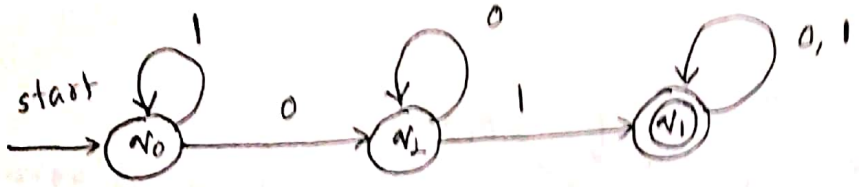
A DFA that supports word "web"




Scanned with CamScanner

Q: Construct a DFA that accepts all and only the strings of 0's and 1's that have the sequence 01 somewhere in the string.

Ans:




Here, $Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

$\delta(q_0, 1) = q_0$, $\delta(q_0, 0) = q_2$

$\delta(q_2, 0) = q_2$, $\delta(q_2, 1) = q_1$

$\delta(q_1, 0) = q_1$, $\delta(q_1, 1) = q_1$

starting state, $q_0$

Final state, $F = \{q_1\}$

Ques: show that the following DFA accept string 10011

Ans:

$\delta(q_0, 1) = q_0$

$\delta(q_0, 10) = \delta(\delta(q_0, 1), 0) = \delta(q_0, 0) = q_2$

$\delta(q_0, 100) = \delta(\delta(q_0, 10), 0) = \delta(q_2, 0) = q_2$

$\delta(q_0, 1001) = \delta(\delta(q_0, 100), 1) = \delta(q_2, 1) = q_1$

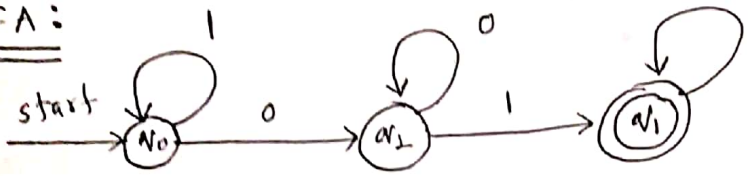
$\delta(q_0, 10011) = \delta(\delta(q_0, 1001), 1) = \delta(q_1, 1) = q_1$

Scanned with CamScanner

## Transition Table:

A transition table which is a tabular listing of function.

DFA:




⇐ Transition diagram

| | 0 | 1 |
|---|---|---|
| → $q_0$ | $q_2$ | $q_0$ |
| * $q_1$ | $q_1$ | $q_1$ |
| $q_2$ | $q_2$ | $q_1$ |

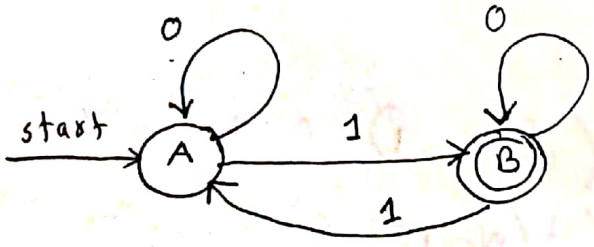
⇐ Transition Table

Ques:

| | 0 | 1 |
|---|---|---|
| → A | A | B |
| * B | B | A |

Describe the language accepted by this DFA.

Ans:



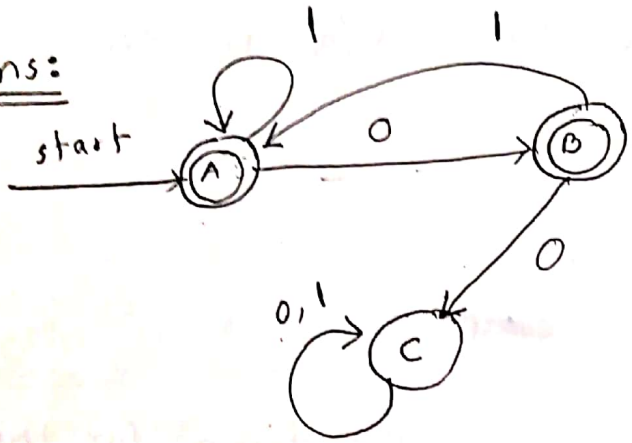

This DFA supports all the strings, ~~that contains ending with 1~~ which have ~~at least one 1 at any position~~ odd number 1's

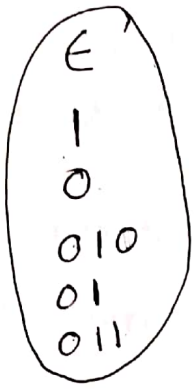
Scanned with CamScanner

| | 0 | 1 |
|---|---|---|
| → *A | B | A |
| *B | C | A |
| C | C | C |

Describe the language accepted by this DFA.

Ans:




This DFA supports all the strings, which ~~have~~ may have at least one zero at any position.

## Non-Deterministic Finite Automata (NFA)

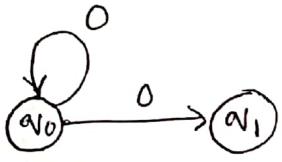
A Non-deterministic finite automation (NFA) has the power to be in several states at once.

### Definition of NFA:

$$NFA = (Q, \Sigma, \delta, q_0, F)$$

1. Finite set of states, $Q$
2. Input symbols $\Sigma$
3. Transition function $\delta$

$$\delta(q_0, 0) = \{q_0, q_1\}$$

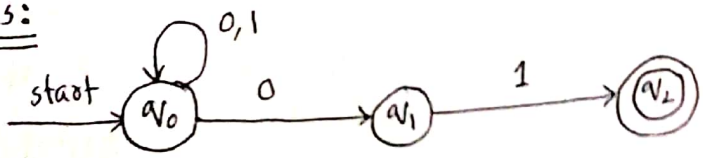



[may return more than one state]

4. starting state $q_0$
5. A set of final or accepting states $F$.

Scanned with CamScanner

**Ans:**




Here, $Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

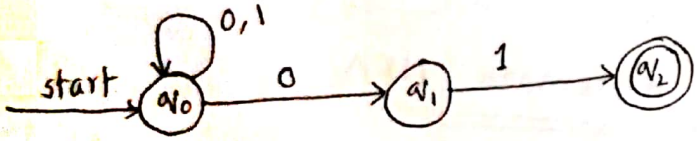
$\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_0\}$

$\delta(q_1, 1) = q_2$

starting state $q_0$

$F_\bullet$ = Final state $\{q_2\}$

**Ques:** show the string 1101 is valid string for the following NFA.




**Ans:**

$\delta(q_0, 1) = \{q_0\}$

$\delta(q_0, 11) = \delta(\delta(q_0, 1), 1)$

$= \delta(q_0, 1)$

$= \{q_0\}$

$\delta(q_0, 110) = \delta(\delta(q_0, 11), 0)$

$= \delta(\{q_0\}, 0)$

$= \{q_0, q_1\}$

$\delta(q_0, 1101) = \delta(\delta(q_0, 110), 1)$

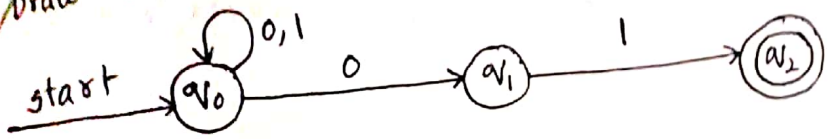
$= \delta(\{q_0, q_1\}, 1)$

$= \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\}$

$= \{q_0, q_2\}$

Scanned with CamScanner

Draw transition table

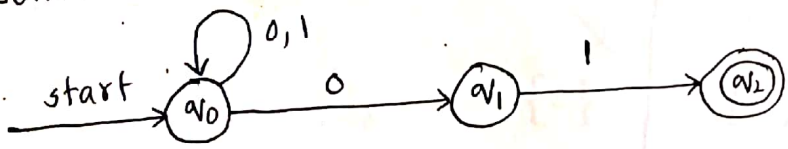



Ans:

| | 0 | 1 |
|---|---|---|
| → $q_0$ | $\{q_0, q_1\}$ | $\{q_0\}$ |
| $q_1$ | $\emptyset$ | $\{q_2\}$ |
| * $q_2$ | $\emptyset$ | $\emptyset$ |

## Convertion of NFA to DFA

Convert the following NFA to DFA



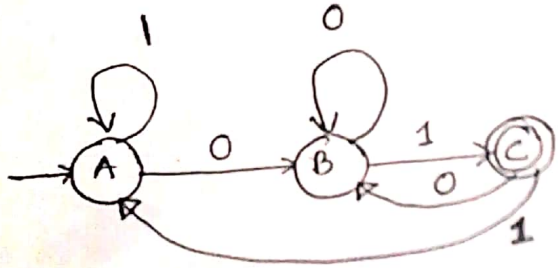

Ans:

| | 0 | 1 |
|---|---|---|
| → $\{q_0\}$ | $\{q_0, q_1\}$ | $\{q_0\}$ |
| $\{q_0, q_1\}$ | $\{q_0, q_1\}$ | $\{q_0, q_2\}$ |
| * $\{q_0, q_2\}$ | $\{q_0, q_1\}$ | $\{q_0\}$ |

⇓

| | 0 | 1 |
|---|---|---|
| → A | B | A |
| B | B | C |
| * C | B | A |

⇒




Scanned with CamScanner

## Exercise:

Give DFA's accepting the following languages over the alphabet $\{0, 1\}$

a) The set of all strings ending in 00.

b) The set of all strings with three consecutive 0's (not necessarily at the end).

c) The set of strings with 011 as a substring

## Exercise

Convert the following NFA into DFA

a)

| | 0 | 1 |
|---|---|---|
| → p | {p, q} | {p} |
| q | {r} | {r} |
| r | {s} | ∅ |
| *s | {s} | {s} |

b)

| | 0 | 1 |
|---|---|---|
| → p | {q, s} | {q} |
| *q | {r} | {q, r} |
| r | {s} | {p} |
| *s | ∅ | {p} |

Application of Finite automation:

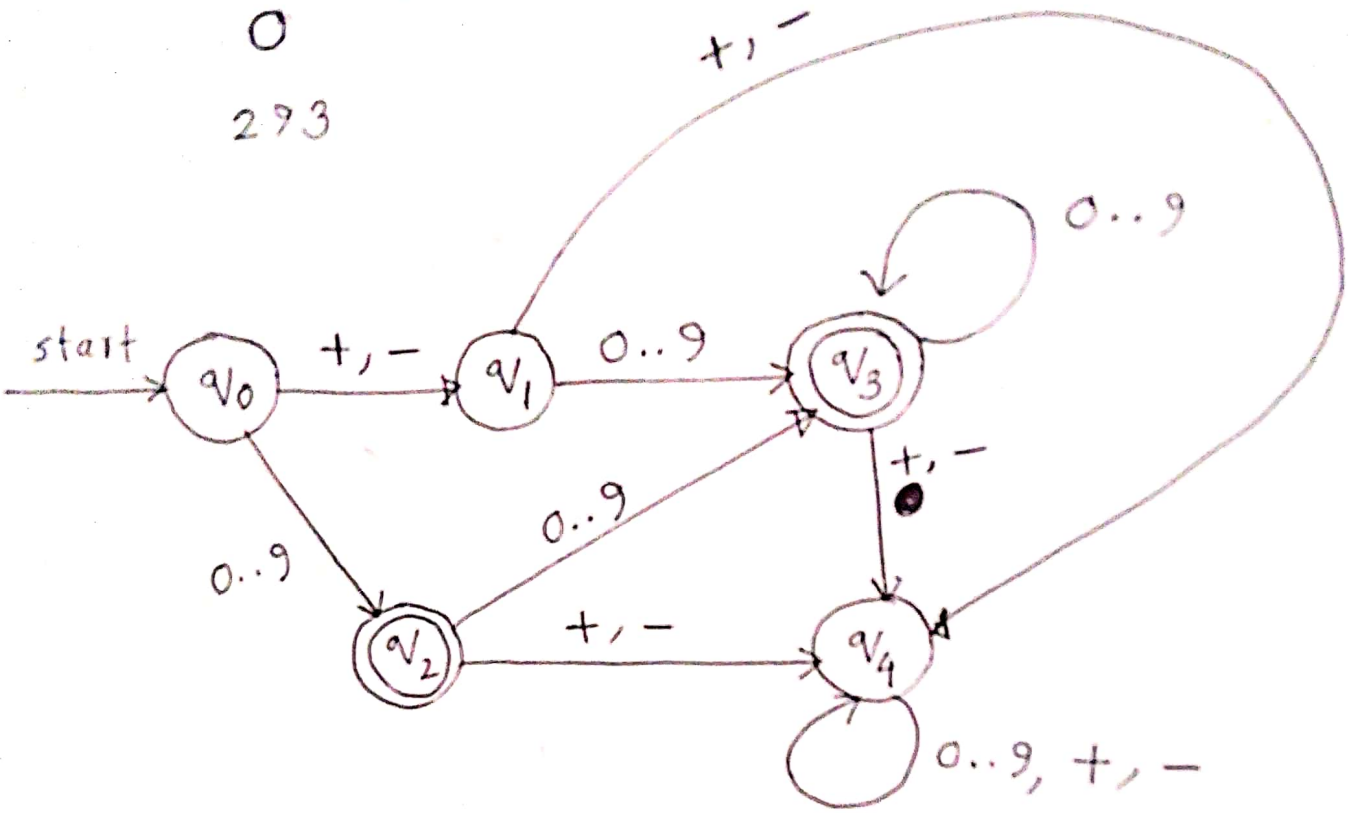
- Web search
- Information retrieval from text.

Scanned with CamScanner

## All Integer Numbers DFA

+9
−10
0
293

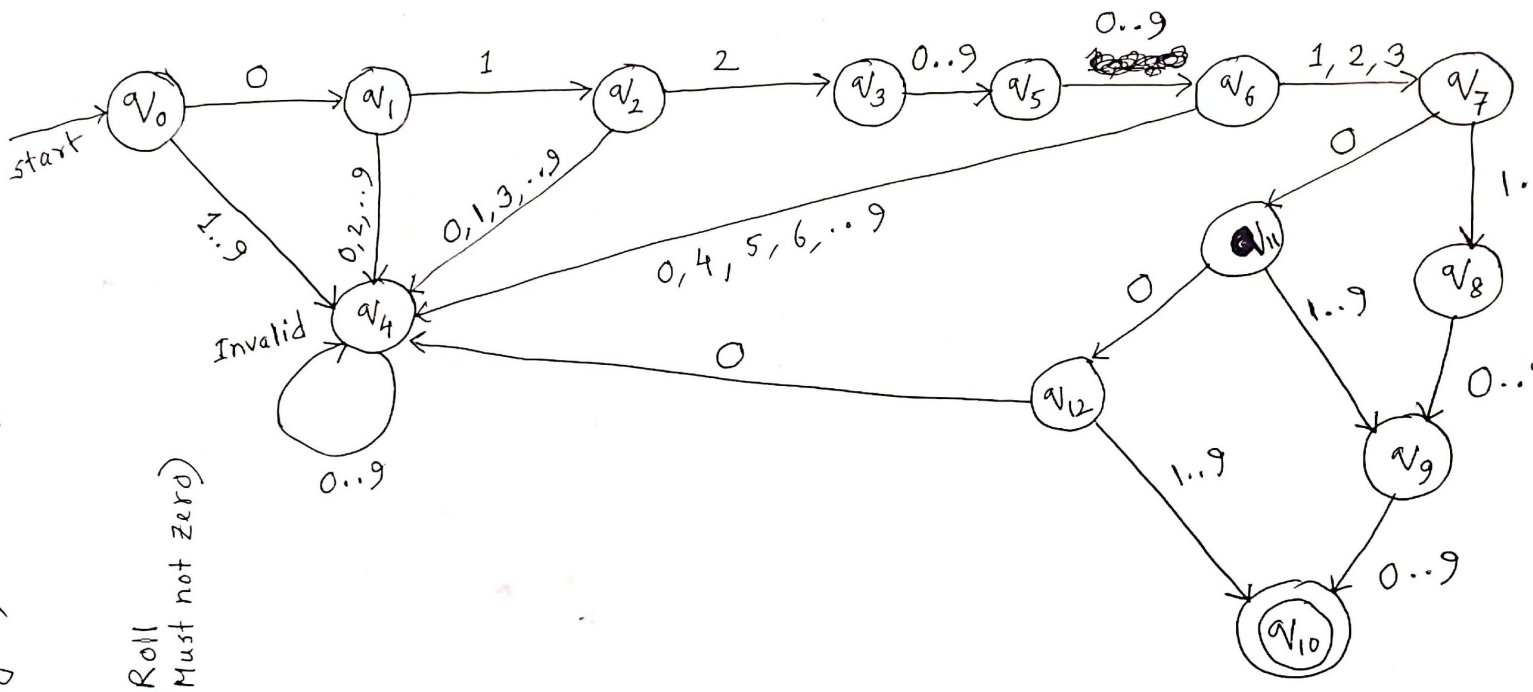
99
9+ ← Invalid
+8−98




## Positive integer numbers DFA

9
98
456

Scanned with CamScanner




MSCSE ID

Spring 1, summer 2, Fall 3

012 20 3001

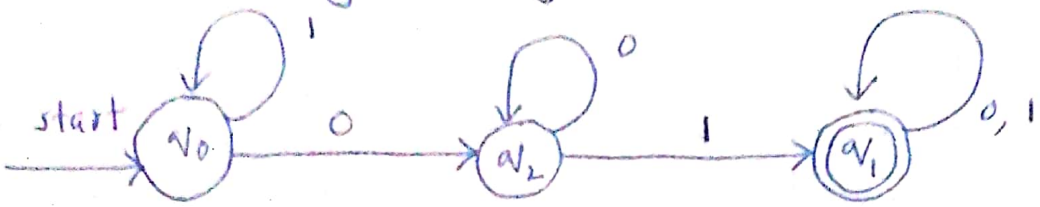
Roll (Must not zero)

year

MSCSE code

Scanned with CamScanner

Draw a DFA (Deterministic Finite Automation) that supports 01 as substring at any place.




Input string: 110011

~~1 1 0 0 1 0 1~~

1 1 0 0 1 1 $\Longrightarrow$ $q_0 \rightarrow q_0 \rightarrow q_2 \rightarrow q_2 \rightarrow q_1 \rightarrow q_1$

Valid string

Final state or Accepting state

~~1 0 1 1 0~~ $\rightarrow q_0$

1 1 1 0 $\Longrightarrow$ $q_0 \rightarrow q_0 \rightarrow q_0 \rightarrow q_2$

Invalid string

Not final state or Not Accepting state

Scanned with CamScanner

Lecture-2
Prof. Dr. Nurul Huda
Computational Linguistics

## Application of DFA

- Web search
- Extraction of information from text

## Finding Strings in Text

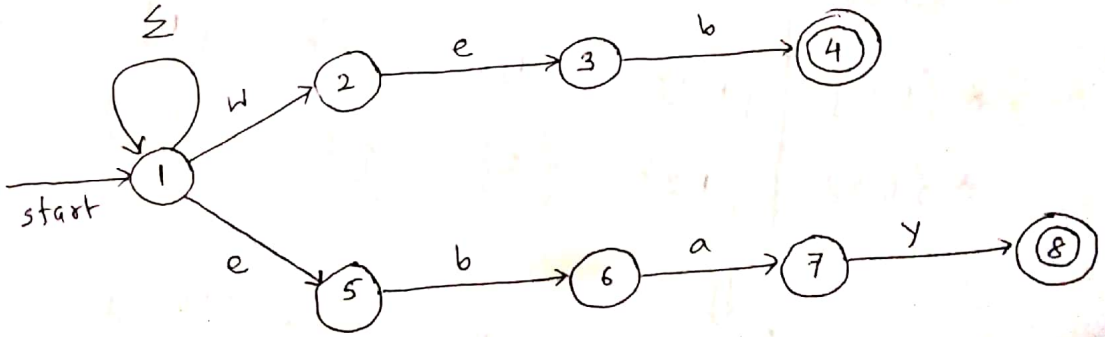
Given a set of words, find all documents that contain one (or all) of those words.

Example: A search engine is a popular examples of this process.

## Non-deterministic Finite Automata for Text search:

**Ques:** Suppose we want to design a NFA to recognize occurrences of the words web and ebay.

**Ans:**



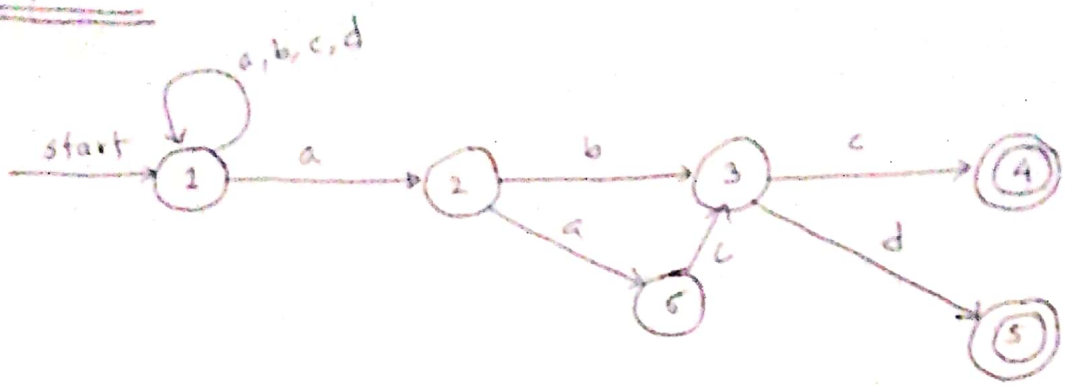

**Ques:**

1. Design NFA's to recognize the following sets of strings
   a) abc, abd, aacc and aacd. Assume the alphabet is {a, b, c, d}
   b) ab, bc, and ca. Assume the alphabet is {a, b, c}

2. Convert each of the NFAs of Question 1 into DFA's.

Scanned with CamScanner

1 a) Ans:




Conversion to DFA

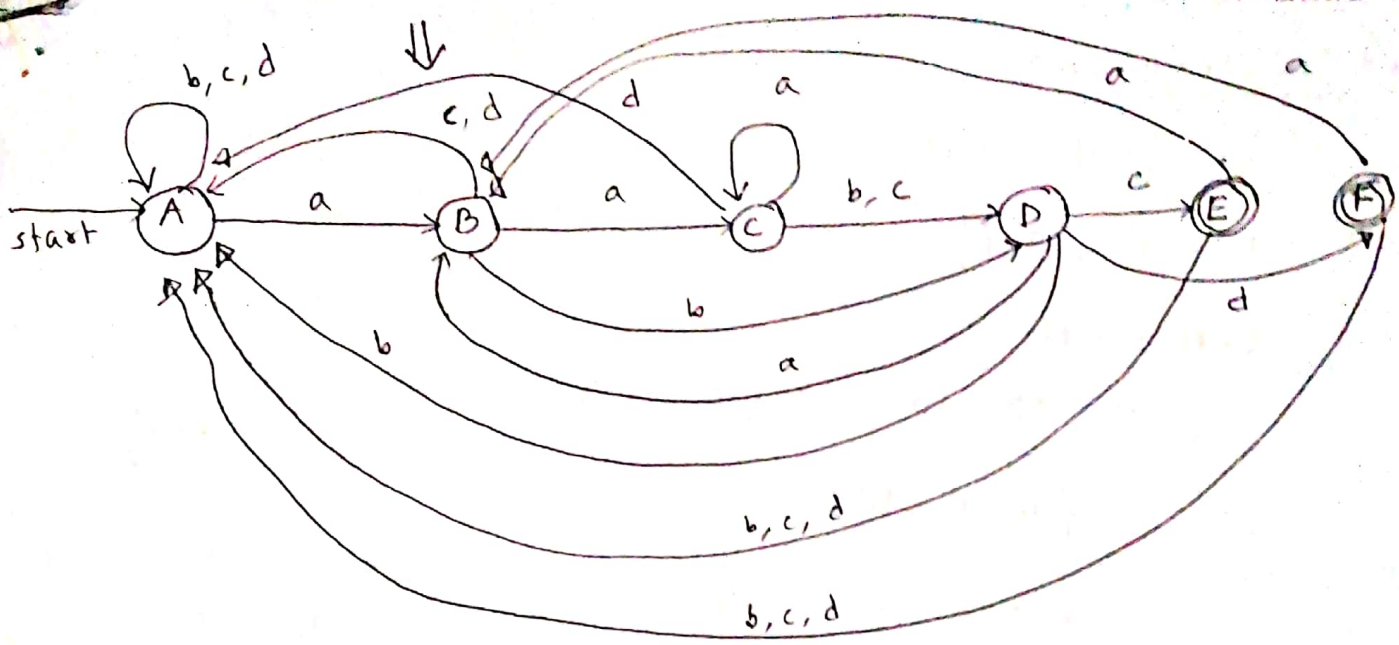
Transition table for DFA:

| | a | b | c | d |
|---|---|---|---|---|
| → {1} | {1,2} | {1} | {1} | {1} |
| {1,2} | {1,2,6} | {1,3} | {1} | {1} |
| {1,2,6} | {1,2,6} | {1,3} | {1,3} | {1} |
| {1,3} | {1,2} | {1} | {1,4} | {1,5} |
| * {1,4} | {1,2} | {1} | {1} | {1} |
| * {1,5} | {1,2} | {1} | {1} | {1} |

⇓

| | a | b | c | d |
|---|---|---|---|---|
| → A | B | A | A | A |
| B | C | D | A | A |
| C | C | D | D | A |
| D | B | A | E | F |
| * E | B | A | A | A |
| * F | B | A | A | A |

Scanned with CamScanner




## Regular expressions:

Regular expressions denote language.

Example:

Variable declaration regular expression:

$$L(L|D)^*$$

Language: A variable should be started with L, then any number of L or D may be occurred.

## Three operations on Languages:

1. Union
2. Concatenation
3. closure

## Union:

Two languages L and M

Let $L = \{001, 10, 111\}$

$M = \{\epsilon, 001\}$

$$L \cup M = \{001, 10, 111\} \cup \{\epsilon, 001\}$$

$$= \{\epsilon, 10, 001, 111\}$$

Scanned with CamScanner

Concatenation:

$L = \{001, 10, 111\}$

$M = \{\epsilon, 001\}$

$L.M = LM$

$= \{001, 10, 111\}\ \{\epsilon, 001\}$

$= \{001\epsilon, 10\epsilon, 111\epsilon, 001001, 10001, 111001\}$

$= \{001, 10, 111, 001001, 10001, 111001\}$

Closure:

$L = \{0, 1\}$

$L^* = \{\epsilon, 0, 1, 01, 10, \ldots\}$

regular expressions • inductive steps

Union: If E and F are regular expressions,

E+F is a regular expression denotes the union of L(E) and L(F) That is, L(E+F) = L(E)UL(F

Concatenation:

If E and F are regular expressions,

EF is a regular expression denotes the concatenate -nation of L(E) and E(F). That is L(EF) = L(E) L(F).

closure

If E is a regular expression, then $E^*$ is a regular expression, denotes the closure of L(E). That is, $L(E^*) = (L(E))^*$

Parenthesised

If E is a regular expression, (E) is also a regular expression. Both E and (E) denote same language, that is L((E)) = L(E).

Scanned with CamScanner

Example: (Union)

$E = 0$

$F = 1$

$E+F = 0+1$ regular expression

$L(E) = \{0\}$

$L(F) = \{1\}$

$L(E) \cup L(F) = \{0\} \cup \{1\} = \{0,1\}$

$L(E+F) = L(0+1) = \{0,1\}$

$\therefore L(E+F) = L(E) \cup L(F)$

Example: (Concatenation)

$E = 0$

$F = 1$

$EF = 01$ regular expression

$L(E) = \{0\}$

$L(F) = \{1\}$

$L(E)\,L(F) = \{0\}\{1\} = \{01\}$

$L(EF) = L(01) = \{01\}$

$\therefore L(EF) = L(E)\,L(F)$

Example: (closure)

$E = 1$

$L(E) = \{1\}$

$L(E^*) = L(1^*) = \{\epsilon, 1, 11, 111, \ldots\}$

$(L(E))^* = \{1\}^* = \{\epsilon, 1, 11, 111, \ldots\}$

$\therefore L(E^*) = (L(E))^*$

Scanned with CamScanner

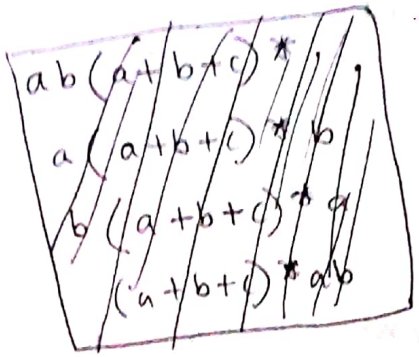
Write regular expression for the following language
All strings both begin and end with 0 over the alphabet {0, 1}

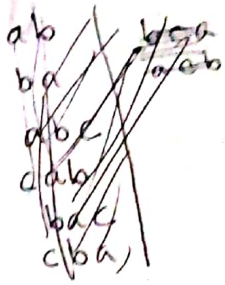
Ans:

$0(10)^*$

Write regular expressions for the following languages
The set of strings over alphabet {a, b, c} containing at least one a and at least one b.

Ans:




~~ab+(a+b+c)*+ba~~

$(ab+ba)(a+b+c)^*$

Find $L^*$

$L = \{0, 11\}$

$L^0 = \{\epsilon\}$

$L^1 = \{0, 11\}$

$L^2 = \{00, 011, 110, 1111\}$

$L^3 = \{000, 0011, 0110, 1100, 01111, 11011, 11110, 111111\}$

⋮

Scanned with CamScanner

Grammar :

Four important components of Grammar

1. Terminals, or terminal symbols (T)
2. Non-terminals or syntactic categories (V)
3. start symbol (S)
4. Production or rules (P)

Grammar, $G = (V, T, P, S)$

V: variable

## Grammar for Palindroms

$S \longrightarrow P$

$P \longrightarrow 0P0 \mid 1P1$

$P \longrightarrow 0 \mid 1 \mid \epsilon$

$\therefore G_{pal} = (\{P\}, \{0,1\}, P, S)$

P: production; {0,1}: terminal; S: start; {P}: variable

## Derivation Using Grammar

### Derivation:

The language of the grammar is all strings of terminals that we can obtain in head to body manner. This use of grammars is called derivation.

Example:

$S \longrightarrow P$

$\longrightarrow 0P0$

$\longrightarrow 01P10$

$\longrightarrow 01\epsilon10 \longrightarrow 0110$

Scanned with CamScanner

## Left most derivation :

In a derivation, if we replace the leftmost variable by one of its production bodies, such derivation is called a leftmost derivation.

step by step

Example:

$S \longrightarrow E$

$E \longrightarrow E+E \mid E-E$

$E \longrightarrow I$

$I \longrightarrow id$

string: id-id+id

Left most derivation:

$S \xrightarrow{lm} E$

$\xrightarrow{lm} E-E$

$\xrightarrow{lm} I-E$

$\xrightarrow{lm} id-E$

$\xrightarrow{lm} id-E+E$

$\xrightarrow{lm} id-I+E$

$\xrightarrow{lm} id-id+E$

$\xrightarrow{lm} id-id+I$

$\xrightarrow{lm} id-id+id$

## Right most derivation :

In a derivation, if we replace the rightmost variable by one of its production bodies step by step, such derivation is called a rightmost derivation.

Scanned with CamScanner

Example:

$S \rightarrow E$

$E \rightarrow E+E \mid E-E$

$E \rightarrow I$

$I \rightarrow id$

string: id-id+id

Right most derivation:

$S \xrightarrow{rm} E$

$\xrightarrow{rm} E+E$

$\xrightarrow{rm} E+I$

$\xrightarrow{rm} E+id$

$\xrightarrow{rm} E-E+id$

$\xrightarrow{rm} E-I+id$

$\xrightarrow{rm} E-id+id$

$\xrightarrow{rm} id-id+id$

Parse Trees:

A tree representation for derivation is called parse tree.

$S \rightarrow E$

$E \rightarrow E+E \mid E-E$

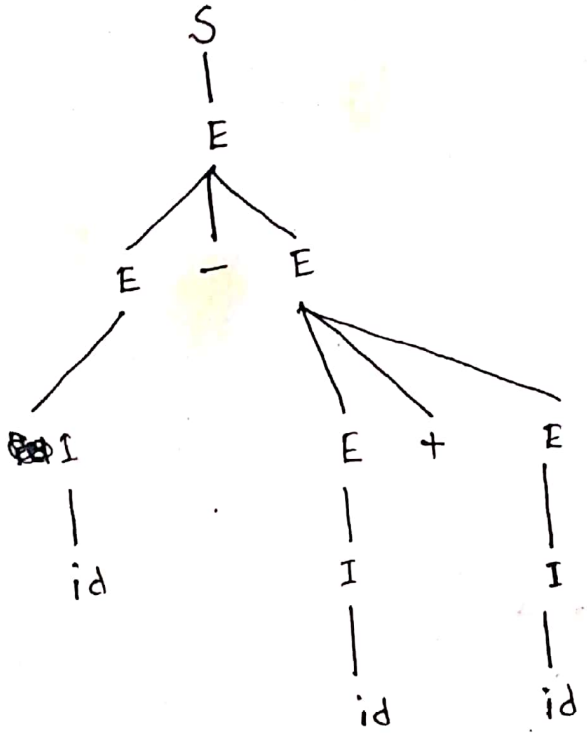
$E \rightarrow I$

$I \rightarrow id$

string: id-id+id

Scanned with CamScanner

parse tree:




If we look at the leaves of any parse tree and concatenate them from the left, we get a string, called the yield of the tree.

Example: id - id + id is a yield of the tree

## Grammar for regular expression

$0^* 1 (0+1)^*$

Grammar:

$S \longrightarrow A\,1\,B$

$A \longrightarrow \epsilon \mid 0A$

$B \longrightarrow \epsilon \mid 0B \mid 1B$

Scanned with CamScanner

## Regular expression (RE) :

(I) $R_1 | R_2 = R_1 + R_2 = R_1 \text{ OR } R_2 = R_2 | R_1$

(II) $R_1 R_2 \neq R_2 R_1$ . $R_1 \cdot R_2 = R_1 R_2$

$R_1$ AND $R_2$ $R_2$ AND $R_1$

(III) $R^* = \{\epsilon, R, RR, RRR, \ldots\}$

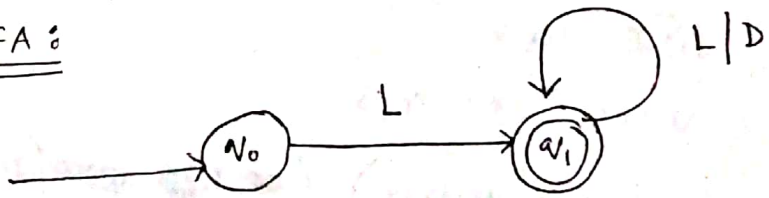
(IV) $(R_1 R_2)^* = \{\epsilon, R_1R_2, R_1R_2R_1R_2, R_1R_2R_1R_2R_1R_2, \ldots\}$

(V) $(R_1 | R_2)^* = \{\epsilon, R_1, R_2, R_1R_2, R_2R_1, R_1R_1R_2, R_1R_2R_2, R_2R_1R_1, \ldots R_1R_1R_1R_2, R_1R_2R_2R_2, R_1R_1R_2R_2, \ldots\}$

(VI) $R^+ = \{R, RR, RRR, \ldots\}$

### Regular expression for variable declaration

DFA :

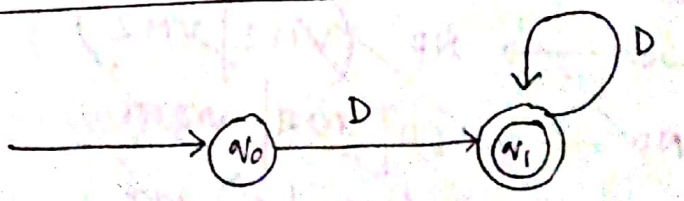



RE :

$$L(L|D)^*$$

### Regular expression for integer number

DFA




RE

$$DD^* = D^+$$

Scanned with CamScanner

Regular expression for unsigned decimal numbers:

10, 20, 10.5, 20.0, 30, 30.65,

RE = $D^+(\epsilon | .D^+)$

Regular expression for ~~all~~ ~~decimal~~ MSCSE ID:

RE = 012 (DD|00) (1|2|3) (DDD|000)

Regular expression for all decimal number:

RE = $(\epsilon | + | -)\left( D^+(\epsilon | .D^+) \middle| .D^+ \right)$

Grammar:

RE = (যে কোন অক্ষর বার)

যে বার

বহুবার বার

অথবা কিছু- বার

(বার (ঐ | কোন অক্ষর | ফাঁকা) বার)

(ঐ | ফাঁকা)

Grammar:

Sen → NP (VM1 | VM2)

NP → ID | বহুবচন | অথবা

VM1 → (AdjM | ϵ) বার

VM2 → (VM | ϵ) বার

AdjM → কোন অক্ষর, VM/→/AdjM

Scanned with CamScanner

Grammar :

S ⟶ NP [illegible] VM

VM ⟶ VM1 | VM2

NP ⟶ [illegible] | [illegible] | [illegible]

VM1 ⟶ AdjM [illegible]

AdjM ⟶ [illegible] | ∈

VM2 ⟶ OBM [illegible]

OBM ⟶ [illegible] | ∈

Grammar 2 :

আমি ভাত খাই  আমি খাই

আমরা ভাত খাই  আমরা খাই

আমি মাছ খাই

আমরা মাছ খাই

আমি ভাত মাছ খাই না

আমরা ভাত মাছ খাই না

RE := (আমি | আমরা) (ভাত | মাছ | ∈) খাই (না | ∈)

Grammar :

S ⟶ NP OBJ VP

NP ⟶ আমি | আমরা

OBJ ⟶ ভাত | মাছ | ∈

VP ⟶ খাই NEG

NEG ⟶ ∈ | না

Scanned with CamScanner

mple:

Grammar:

$E \rightarrow 2E2$

$E \rightarrow 3E3$

$E \rightarrow 4$

Perform shift Reduce Parsing for input string 32423

Parsing in NLP is the process of determining the syntactic structure of a text by analyzing its constituent words based on an underlying grammar (of the language)

| Input Buffer | Stack | Parsing Action |
|---|---|---|
| 32423$ | $ | — |
| 2423$ | $3 | shift |
| 423$ | $32 | shift |
| 23$ | $324 | shift |
| 3$ | $32E | Reduce by E→4 |
| 3$ | $32E2 | shift |
| $ | $3E | Reduce by E→2E2 |
| $ | $3E3 | shift |
| $ | $E | Reduce by E→3E3 |
| $ | $E | Accept |

Scanned with CamScanner

Example :

Grammar :

- start → FSUB OBJ FVERB | TSUB OBJ TVERB
- ~~SUB → FSUB | TSUB~~
- FSUB →
- TSUB →
- OBJ →
- FVERB →
- TVERB →

string :

| Input Buffer | stack | Parsing Action |
|---|---|---|
| $ | - $ | - |
| $ | $ | shift |
| $ | $ TSUB | Reduce by TSUB → |
| $ | $ TSUB | shift |
| $ | $ TSUB OBJ | Reduce by OBJ → |
| $ | $ TSUB OBJ | shift |
| $ | $ TSUB OBJ TVERB | Reduce by TVERB |
| $ | $ start | Reduce by start → TSUB OBJ TVERB |
| $ | $ start | Accept |

Scanned with CamScanner

rsing — Top down
— Bottom up

Grammar:

S ⟶ NP VP

NP ⟶ det n

det ⟶ {an, the}

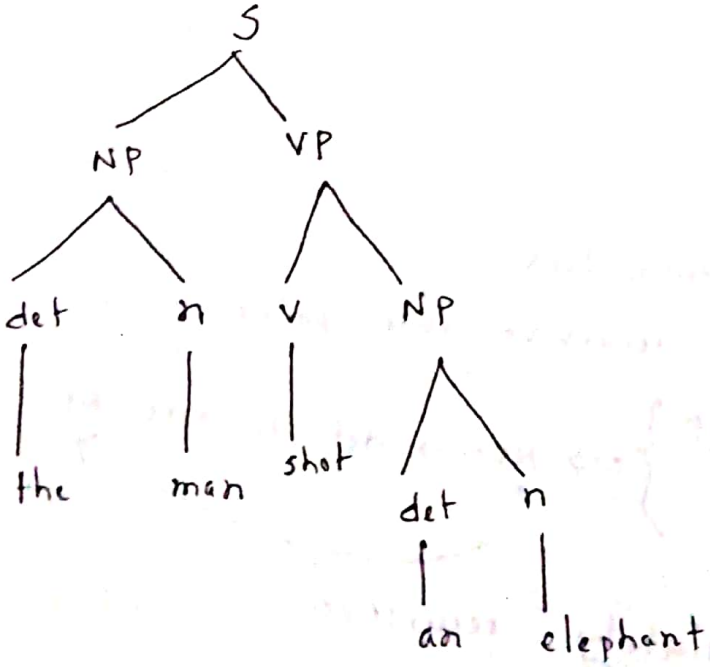
n ⟶ {elephant, man}

VP ⟶ V NP

V ⟶ shot

string: The man shot an elephant

Top-down




Lexicon:

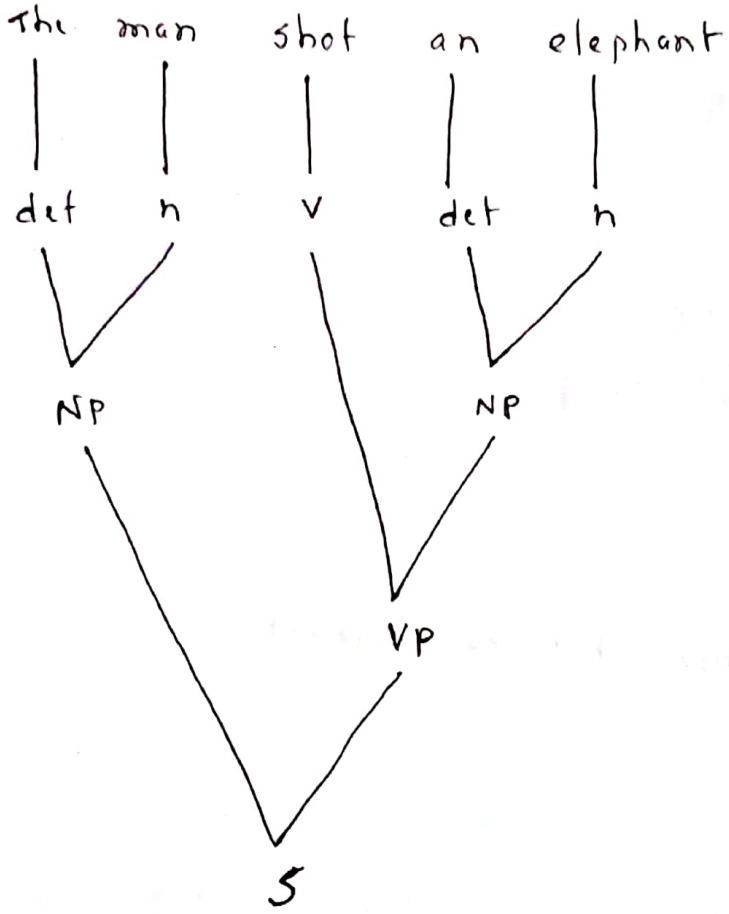
det ⟶ {an, the}

n ⟶ {elephant, man}

V ⟶ shot

Scanned with CamScanner

## Bottom up parsing




## Recursive Rules

- Recursive rules call themselves
- We already have some recursive rule pairs

NP → det n PP
PP → prep NP
⟹ NP → det n prep NP

- Rules can be immediately recursive

AdjG → adj AdjG

Scanned with CamScanner

## Recursive Rules

- Left recursive
- Right recursive

### Left recursive

AdjG ⟶ AdjG adj

AdjG ⟶ adj

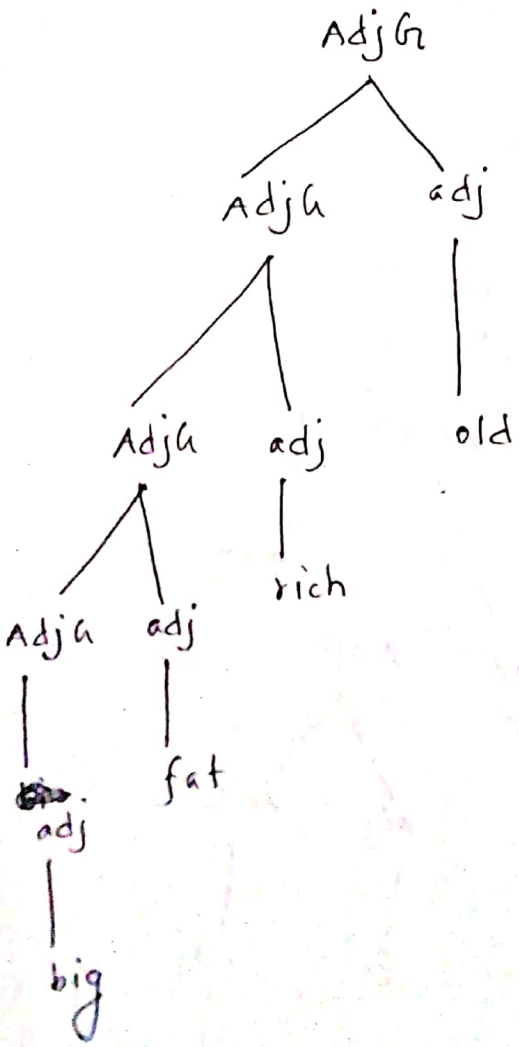
### Right Recursive

AdjG ⟶ adj AdjG

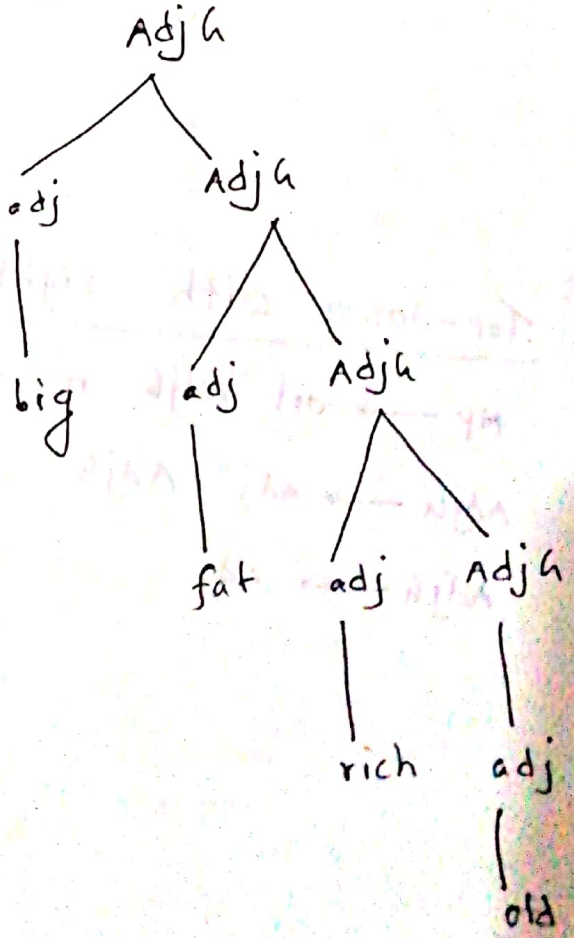
AdjG ⟶ adj

string: big fat rich old

### Left Recursion Tree




### Right Recursion Tree




Scanned with CamScanner

## Top-down with left recursion

Np ⟶ det Adjh n

Adjh ⟶ Adjh adj

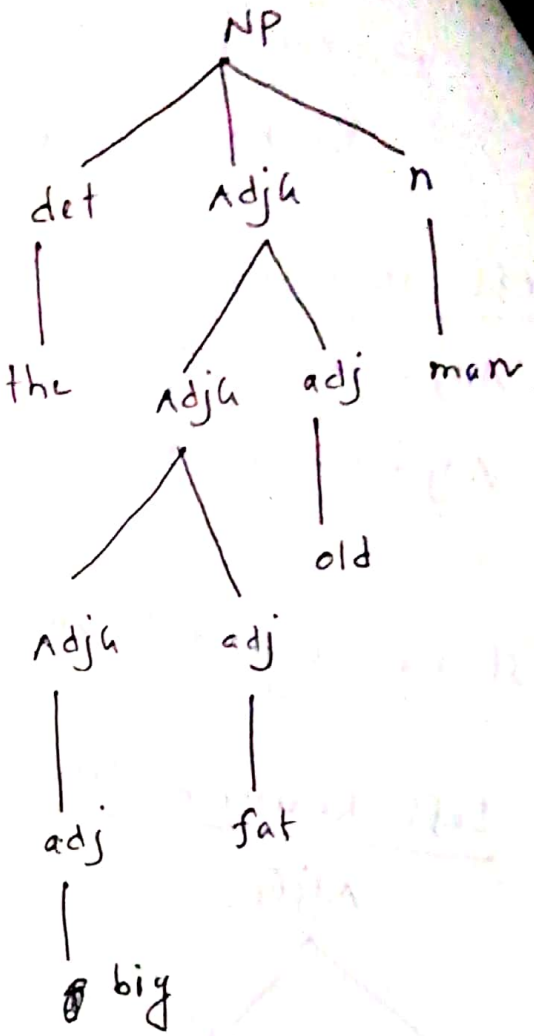
Adjh ⟶ adj

det ⟶ {an, the}

n ⟶ maw

adj ⟶ {big, fat, old}

string: the big fact rich old man

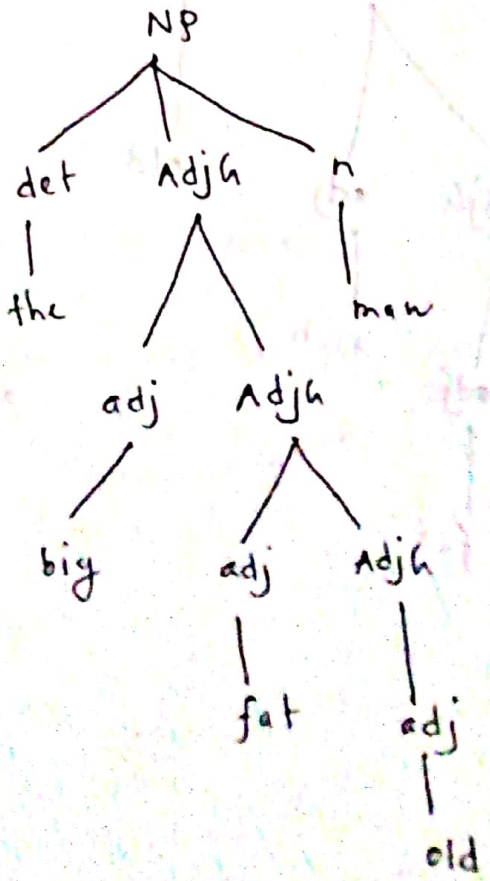



## Top-down with right recursion

Np ⟶ det Adjh n

Adjh ⟶ adj Adjh

Adjh ⟶ adj




Scanned with CamScanner

## Empty Rules

NP ⟶ det AdjG n

AdjG ⟶ adj AdjG

AdjG ⟶ ∈

### Equivalent to

NP ⟶ det AdjG n | NP ⟶ det n

AdjG ⟶ adj AdjG | AdjG ⟶ adj

∴ NP ⟶ det AdjG n

NP ⟶ det n

AdjG ⟶ adj AdjG

AdjG ⟶ adj

### Equivalent to

NP ⟶ det [AdjG] n  → optional

AdjG ⟶ adj [AdjG]  → optional

## Top-down with empty rules

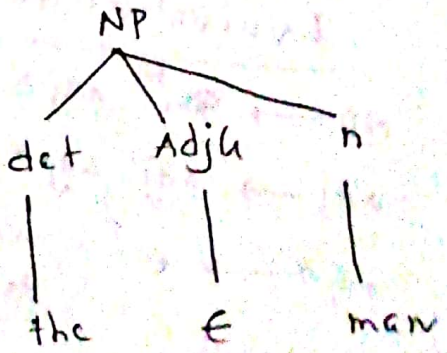
Grammar:

NP ⟶ det AdjG n

AdjG ⟶ adj AdjG

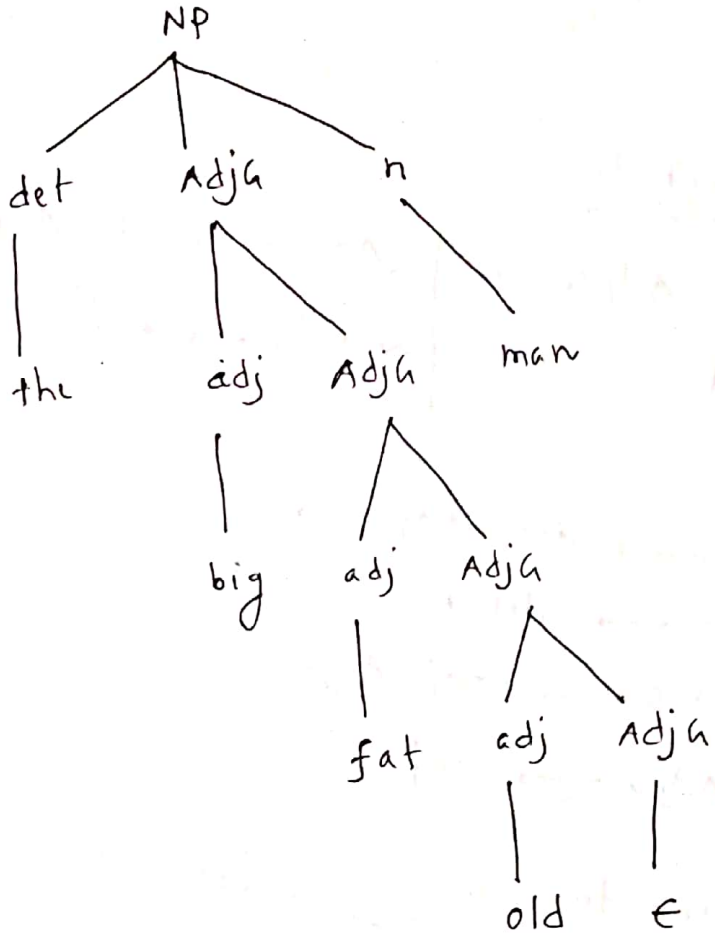
AdjG ⟶ ∈ , det → {the}
n → {man}

string: the man




Scanned with CamScanner

string:
the big fat old man




Kleene star :

$NP \longrightarrow$ the Adjc n

$Adjc \longrightarrow adj*$

Equivalent to

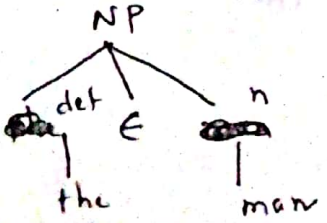
$NP \longrightarrow$ det $adj*$ n

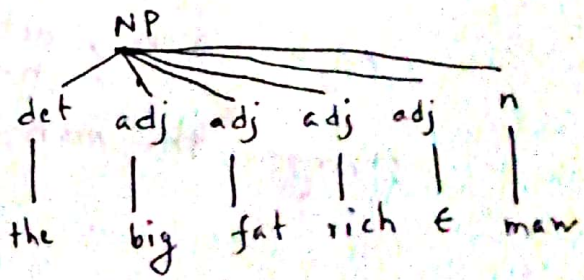
$adj \longrightarrow adj\ adj*$

$adj* \longrightarrow \epsilon$

$n \rightarrow \{man\}$
$det \rightarrow \{the\}$

$adj \rightarrow \{big, fat, rich\}$




Scanned with CamScanner

## Regular expression (RE) :

(I) $R_1 | R_2 = R_1 + R_2 = R_1 \text{ OR } R_2 = R_2 | R_1$

(II) $R_1 R_2 \neq R_2 R_1$ . $R_1 \cdot R_2 = R_1 R_2$

$R_1$ AND $R_2$ $R_2$ AND $R_1$

(III) $R^* = \{\epsilon, R, RR, RRR, \ldots\}$

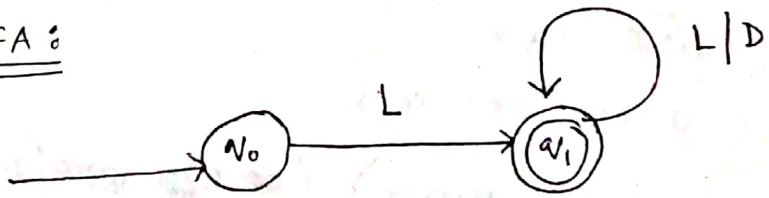
(IV) $(R_1 R_2)^* = \{\epsilon, R_1R_2, R_1R_2R_1R_2, R_1R_2R_1R_2R_1R_2, \ldots\}$

(V) $(R_1 | R_2)^* = \{\epsilon, R_1, R_2, R_1R_2, R_2R_1, R_1R_1R_2, R_1R_2R_2, R_2R_1R_1, \ldots R_1R_1R_1R_2, R_1R_2R_2R_2, R_1R_1R_2R_2, \ldots\}$

(VI) $R^+ = \{R, RR, RRR, \ldots\}$

### Regular expression for variable declaration

DFA :

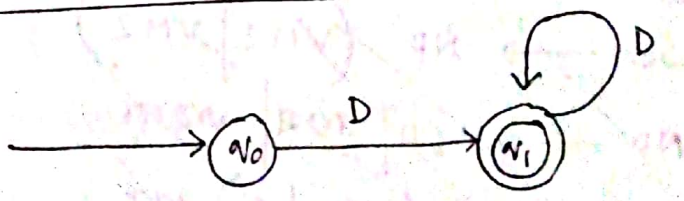



RE :

$$L(L|D)^*$$

### Regular expression for integer number

DFA




RE

$$DD^* = D^+$$

Scanned with CamScanner

Regular expression for unsigned decimal numbers:

10, 20, 10.5, 20.0, 30, 30.65,

RE = $D^+(\epsilon | .D^+)$

Regular expression for ~~all~~ ~~decimal~~ MSCSE ID:

RE = 012 (DD|00) (1|2|3) (DDD|000)

Regular expression for all decimal number:

RE = $(\epsilon | + | -)\left( D^+(\epsilon | .D^+) \middle| .D^+ \right)$

Grammar:

RE = (যে কোন অক্ষর বার)

যে বার

বহুবার বার

অথবা কিছু- বার

(বার (ঐ | কোন অক্ষর | ফাঁকা) বার)

(ঐ | ফাঁকা)

Grammar:

Sen → NP (VM1 | VM2)

NP → ID | বহুবচন | অথবা

VM1 → (AdjM | ϵ) বার

VM2 → (VM | ϵ) বার

AdjM → কোন অক্ষর, VM/→/AdjM

Scanned with CamScanner

Example :

Grammar :

- start → FSUB OBJ FVERB | TSUB OBJ TVERB
- ~~SUB → FSUB | TSUB~~
- FSUB →
- TSUB →
- OBJ →
- FVERB →
- TVERB →

string :

| Input Buffer | stack | Parsing Action |
|---|---|---|
| $ | - $ | - |
| $ | $ | shift |
| $ | $ TSUB | Reduce by TSUB → |
| $ | $ TSUB | shift |
| $ | $ TSUB OBJ | Reduce by OBJ → |
| $ | $ TSUB OBJ | shift |
| $ | $ TSUB OBJ TVERB | Reduce by TVERB |
| $ | $ start | Reduce by start → TSUB OBJ TVERB |
| $ | $ start | Accept |

Scanned with CamScanner

rsing — Top down
— Bottom up

Grammar:

S ⟶ NP VP

NP ⟶ det n

det ⟶ {an, the}

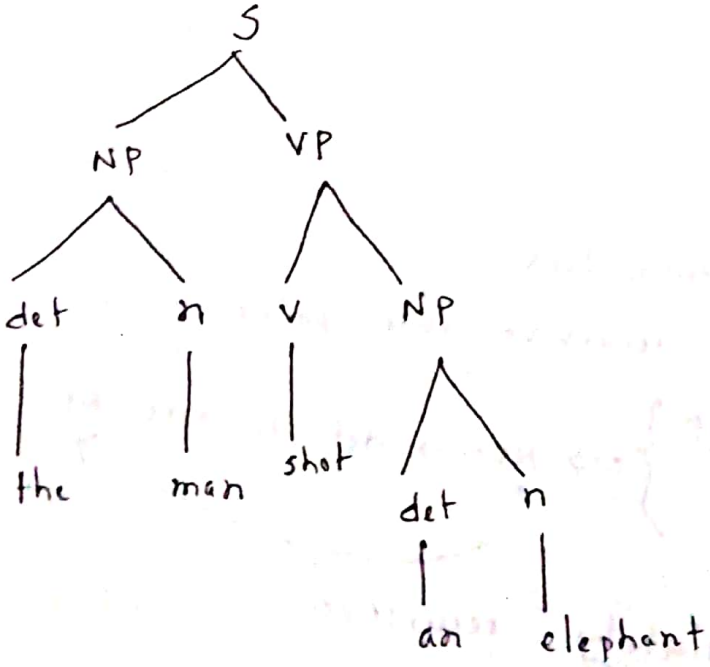
n ⟶ {elephant, man}

VP ⟶ V NP

V ⟶ shot

string: The man shot an elephant

Top-down




Lexicon:

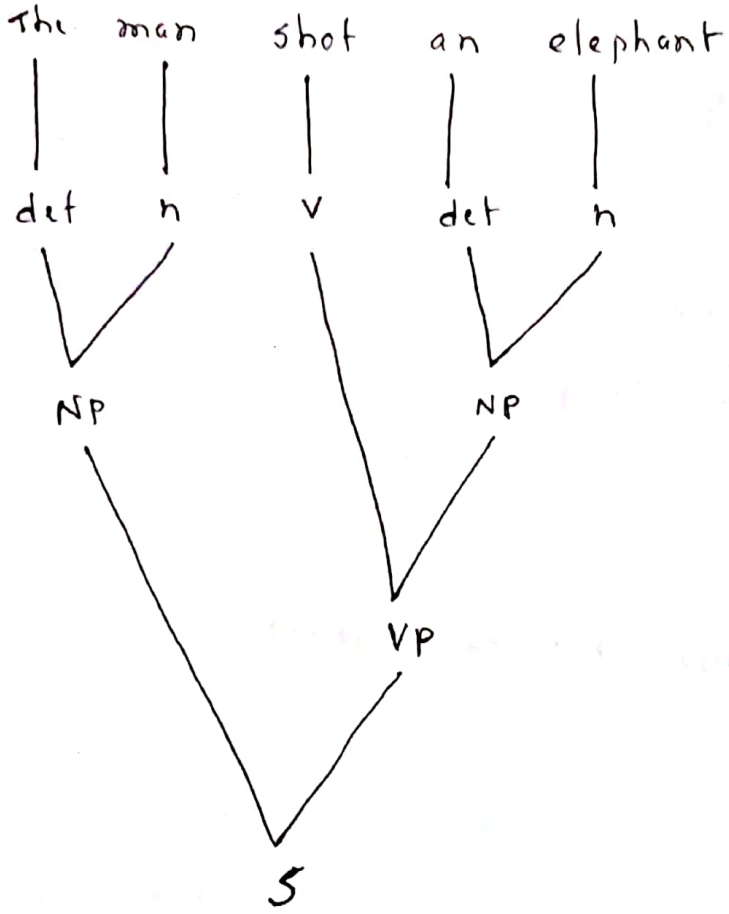
det ⟶ {an, the}

n ⟶ {elephant, man}

V ⟶ shot

Scanned with CamScanner

## Bottom up parsing




## Recursive Rules

- Recursive rules call themselves
- We already have some recursive rule pairs

NP → det n PP
PP → prep NP
⟹ NP → det n prep NP

- Rules can be immediately recursive

AdjG → adj AdjG

Scanned with CamScanner

## Recursive Rules

- Left recursive
- Right recursive

### Left recursive

AdjG ⟶ AdjG adj

AdjG ⟶ adj

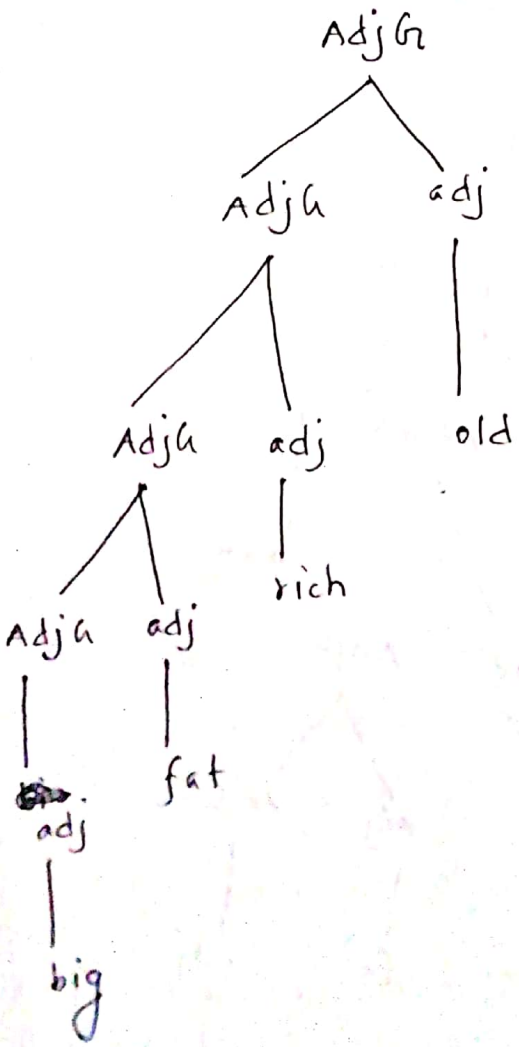
### Right Recursive

AdjG ⟶ adj AdjG

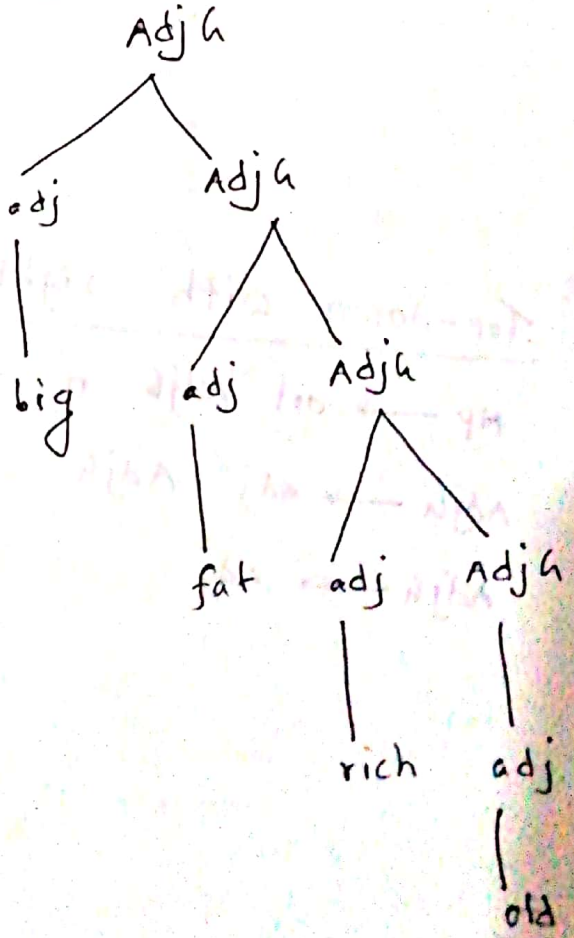
AdjG ⟶ adj

string: big fat rich old

### Left Recursion Tree




### Right Recursion Tree




Scanned with CamScanner

## Top-down with left recursion

Np ⟶ det Adjh n

Adjh ⟶ Adjh adj

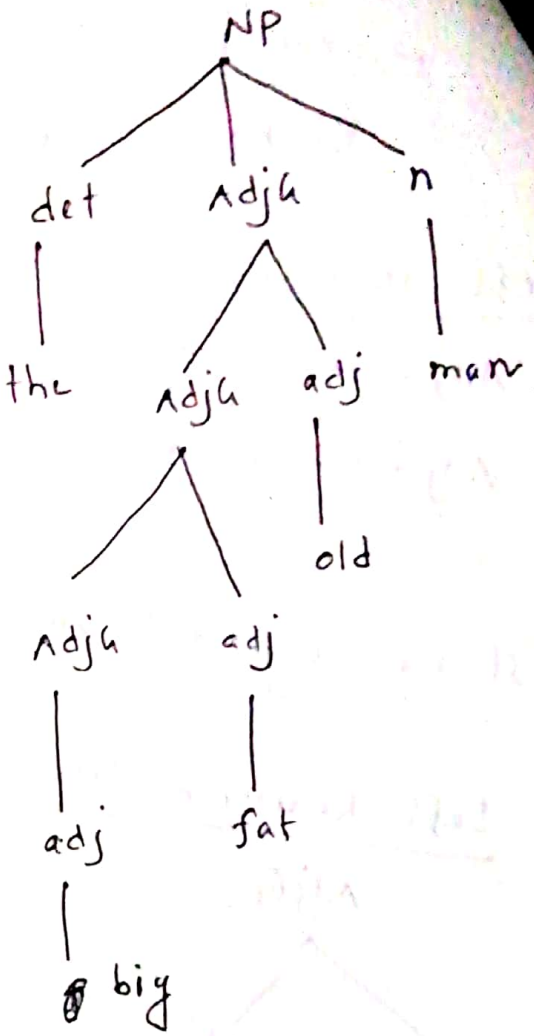
Adjh ⟶ adj

det ⟶ {an, the}

n ⟶ maw

adj ⟶ {big, fat, old}

string: the big fact rich old man

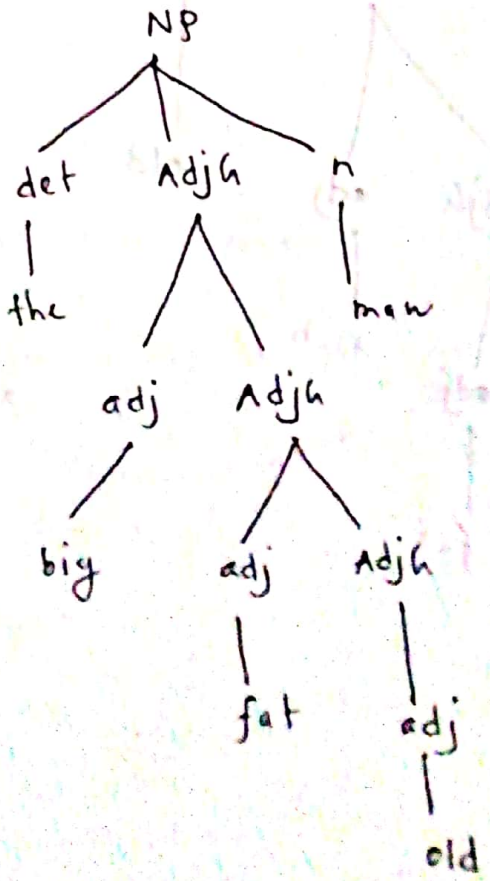



## Top-down with right recursion

Np ⟶ det Adjh n

Adjh ⟶ adj Adjh

Adjh ⟶ adj




Scanned with CamScanner

## Empty Rules

NP ⟶ det AdjG n

AdjG ⟶ adj AdjG

AdjG ⟶ ∈

### Equivalent to

NP ⟶ det AdjG n | NP ⟶ det n

AdjG ⟶ adj AdjG | AdjG ⟶ adj

∴ NP ⟶ det AdjG n

NP ⟶ det n

AdjG ⟶ adj AdjG

AdjG ⟶ adj

### Equivalent to

NP ⟶ det [AdjG] n  → optional

AdjG ⟶ adj [AdjG]  → optional

## Top-down with empty rules

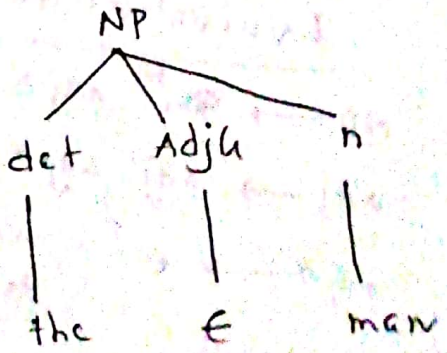
Grammar:

NP ⟶ det AdjG n

AdjG ⟶ adj AdjG

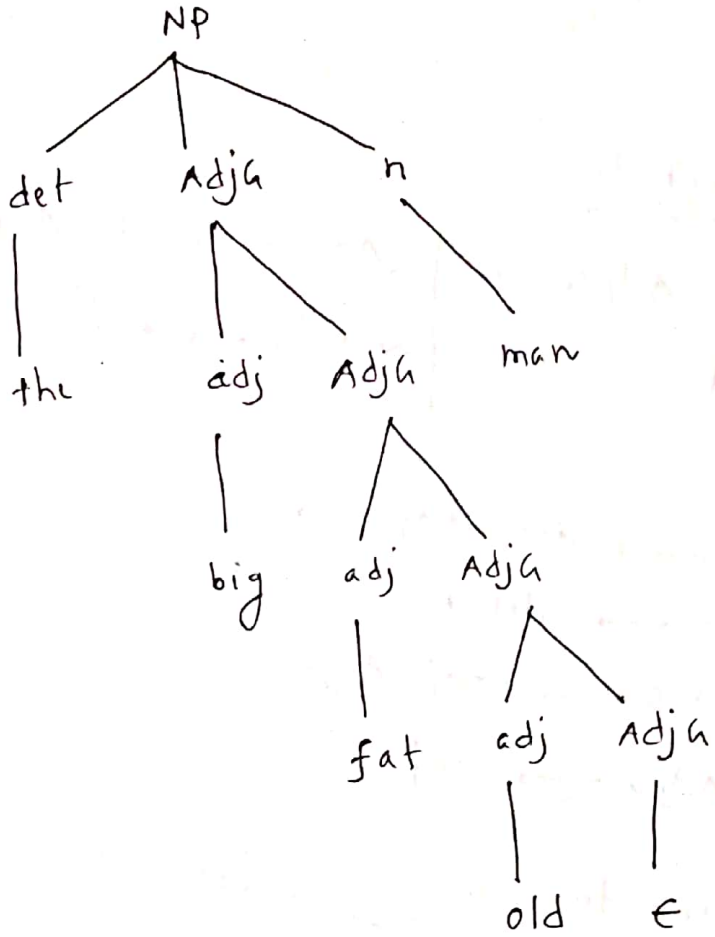
AdjG ⟶ ∈ , det → {the}
n → {man}

string: the man




Scanned with CamScanner

string:
the big fat old man




Kleene star :

$NP \longrightarrow$ the Adjc n

$Adjc \longrightarrow adj*$

Equivalent to

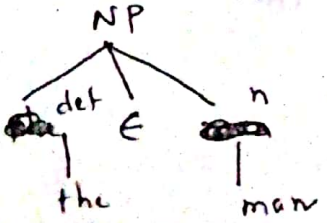
$NP \longrightarrow$ det $adj*$ n

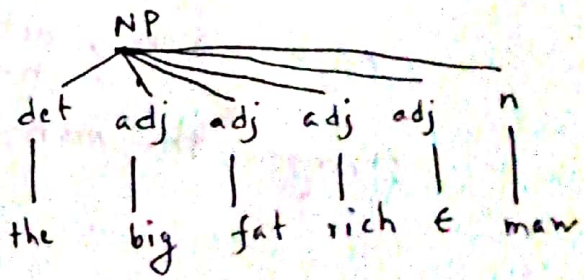
$adj \longrightarrow adj\ adj*$

$adj* \longrightarrow \epsilon$

$n \rightarrow \{man\}$
$det \rightarrow \{the\}$

$adj \rightarrow \{big, fat, rich\}$




Scanned with CamScanner

5 Grammar for Regular expression: (1)

$0^* 1 (0+1)^*$

Grammar:

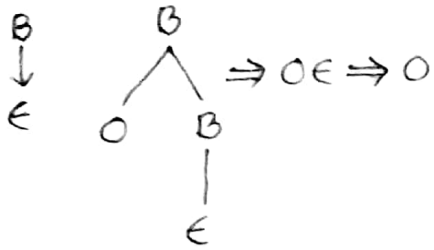
$S \longrightarrow A\ 1\ B$

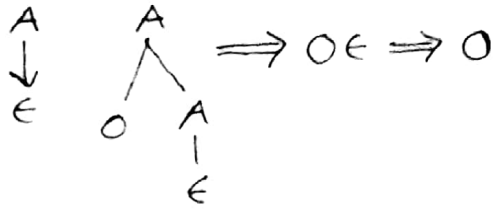
$A \longrightarrow \epsilon \mid 0A$

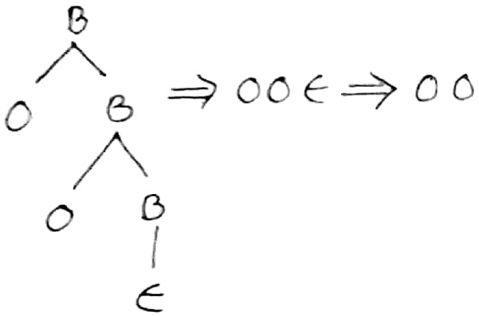
$B \longrightarrow \epsilon \mid 0B \mid 1B$

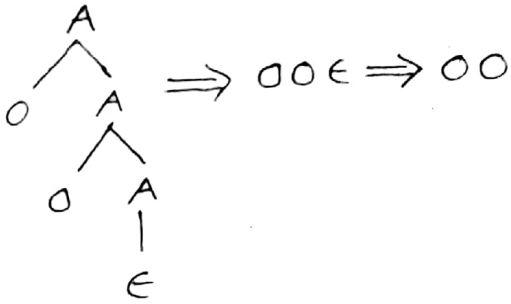
$A = 0^* = \{\epsilon, 0, 00, 000, 0000, \cdots\}$

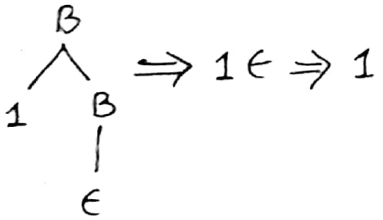
$B = (0+1)^* = \{\epsilon, 0, 1, 01, 10, 00, 11, 000, 111, \cdots\}$

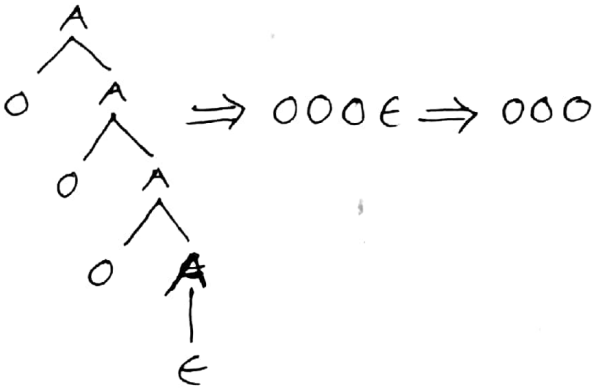


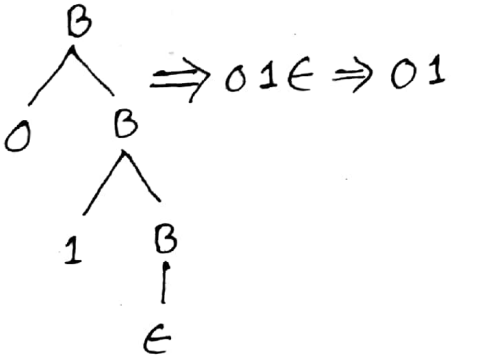



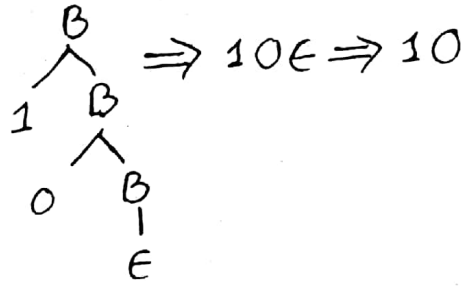




Scanned with CamScanner

$DD^* = D^+$ (Regular expression for positive integer numbers)

$DD^* = D\{\epsilon, D, DD, DDD, \ldots\}$

$= \{D\epsilon, DD, DDD, DDDD, \ldots\}$

$= \{D, DD, DDD, DDDD, \ldots\}$

$= D^+$

$D = 0 | 1 | 2 | \cdots | 9$

For example,

$D = 8$

$DD = 19$

$DDD = 100$

$DDDD = 1009$

Regular expression for unsigned decimal numbers:

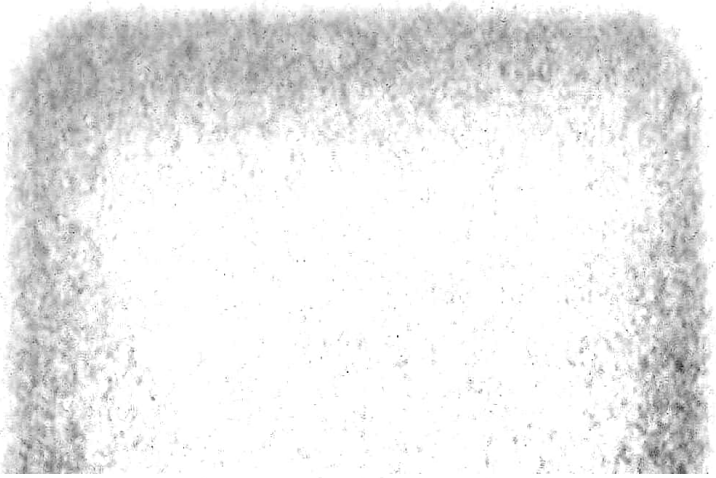
10, 20, 10.5, 20.0, 30, 30.65

$RE = D^+(\epsilon | .D^+)$

$= D^+\epsilon | D^+.D^+$ For integer only $D^+$

$= D^+ | D^+.D^+$ For fractional numbers $D^+.D^+$

Here, $D^+ = \{D, DD, DDD, DDDD, \ldots\}$



Scanned with CamScanner

Regular expression for MSCSE id:

RE = 012 DD (1|2|3) (DDD\000)

0 1 2 2 0 3 0 6 5 = 012203065

Regular expression for all decimal numbers:

Positive number (integer)

Negative number (integer)

Positive fractional number

Negative fractional number

$$RE = (\epsilon|+|-)\left(D^{+}(\epsilon|.D^{+})\,|\,.D^{+}\right)$$

$$= (\epsilon|+|-)\left((D^{+}|D^{+}.D^{+})\,|\,.D^{+}\right)$$

For example,

$-20 \longrightarrow -D^{+}$

$\longrightarrow -DD$

$\longrightarrow -20$

$+20.5 \longrightarrow +D^{+}.D^{+}$

$\longrightarrow +DD.D$

$\longrightarrow +20.5$

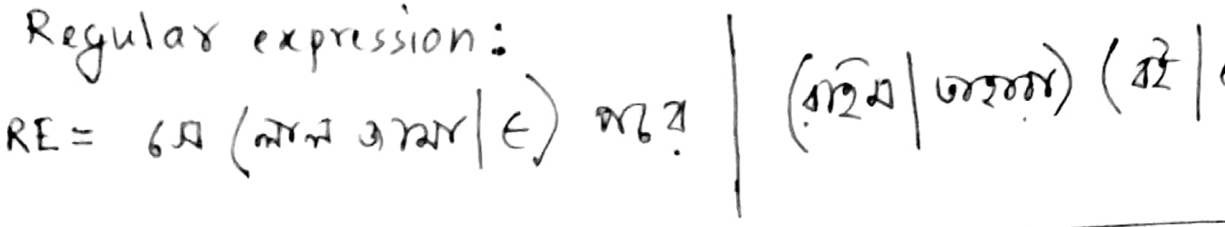
$-.3 \longrightarrow -.D^{+}$

$\longrightarrow -.D$

$\longrightarrow -.3$

Scanned with CamScanner

Regular expression:

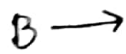
RE = 

Grammar:

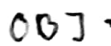
$S \longrightarrow A \mid B$

$A \longrightarrow NP\ ADJ\ V$

$ADJ \longrightarrow ADJ\ N \mid \epsilon$

$ADJ \longrightarrow$ 

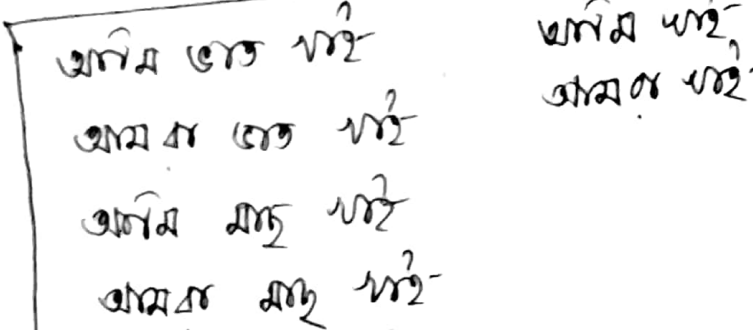
$N \longrightarrow$ 

$V \longrightarrow$ 

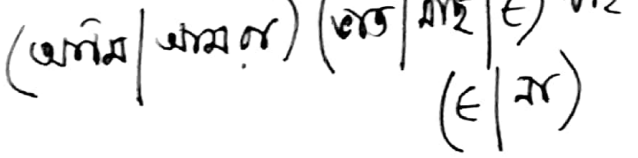
$B \longrightarrow NP'\ OBJ\ V'$

$OBJ \longrightarrow$ $\mid \epsilon$

$V' \longrightarrow$ 



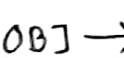
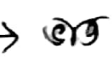
RE:



Grammar:

$S \longrightarrow NP\ OBJ\ VP$

$NP \longrightarrow$ $\mid$

$OBJ \longrightarrow$  $\mid$  $\mid \epsilon$

$VP \longrightarrow$ $NEG$

$NEG \longrightarrow \epsilon \mid$ 

Scanned with CamScanner

## Shift Reduce Parser:

Grammar:

$E \rightarrow 2E2$

$E \rightarrow 3E3$

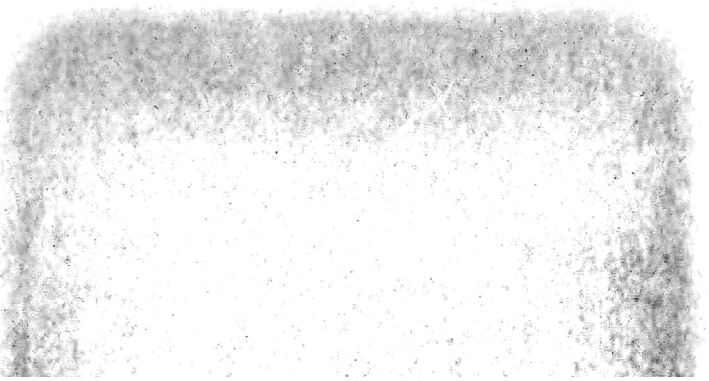
$E \rightarrow 4$

Perform shift Reduce Parsing for input string

32423

Ans:

| Input Buffer | stack | Parsing Action |
|---|---|---|
| 32423$ | $ | — |
| 2423$ | $3 | shift |
| 423$ | $32 | shift |
| 23$ | $324 | shift |
| 23$ | $3E | Reduce by E→4 |
| 3$ | $3E2 | shift |
| 3$ | $E | Reduce by E→2E2 |
| $ | $E3 | shift |
| $ | E | Reduce by E→3E3 |
| $ | $E | Accept |



Scanned with CamScanner

Dr. Mohammad Nurul Huda
CSE 6015, SMT
Lecture sheet – 8

# Language Modeling using N-grams

Assigning a probability to each unit of text. In SMT, a unit of text is a sentence.

Given a sentence e
our task is to compute P(e)

A sentence e containing the word sequence $W_1 W_2 \ldots W_n$

$e \Rightarrow W_1 W_2 \ldots W_n$

$$P(e) = P(W_1 W_2 \ldots W_n)$$

$$= P(W_1)\, P(W_2|W_1)\, P(W_3|W_1 W_2) \ldots P(W_n|W_1 W_2 \ldots W_{n-1})$$

Example:

[illegible]
আমি ভাত খাই।

$$P(\text{আমি ভাত খাই}) = P(\text{আমি}) * P(\text{ভাত}|\text{আমি}) * P(\text{খাই}|\text{আমি ভাত})$$

$P(W_n|W_1 W_2 \ldots W_{n-1})$ calculation:

Data sparsity problem

## The N-gram approximation:

N-gram model:

The probability of a word given all previous words is approximated by the probability of the word given the previous N−1 words.

| | |
|---|---|
| N = 1 | unigram model |
| N = 2 | Bigram model |
| N = 3 | Trigram model |

Scanned with CamScanner

## Unigram Model :



Corpus

| |
|---|
| আমি ভাত খাই- |
| সে ভাত খায় |
| তুমি ভাত খাও |
| আমরা ভাত খাই- |

$$P(W_i) = \frac{count(W_i)}{count(W)}$$

Count ($W_i$) = Total number of $W_i$ words

Count ($W$) = Total number of any words.

$$P(\text{ভাত}) = \frac{count(\text{ভাত})}{count(\text{all words})} \quad \ldots \text{eq(1)}$$

count (ভাত) = 4

count (all words including ভাত)

= আমি + ভাত + খাই- + সে + খায় + তুমি + খাও + আমরা

= 1 + 4 + 1 + 1 + 1 + 1 + 1 + 1

= 12

From eq (1) $\Rightarrow$

$$P(\text{ভাত}) = \frac{count(\text{ভাত})}{count(\text{all words including ভাত})}$$

$$= \frac{4^{1}}{12_{3}}$$

$$= \frac{1}{3}$$

$$= 0.333$$

Scanned with CamScanner

Model :

(3)

$$P(w_n | w_{n-1}) = \frac{count(w_{n-1} w_n)}{\sum_{w} count(w_{n-1} w)}$$

Here, $count(w_{n-1} w)$ denotes the number of occurrences of the sequence $w_{n-1} w_n$.

$count(w_{n-1} w)$ denotes the number of times $w_{n-1}$ occurs before any word $w$.

So, $count(w_{n-1} w) = count(w_{n-1})$

$$P(w_n | w_{n-1}) = \frac{count(w_{n-1} w_n)}{count(w_{n-1})} \quad \ldots$$

আমি ভাত খাই
তুমি ভাত খাও
তুমি ভাত খাও
আমরা ভাত খাই
তারা ভাত খায়

$$P(\text{খাই} | \text{ভাত}) = \frac{count(\text{ভাত খাই})}{count(\text{ভাত})} \quad \ldots \text{eq (2)}$$

$count(\text{ভাত খাই}) = 2$

$count(\text{ভাত}) = 5$

From eq (2) $\Rightarrow$

$$P(\text{খাই} | \text{ভাত}) = \frac{count(\text{ভাত খাই})}{count(\text{ভাত})} = \frac{2}{5} = 0.4$$

Scanned with CamScanner

Trigram Model :

$$P(w_n | w_{n-2} w_{n-1}) = \frac{count(w_{n-2} w_{n-1} w_n)}{\sum_w count(w_{n-2} w_{n-1} w)}$$

Since, $\sum_w count(w_{n-2} w_{n-1} w) = count(w_{n-2} w_{n-1})$

So, $P(w_n | w_{n-2} w_{n-1}) = \frac{count(w_{n-2} w_{n-1} w_n)}{count(w_{n-2} w_{n-1})}$

corpus

আমাদের স্কুলে যাওয়া উচিত

আমাদের স্কুলে যাওয়া উচিত হয় নাই

আমাদের স্কুলে যাওয়া ঠিক নই।

কখন স্কুলে যাওয়া উচিত।

আমাদের স্কুলে যাওয়ার সিদ্ধান্ত ঠিক ছিল

$$P(\text{উচিত} | \text{স্কুলে যাওয়া}) = \frac{count(\text{স্কুলে যাওয়া উচিত})}{count(\text{স্কুলে যাওয়া})} \quad \ldots\ldots \text{eq}^n (3)$$

count(স্কুলে যাওয়া উচিত) = 3

count(স্কুলে যাওয়া) = 4

From eq$^n$ (3) $\Rightarrow$

$$P(\text{উচিত} | \text{স্কুলে যাওয়া}) = \frac{count(\text{স্কুলে যাওয়া উচিত})}{count(\text{স্কুলে যাওয়া})} = \frac{3}{4}$$

Scanned with CamScanner

| Bigram | Probability |
|---|---|
| START all | 0.16 |
| all men | 0.09 |
| men are | 0.24 |
| are equal | 0.08 |

Find P(all men are equal). After that find P(all animals are equal).

Ans:

$$P(\text{all men are equal}) = P(\text{all} \mid \text{START}) * P(\text{men} \mid \text{all}) * P(\text{are} \mid \text{men}) * P(\text{equal} \mid \text{are})$$

$$= 0.16 * 0.09 * 0.24 * 0.08$$

$$= ?$$

$$P(\text{all animals are equal}) = P(\text{all} \mid \text{START}) * P(\text{animals} \mid \text{all}) * P(\text{are} \mid \text{animals}) * P(\text{equal} \mid \text{are})$$

(if zero)

$$= 0$$

If P(animals|all) is not found in corpus, P(all animals are equal) will be zero.

Problem arises

Scanned with CamScanner

The task of the language model is not just to give probabilities to those sequences that are present in the corpus, but also to make reasonable estimates when faced with a new sequence.

## Smoothing:

The task of distributing some of the probability to unseen or zero-probability sequences is called smoothing.

## Algorithm:

Add one to all possible sequences' count
- that occur in corpus.
- those that do not occur.

$$p^*(w_n | w_{n-1}) = \frac{count(w_{n-1} w_n) + 1}{count(w_{n-1}) + V}$$

where $V$ = The number of word types in the vocabulary.

| Corpus | |
|---|---|
| আমি ভাত খাই<br>সে ভাত খায়<br>তুমি ভাত খাও<br>আমরা ভাত খাই<br>তোমরা ভাত খাও | সে ভাত খাইতেছে |

$P(\text{সে ভাত খাইতেছে}) = P(\text{সে} | \text{START}) * P(\text{ভাত} | \text{সে}) * P(\text{খাইতেছে} | \text{ভাত})$ ← if zero

$= 0$

Scanned with CamScanner

P(আমি | START) = ⟶ count = 1+1

P(ভাত | আমি) = ⟶ count = 1+1

P(খাই | ভাত) = ⟶ count = 2+1

P(চা | START) = ⟶ count = 2+1

P(ভাত | চা) = ⟶ count = 2+1

P(খাব | ভাত) = ⟶ count = 2+1

P(তুমি | START) = ⟶ count = 1+1

P(ভাত | তুমি) = ⟶ count = 1+1

P(খাও | ভাত) = ⟶ count = 1+1

P(আমরা | START) = ⟶ count = 1+1

P(ভাত | আমরা) = ⟶ count = 1+1

P(খাই | ভাত) = ⟶ ~~count~~

P(আপনারা | START) = ⟶ count = 1+1

P(ভাত | আপনারা) = ⟶ count = 1+1

P(খাব | ভাত) =

P(চা | START) =

P(চা | চা) =

P(খাইনাই | ভাত) = ⟶ count = 0+1

$$P(\text{খাইনাই} \mid \text{ভাত}) = \frac{\text{Count}(\text{ভাত খাইনাই}) + 1}{\text{count}(\text{ভাত}) + V}$$

$$= \frac{0+1}{6+14} = \frac{1}{20}$$

$$= 0.05$$

Scanned with CamScanner

# Feature Extraction in NLP



## Feature extraction Method

- Bag-of-words
- TF-IDF

## Bag-of-words

- Feature extraction method from text for use in modeling such as with machine learning algorithms.
- A bag-of-words is a representation of text that describes the occurrence of words within a document.
- It involves two things
  - A vocabulary of known words
  - A measure of the presence of known words

## Example of the Bag-of-Words model

- First few lines of text from the book "A tale of Two cities" by Charles Dickens

doc1 ← • It was the best of times,
doc2 ← • it was the worst of times,
doc3 ← • it was the age of wisdom,
doc4 ← • it was the age of foolishness

} Data collection step1

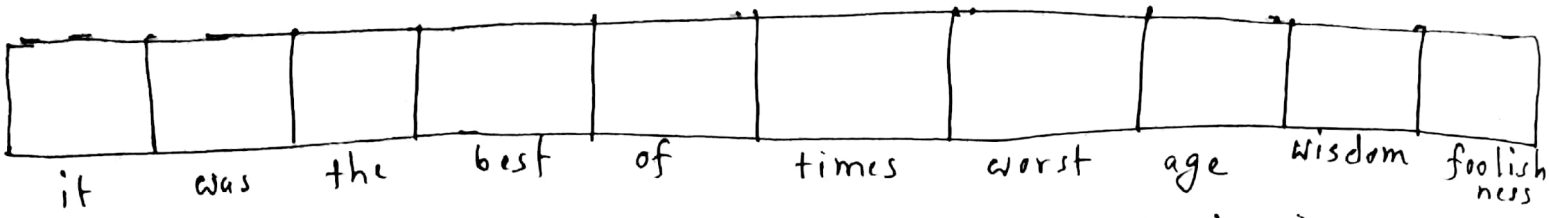
- Let's treat each line as a separate "document" and the 4 lines as our entire corpus of documents
- Make a list of all of the words in our model vocabulary.

step2 Design vocabulary

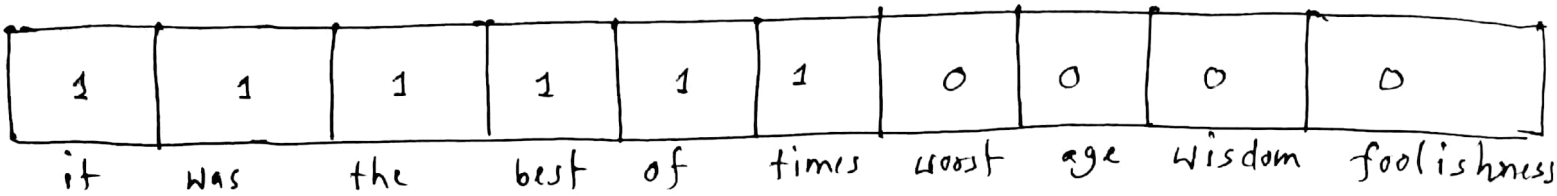
- The unique words here (ignoring case and punctuation) are:
  - it • was • the • best • of • times • worst • age • wisdom • foolishness → 10 words from a corpus containing 24 words

Scanned with CamScanner

- step3: To score the words in each document
(To mark the presence of words as a boolean value, 0 for absent, 1 for present)




Now scoring for document-1 (it was the best of times)




Vector will be [1, 1, 1, 1, 1, 1, 0, 0, 0, 0]

↓

Machine Learning tool
- Neural Network
- Support vector Machine
- Decision Tree
- Nearest Neighbor
- Hidden Markov Model
- k-Nearest Neighbor
- Logistic Regression
- ⋮

Scoring Words

counts → Count the number of times each words in a document

Frequency → each word appears in a document out of all the words in the document

Scanned with CamScanner

## Managing Vocabulary

- To reduce the vector size

Text cleaning Technique in first step

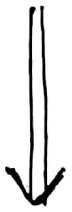
- Ignoring case
- Ignoring punctuation
- Ignore stop words like "a", 'of", etc
- Fixing misspelled words
- Reducing words to their stem (play from "playing") using stemming algorithms.

## Modified Bag-of-Words Method

- To create a vocabulary of grouped words (Bigram, Trigram)
- Only the bigrams that appear in the corpus are modeled, not all possible bigrams

For example,

"It was the best of times"



Bigram generation

it was, was the, the best, best of, of times

## Word Hashing (Hash trick/ feature hashing)

CAT, DOG

Draw a Hash Table using the given two words where table size is 4.

Scanned with CamScanner

| Words | Bit Representation | Hash Index Generation |
|---|---|---|
| CAT | 00010 00000 10011 = 2067 | 2067 mod 4 = 3 |
| DOG | 00011 01110 00110 = 3526 | 3526 mod 4 = 2 |

A → 00000 → 0
B → 00001 → 1
C → 00010 → 2
D → 00011 → 3
⋮
Z → 11001 → 25

English. Total 26 characters
Total encoded bit
$= \lceil \log_2 26 \rceil$
= 5 bits

CAT bit stream: 00010 00000 10011

$= 0*2^{14} + 0*2^{13} + 0*2^{12} + \underline{1*2^{11}} + 0*2^{10} + \cdots + \underline{1*2^{4}} + 0*2^{3} + 0*2^{2} + \underline{1*2^{1}} + \underline{1*2^{0}}$

$= 2048 + 16 + 2 + 1$

$= 2067$

Hash Table

| | |
|---|---|
| | 0 |
| | 1 |
| DOG | 2 |
| CAT | 3 |

Vector

| | |
|---|---|
| 0 | 0 |
| 0 | 1 |
| 1 | 2 |
| 1 | 3 |

DOG bit stream

$= 1*2^{11} + 1*2^{10} + 1*2^{8} + 1*2^{7} + 1*2^{6} + 1*2^{2} + 1*2^{1}$

$= 2048 + 1024 + 256 + 128 + 64 + 4 + 2$

$= 3526$

Scanned with CamScanner

# TF-IDF

TF ⟶ Term frequency

IDF ⟶ Inverse Document Frequency -

## TF: Term Frequency

- Frequency of the word in the current document

## IDF: Inverse Document Frequency

- How reare the word is across documents

## Formula

$$tf(t,d) = \frac{f_{t,d}}{\text{number of words in } d}$$

$$idf(t,D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

$$tf\text{-}idf(t,d,D) = tf(t,d) * idf(t,D)$$

## Example 1 considering Word "this"

Document 1

| Term | Term count |
|---|---|
| this | 1 |
| is | 1 |
| a | 2 |
| sample | 1 |

Document 2

| Term | Term Count |
|---|---|
| this | 1 |
| is | 1 |
| another | 2 |
| example | 3 |

Scanned with CamScanner

$$tf("this", d_1) = \frac{1}{1+1+2+1} = \frac{1}{5} = 0.2$$

$$tf("this", d_2) = \frac{1}{1+1+2+3} = \frac{1}{7} = 0.14$$

$$idf("this", D) = \log\left(\frac{\text{Total no. of documents}}{\text{Total no. of documents that contain term}}\right)$$

$$= \log_{10}\left(\frac{Doc1 + Doc2}{Doc1 + Doc2}\right)$$

$$= \log_{10}\left(\frac{2}{2}\right) = 0$$

So, $tf\text{-}idf("this", d_1, D) = 0.2 * 0 = 0$

$tf\text{-}idf("this", d_2, D) = 0.14 * 0 = 0$

Since tf-idf is zero for the word "this", which implies that the word is not very informative as it appears in all documents.

Example-2 : considering word "example"

$$tf("example", d_1) = \frac{0}{5} = 0$$

$$tf("example", d_2) = \frac{3}{7} = 0.429$$

$$idf("example", D) = \log_{10}\left(\frac{Doc1 + Doc2}{Doc2}\right) = \log_{10}\left(\frac{2}{1}\right) = 0.301$$

Finally,

$$tf\text{-}idf("example", d_1, D) = tf("example", d_1) * idf("example", D)$$
$$= 0 * 0.301 = 0$$

$$tf\text{-}idf("example", d_2, D) = tf("example", d_2) * idf("example", D)$$
$$= 0.429 * 0.301 \approx 0.129$$

Scanned with CamScanner

# Similarity Measure



| | T1 | T2 | T3 | T4 | T5 | T6 | T7 | T8 |
|---|---|---|---|---|---|---|---|---|
| DOC1 | 0 | 4 | 0 | 0 | 0 | 2 | 1 | 3 |
| DOC2 | 3 | 1 | 4 | 3 | 1 | 2 | 0 | 1 |
| DOC3 | 3 | 0 | 0 | 0 | 3 | 0 | 3 | 0 |
| DOC4 | 0 | 1 | 0 | 3 | 0 | 0 | 2 | 0 |
| DOC5 | 2 | 2 | 2 | 3 | 1 | 4 | 0 | 2 |

Find similarity index using DOC2, DOC4 for

i) Cosine Similarity Measure

~~ii) Jaccard Similarity~~

~~iii) Pearson's correlation~~

## Cosine Similarity Measure

$$\text{Dot_Product}(DOC2, DOC4) = \langle 3,1,4,3,1,2,0,1 \rangle * \langle 0,1,0,3,0,0,2,0 \rangle$$

$$= 3*0 + 1*1 + 4*0 + 3*3 + 1*0 + 2*0 + 0*2 + 1*0 = 10$$

$$\text{Norm}(DOC2) = \sqrt{3^2+1^2+4^2+3^2+1^2+2^2+0^2+1^2} = 6.4$$

$$\text{Norm}(DOC4) = \sqrt{0^2+1^2+0^2+3^2+0^2+0^2+2^2+0^2} = 3.74$$

$$\text{Cosine}(DOC2, DOC4) = \frac{\text{Dot_Product}(DOC2, DOC4)}{\text{Norm}(DOC2) * \text{Norm}(DOC4)}$$

$$= \frac{10}{6.4 * 3.74} = 0.42$$

So, DOC2 and DOC4 are 42% similar

Scanned with CamScanner

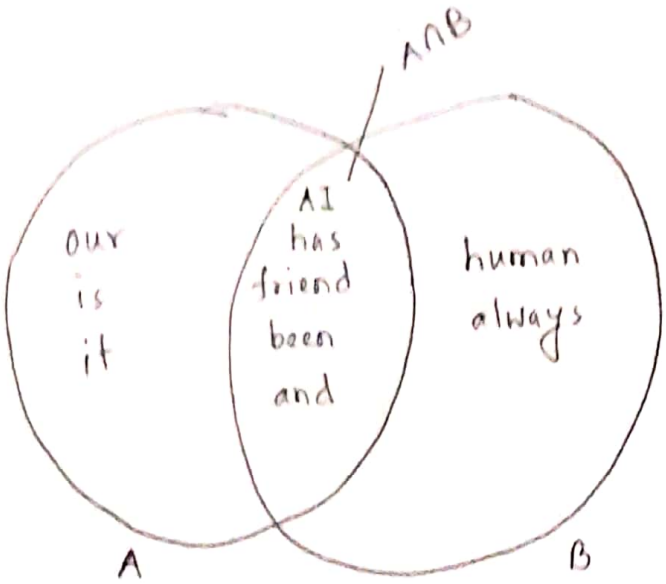
## Jaccard Similarity

Ques: Sentence-I : AI is our friend and it has been friendly

Sentence-II : AI and humans have always been friendly

Find Jaccard similarity Index for sentence-I and sentence-II

Ans:




Lemmatization: (Reduce words to the same root word)

Here, friend, friendly → friend

has, have → has

$$J(\text{Sentence 1, Sentence 2}) = \frac{|A \cap B|}{|A \cup B|}$$

$$= \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

$$= \frac{5}{8+7-5}$$

$$= \frac{5}{10}$$

$$= 0.5$$

Scanned with CamScanner

## ...on's correlation

$$P(x,y) = \frac{S_{xy}}{S_x \cdot S_y}$$

$$= \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n} (x_i - \bar{x})^2} * \sqrt{\sum_{i=1}^{n} (y_i - \bar{y})^2}}$$

$$\bar{x} = \text{mean}(x) = \frac{1}{n} \sum_{i=1}^{n} x_i$$

$$\bar{y} = \text{mean}(y) = \frac{1}{n} \sum_{i=1}^{n} y_i$$

n is the number of attributes in x and y

**Ques:**

DOC2 = ⟨3, 1, 4, 3, 1, 2, 0, 1⟩

DOC4 = ⟨0, 1, 0, 3, 0, 0, 2, 0⟩

Find pearson correlation index

**Ans:**

$x = \langle 3, 1, 4, 3, 1, 2, 0, 1 \rangle$

$y = \langle 0, 1, 0, 3, 0, 0, 2, 0 \rangle$

$$\therefore \bar{x} = \frac{3+1+4+3+1+2+0+1}{8} = \frac{15}{8} = 1.875$$

$$\therefore \bar{y} = \frac{0+1+0+3+0+0+2+0}{8} = 0.75$$

$$\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) = (3-1.875) * (0-0.75) + (1-1.875) * (1-0.75) + (4-1.875) * (0-0.75) + (3-1.875) * (3-0.75) + (1-1.875) * (0-0.75) + (2-1.875) * (0-0.75) + (0-1.875) * (2-0.75) +$$

Scanned with CamScanner

$= (1-1.875) \times (0-0.75)$

$= ?$

$$\sqrt{\sum_{i=1}^{n}(x_i-\bar{x})^2} = \sqrt{(3-1.875)^2+(1-1.875)^2+(4-1.875)^2+(3-1.875)^2+(1-1.875)^2+(2-1.875)^2+(0-1.875)^2+(1-1.875)^2}$$

$= ?$

$$\sqrt{\sum_{i=1}^{n}(y_i-\bar{y})^2} = \sqrt{(0-0.75)^2+(1-0.75)^2+(0-0.75)^2+(3-0.75)^2+(0-0.75)^2+(0-0.75)^2+(2-0.75)^2+(0-0.75)^2}$$

$= ?$

$$\therefore p(x,y) = \frac{\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i-\bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i-\bar{y})^2}}$$

$= ?$

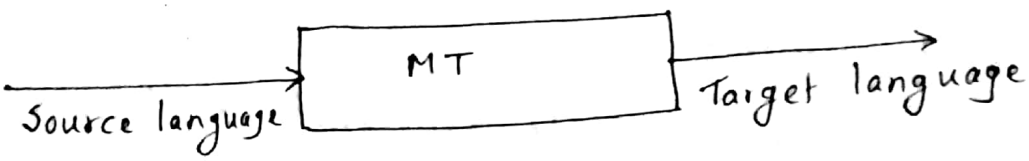
Scanned with CamScanner

Dr. Mohammad Nurul Huda
Statistical Machine Trans
Lecture sheet - I



## Machine Translation (MT):

MT can be defined as the use of computers to automate some or all of the process of translating from one language to another.




## MT uses ideas and techniques

- Linguistics
- Computer Science
- Artificial Intelligence (AI)
- Translation theory
- Statistics

## Commercially Interesting:

- US has invested in MT for intelligence purposes.
- EU spends more than €1,000,000,000 on translation costs each year.
- MT is used in UNL (Universal Network Language)
- MT is used to translate technical documents, reports, instruction manuals etc.

## Objective:

- The goal is not fluent translation
- Only correct and understandable output.



Scanned with CamScanner

Various approaches to Machine Translation:

- The transfer approach
- The Interlingua approach
- The direct approach
- Corpus-based approach
- Word-for-word translation
- statistical translation

## Word-for-word translation:

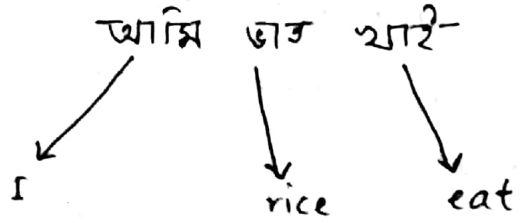
- Use a machine-readable bilingual dictionary to translate each word in a text.

Dictionary:

| Bangla | English |
|---|---|
| আমি | I |
| ভাত | rice |
| খাই- | eat |
| . | . |
| . | . |
| . | . |

Bangla: আমি ভাত খাই-

English: ?




English: I rice eat

Advantage:

Easy to implement, results give a rough idea about what the text is about.

Scanned with CamScanner

vantage:

Problems with word order means that this results in low-quality translation.

## The Transfer Approach:

The transfer model involves three stages

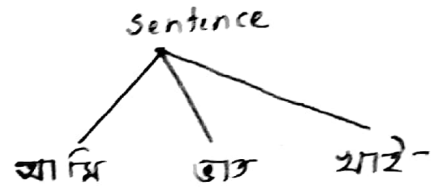
- Analysis
- transfer
- generation

### Analysis:

The source language sentence is parsed, and the sentence structure and the constituents of the sentence are identified.

Example:

আমি ভাত খাই-

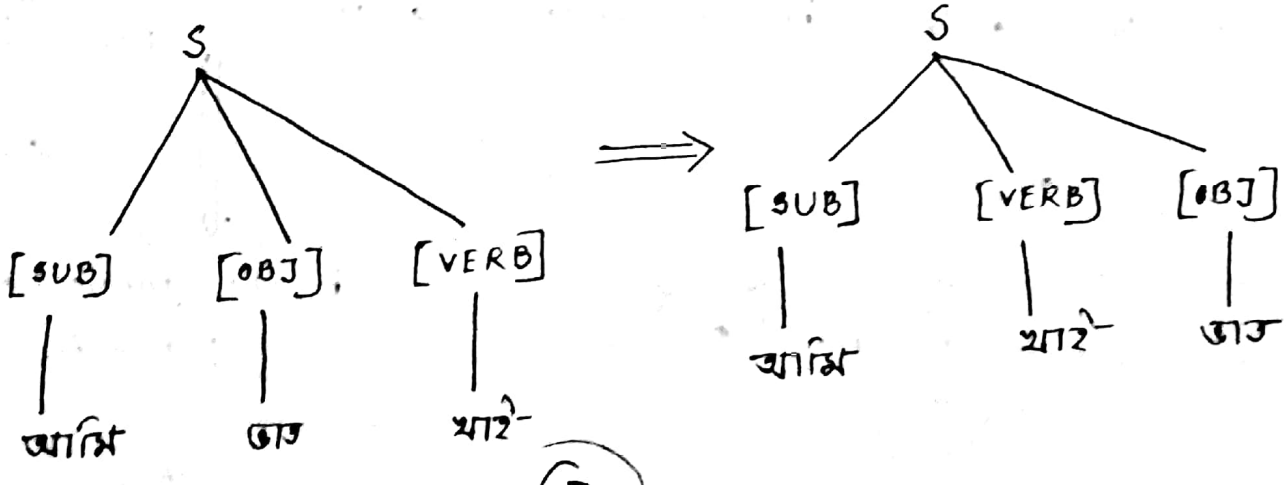



Words: আমি, ভাত, খাই-

Sentence structure: [subject] [object] [verb]

### Transfer stage:

Transformations are applied to the source language parse tree to convert the structure to that of the target language.

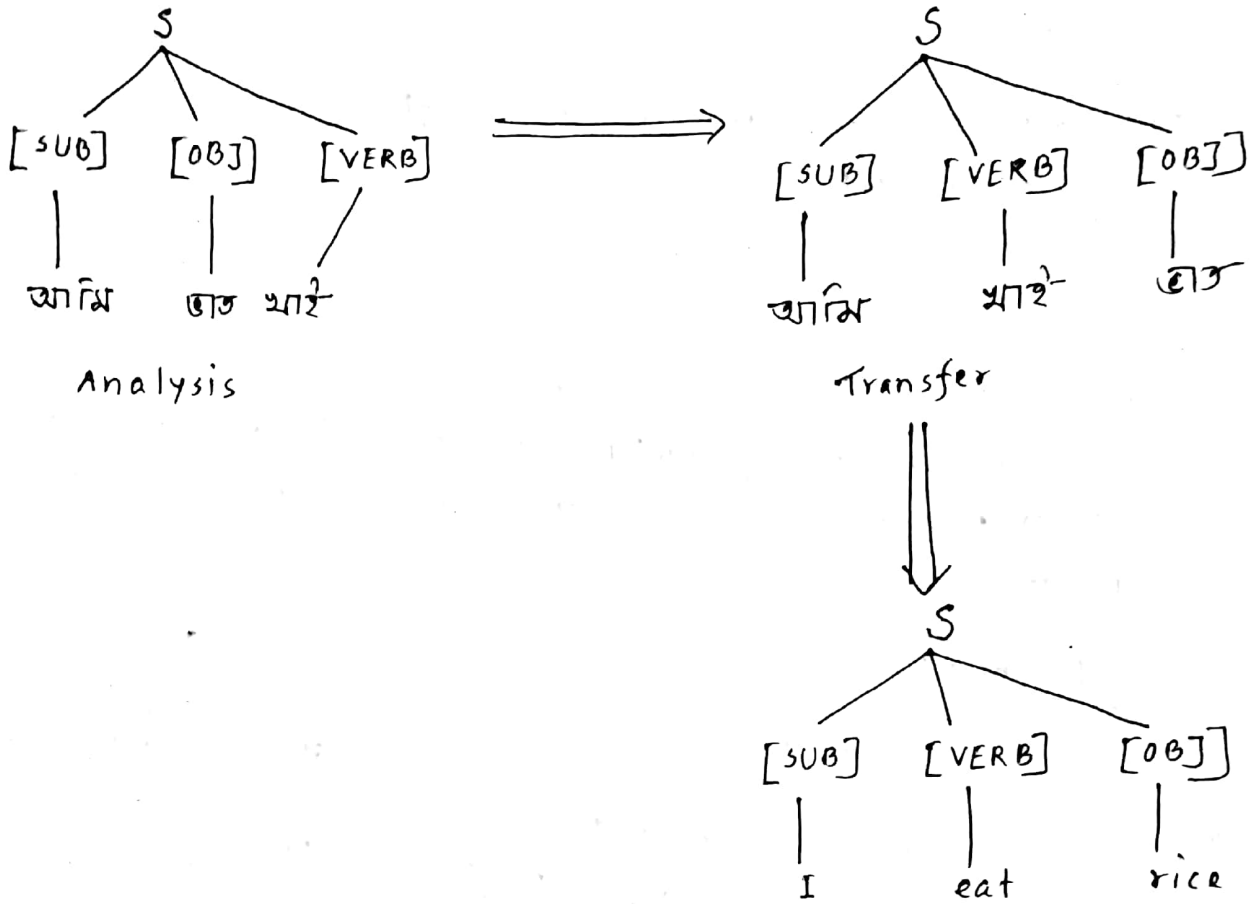





Scanned with CamScanner

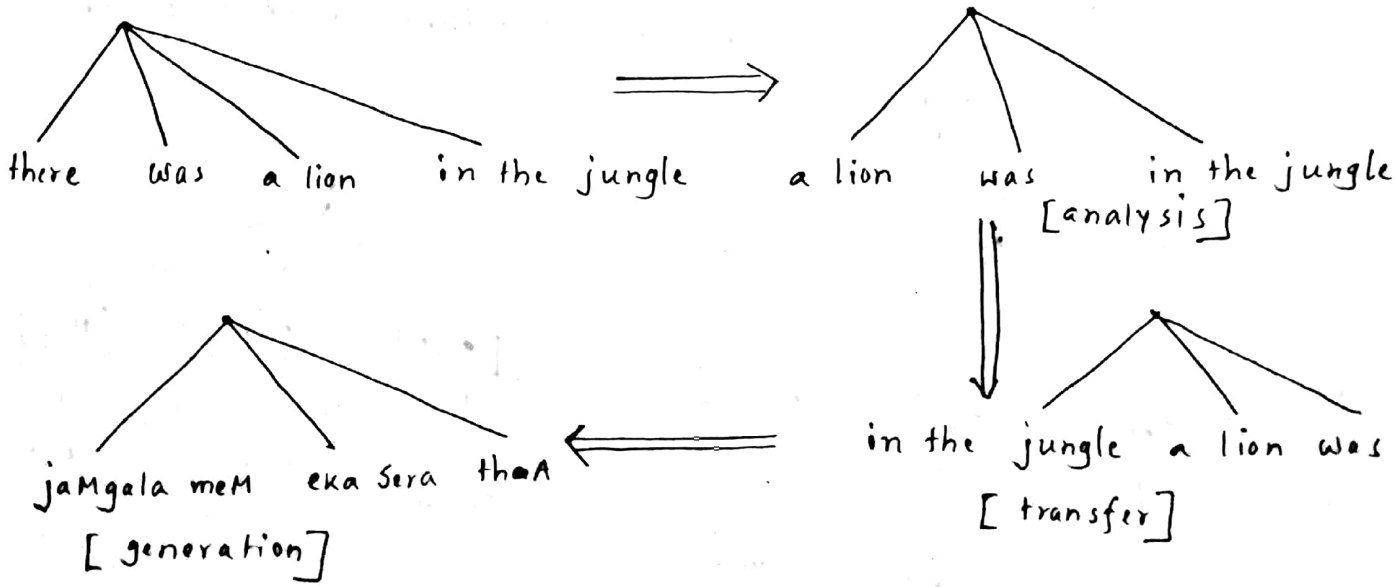
## The generation stage :

Translates the words and expresses the tense, number, gena etc. in the taget language.




## Example :

S : There was a lion in the jungle




Scanned with CamScanner

## Interlingua Approach:

It is a two stage process.

1. extracting the meaning of a source language sentence in a language-independent form.
2. Generating a target language sentence from the meaning.

**Importance:**

It expresses the complete meaning of any sentence by using a set of universal concepts and relations.

**Example:**

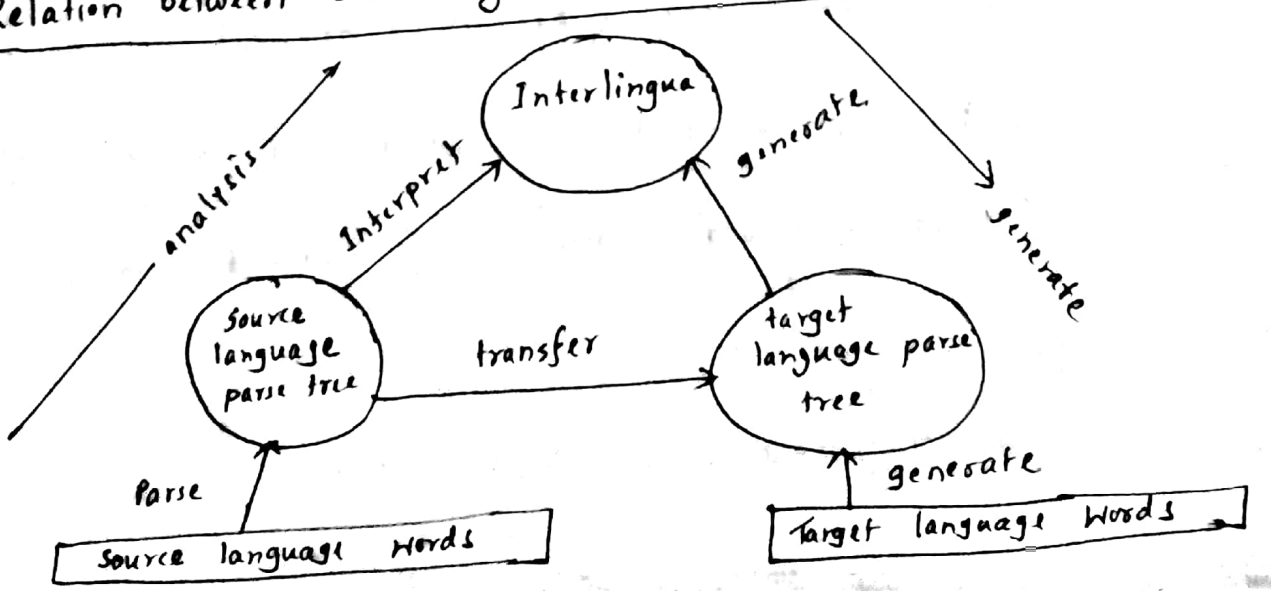
The child broke the toys with a hammer.

**Interlingual Representation**

| | |
|---|---|
| PREDICATE | BREAK |
| AGENT | CHILD number: SING |
| THEME | TOY number: PLUR |
| INSTRUMENT | HAMMER number: SING |
| TENSE | PAST |

⟨AGENT⟩ ⟨⟨PREDICATE⟩ ⟨TENSE⟩⟩ ⟨THEME⟩ ⟨INSTRUMENT⟩

## Relation between Interlingua and Transfer approach:






Scanned with CamScanner

## The Direct Approach:

step1: Morphological analysis

step2: Identify constituents

step3: Reorder them according to the constituent order in target language

step3: look up the words in an source-target language dictionary

step4: Inflect the target language words appropriately.

### Example:

We will demand peace in the country.

### English Sentence:

We will demand peace in the country.

### Morphological Analysis:

We demand FUTURE peace in country

### Identify constituents:

<We> <demand FUTURE> <peace> <in country>

### Reorder according to target language:

<We> <in country> <peace> <demand FUTURE>

### Look-up in the source-target language dictionary

<আমরা> <দেশের মধ্যে> <শান্তি> <দাবি FUTURE>

### Inflect:

আমরা দেশের শান্তি দাবি করব।

Scanned with CamScanner

## pus-Based Approaches:

- Not use human-encoded linguistic knowledge to solve the translation problem
- Used a training corpus of already translated texts - a parallel corpus - to guide the translation process.

A Parallel Corpus

| Source language collection | Target language collection |
|---|---|
| | |

## Example Based Machine Translation:

- Split the problem into subproblems.
- Recall how they solved similar subproblems in the past.
- Adapt these solutions to the new situation
- Combine the solutions to solve the bigger problem.

Example:

He bought a huge house.

Corpus:

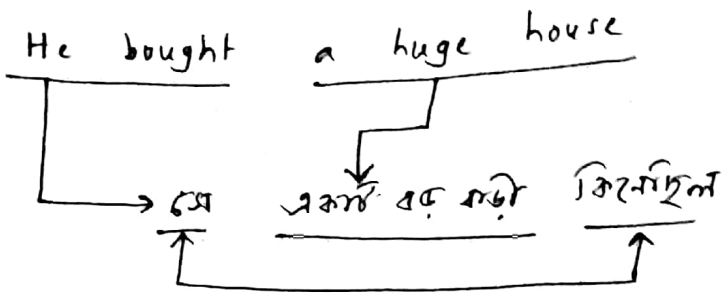
1a) He bought a pen

1b) সে একটি কলম কিনেছিল

2a) All ministers have a huge house

2b) সব মন্ত্রীর একটি বড় বাড়ি আছে

Scanned with CamScanner




Example based MT (EBMT) entails three steps:

1. Matching fragments against the parallel corpus.
2. Adapting the matched fragments to the target languege.
3. Recombing these translated fragments appropriately.

## Why machine Translation is hard?:

- Word order
- Word sense
- Pronouns
- Tense
- Idioms

### Word order:

English: SUB- VERB- OBJECT

বাংলা: SUB- OBJECT- VERB

English: I eat rice

বাংলা: আমি ভাত খাই-

Scanned with CamScanner

## Sense ambiguity:

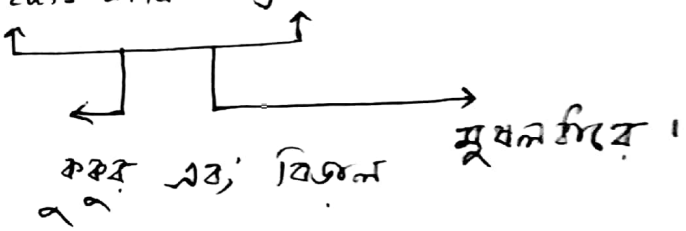
Example 1:
- Bank as in river
- Bank as in financial Institution

Example 2:
- Plant as in a tree
- Plant as in a factory

** Different word senses will likely translate into different words in another language.

## Idioms:

It has been raining cats and dogs




Which one will be appropriate
- বিড়াল এবং কুকুর
- মুষলধারে।

## Pronouns:

When should we use 'she' or 'he'.

সে ভাত খায় — He eats rice
সে ভাত খায় — She eats rice
} Which one is correct?



Scanned with CamScanner

Resources for Interlingual machine translation system

- Dictionaries (For analysis and generation)
- A conceptual knowledge based lexicon
- A set of projection rules
- Grammars for the analysis and generation of the languages involved

Characteristics of Interlinguas

- Unambiguous
  - should be more explicit.
- Represent the full content of the input text
  - Morphological
  - Syntactic
  - Semantic
  - Pragmatic char c
- should be Universal
  - Abstract meaning of any text.
- should represent the content of the input alone
- should be independent of both the source language text and the target language text.

Scanned with CamScanner

## Interlingua:



Interlingua machine translation is a classical approach.

Two steps of Interlingua:

1. Source language text is transformed into an interlingua (Here, interlingua is an abstract language-independent representation)
2. Target language is generated from the Interlingua.

[Alternative machine translation approach to the direct approach and the transfer approach]

### Advantages of Interlingual approach:

- Requires fewer components in order to relate each source language to each target language.
- Takes fewer components to add a new language.
- Supports paraphrases of the input in the original language
- Allows both the analysers and generatoos to be written by monolingual system developers.
- Handles languages that are very different from each other

### Disadvantages of Interlingual approach:

- Definition of an interlingua is difficult.

### Objective of Interlingua approach:

Analysis of text of a source language with respect to

i) Morphology
ii) Syntax
iii) Semantic

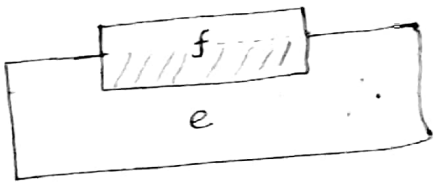
} Meaning

Scanned with CamScanner

that the best translation is the sentence that is both faithful to the original sentence and fluent in the target language

Ans:

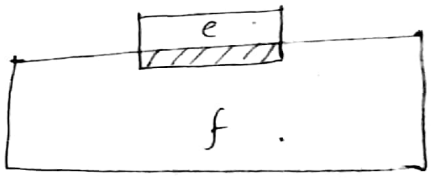
$$P(f|e) = \frac{\text{Total } f \text{ in } e}{\text{Total } e}$$




$$= \frac{P(f \wedge e)}{P(e)}$$

$$\Rightarrow P(f \wedge e) = P(f|e)\, P(e) \quad \cdots \quad eq^{(1)}$$

$$P(e|f) = \frac{\text{Total } e \text{ in } f}{\text{Total } f}$$




$$= \frac{P(f \wedge e)}{P(f)}$$

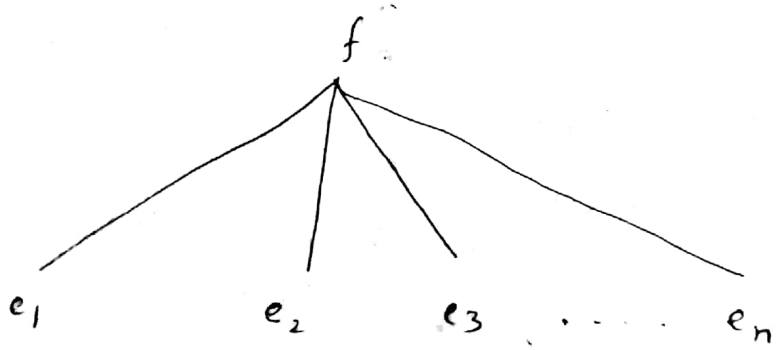
$$\Rightarrow P(f \wedge e) = P(e|f)\, P(f) \quad \cdots \quad eq^{(2)}$$

From $eq^{(1)}$ and $eq^{(2)} \Rightarrow$

$$P(e|f)\, P(f) = P(f|e)\, P(e)$$

$$\Rightarrow P(e|f) = \frac{P(f|e)\, P(e)}{P(f)}$$

If there are n sentences in target language for source language f




Scanned with CamScanner

$$P(e_1|f) = \frac{P(f|e_1)\ P(e_1)}{P(f)}$$

$$P(e_2|f) = \frac{P(f|e_2)\ P(e_2)}{P(f)}$$

$$P(e_3|f) = \frac{P(f|e_3)\ P(e_3)}{P(f)}$$

$$P(e_n|f) = \frac{P(f|e_n)\ P(e_n)}{P(f)}$$

$$P(e|f) = \max\left\{ P(e_1|f),\ P(e_2|f),\ P(e_3|f),\ \ldots\ldots,\ P(e_n|f) \right\}$$

$$= \max\left\{ \frac{P(f|e_1)\,P(e_1)}{P(f)},\ \frac{P(f|e_2)\,P(e_2)}{P(f)},\ \ldots\ldots,\ \frac{P(f|e_n)\,P(e_n)}{P(f)} \right\}$$

$$\cong \max\left\{ P(f|e_1)\,P(e_1),\ P(f|e_2)\,P(e_2),\ \ldots\ldots,\ P(f|e_n)\,P(e_n) \right\}$$

$$= \max\left\{ P(f|e_i)\ P(e_i) \right\}$$

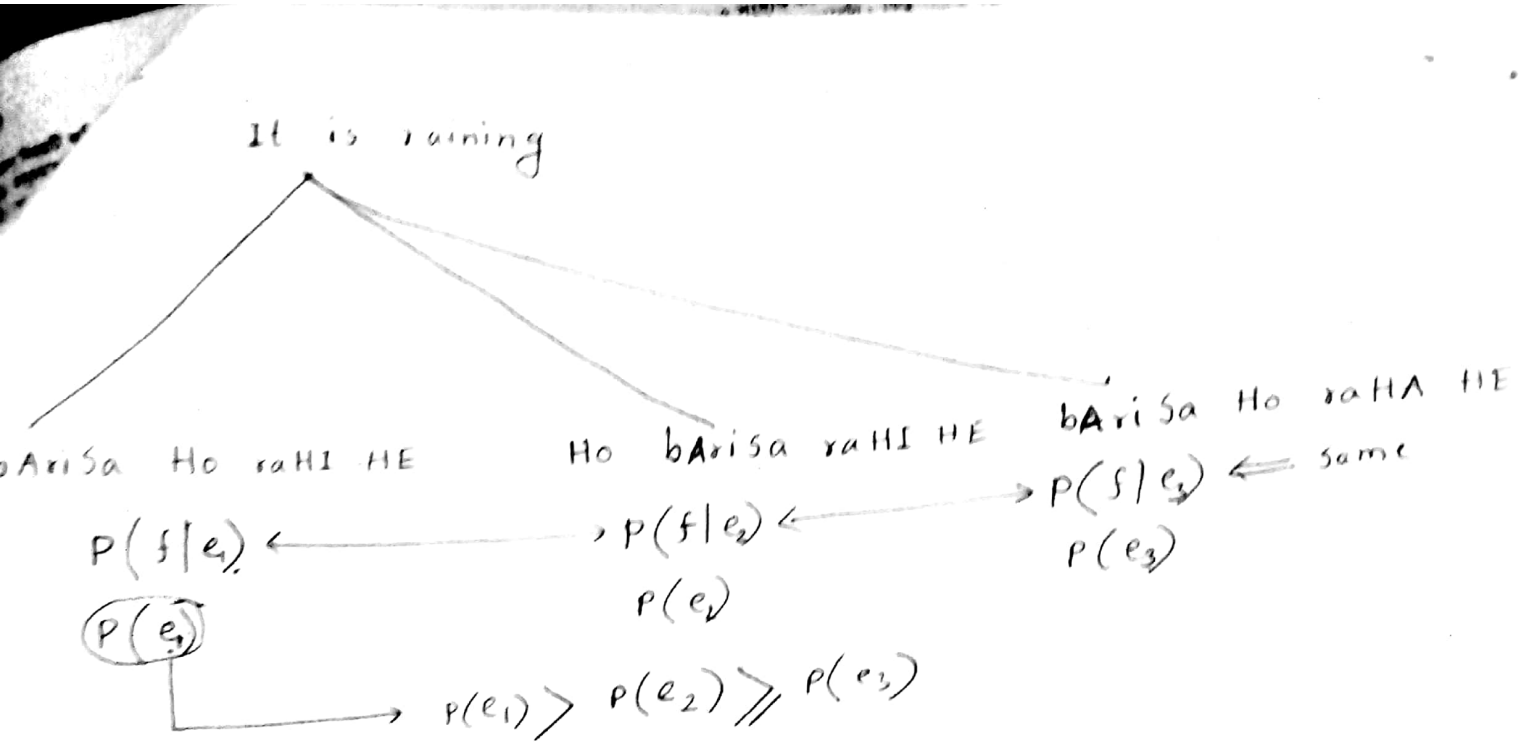
where $i = 1 \ldots n$

$$\hat{e} \in \{e_1, e_2, \ldots e_n\}$$

$$\hat{e} = \arg\max_{e_i} \left\{ P(f|e_i)\ P(e_i) \right\}$$

- $P(f|e_i)$ → Faithful sentence for source text
- $P(e_i)$ → fluent sentence in target language

Scanned with CamScanner




Since $P(f|e_1)\,P(e_1)$ maximum

It is raining $\Longrightarrow$ bArisa Ho raHI HE

## Noisy channel Model for Machine Translation :

Let source sentence $f$ to be translated was initially conceived in language E as some target sentences $e$.

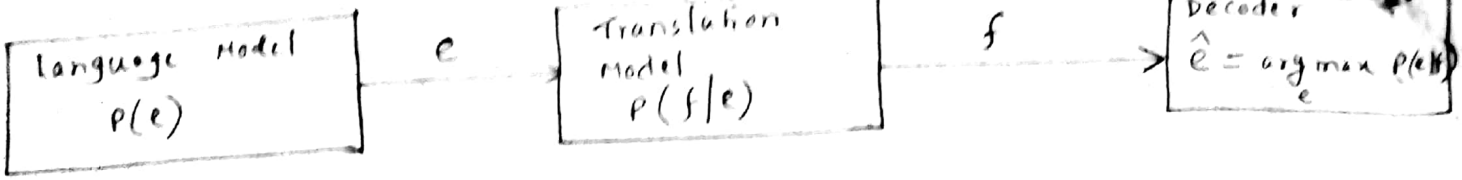
During communication $e$ was corrupted by the channel to $f$

Each sentence in E is a translation of $f$ with some probability

The sentence we choose as the translation ($\hat{e}$) is the one that has the highest probability

$$\hat{e} = \underset{e}{\operatorname{argmax}}\ P(e|f)$$

Scanned with CamScanner



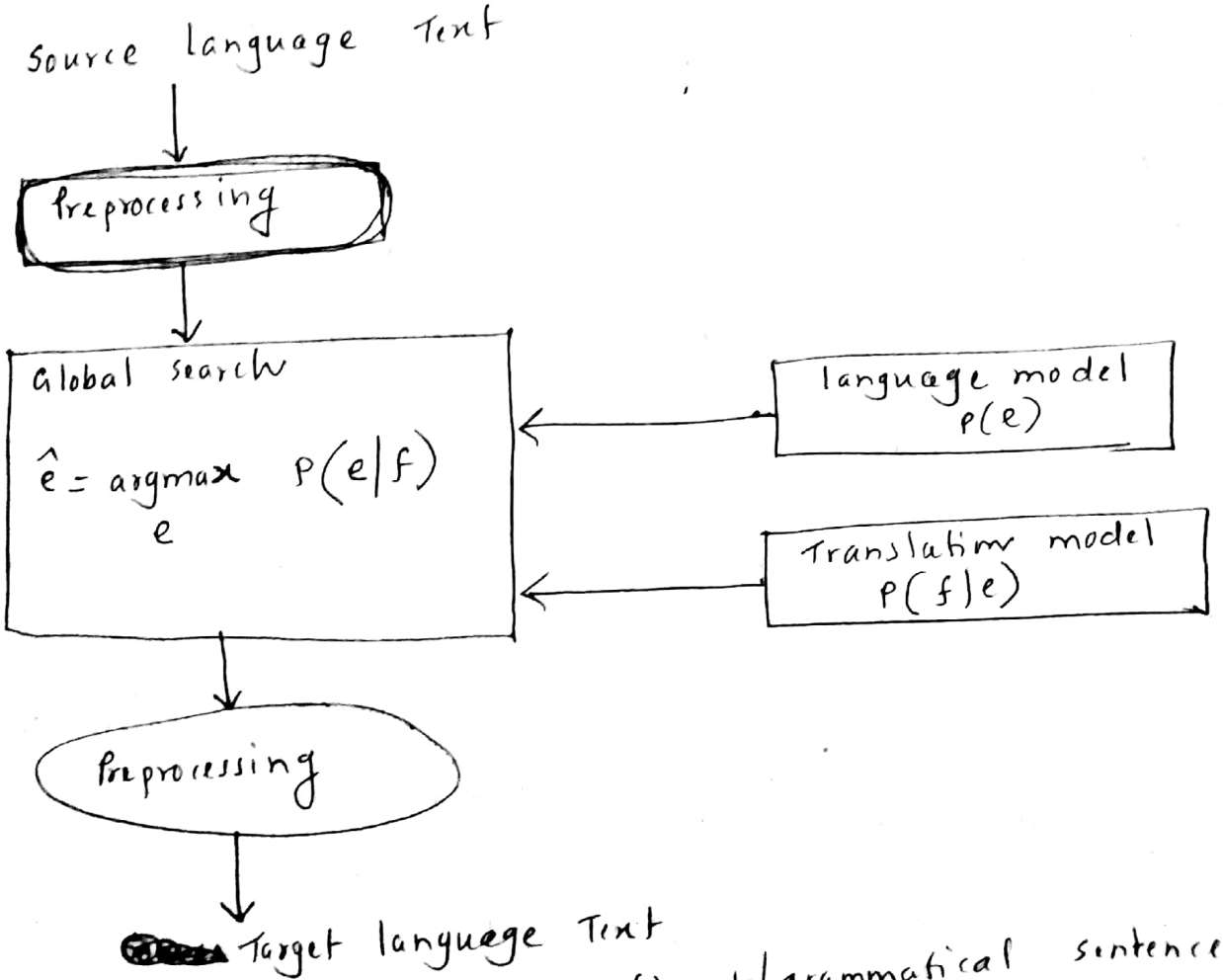

P(e): language model

The kind of sentences that are likely in the language E.

P(f|e): Translation model

The way sentences in E get converted to sentences in F. This is called translation model.

System diagram of Machine Translation:




P(e): Assigns higher probability to fluent/grammatical sentences
: Estimated using monolingual corpora

P(f|e): Assign higher probability to sentences that have corresponding meaning
: Estimated using bilingual corpora

Scanned with CamScanner

# Lexical Translation

Haus – house, building, home, household, shell

German Text — English Text

**Step1:** Collect statistics

Look at a parallel corpus (German Text vs English translation)

| Haus | Count |
|---|---|
| house | 8000 |
| building | 1600 |
| home | 200 |
| household | 150 |
| shell | 50 |

**Step2:** Estimate translation probabilities

Total = 8000 + 1600 + 200 + 150 + 50

= 10000

$$P_{Haus}(house) = \frac{8000}{10000} = 0.8$$

$$P_{Haus}(building) = \frac{1600}{10000} = 0.16$$

$$P_{Haus}(home) = \frac{200}{10000} = 0.02$$

$$P_{Haus}(household) = \frac{150}{10000} = 0.015$$

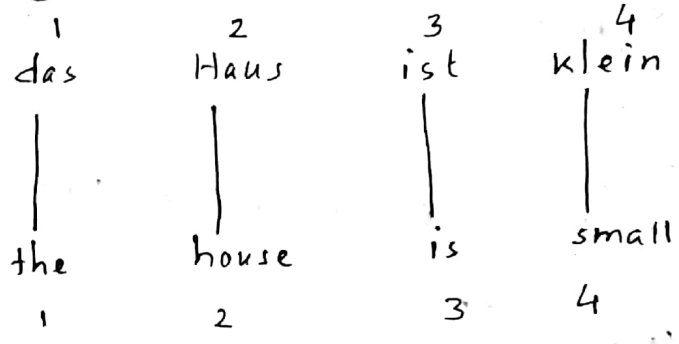
$$P_{Haus}(shell) = \frac{50}{10000} = 0.005$$

$$P_f(e) = \begin{cases} 0.8 & \text{if } e = \text{house} \\ 0.16 & \text{if } e = \text{building} \\ 0.02 & \text{if } e = \text{home} \\ 0.015 & \text{if } e = \text{household} \\ 0.005 & \text{if } e = \text{shell} \end{cases}$$

Scanned with CamScanner

## Alignment:

In a parallel text (or when we translate), we align words in one language with the words in the other.



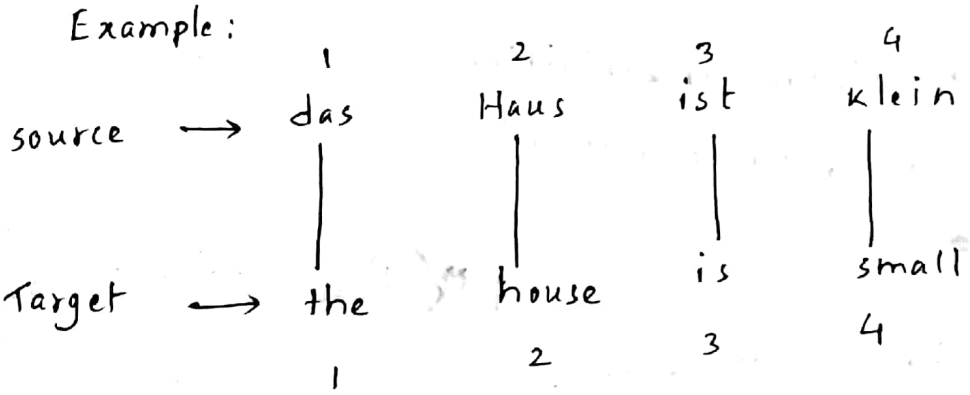

Word positions are numbered 1–4

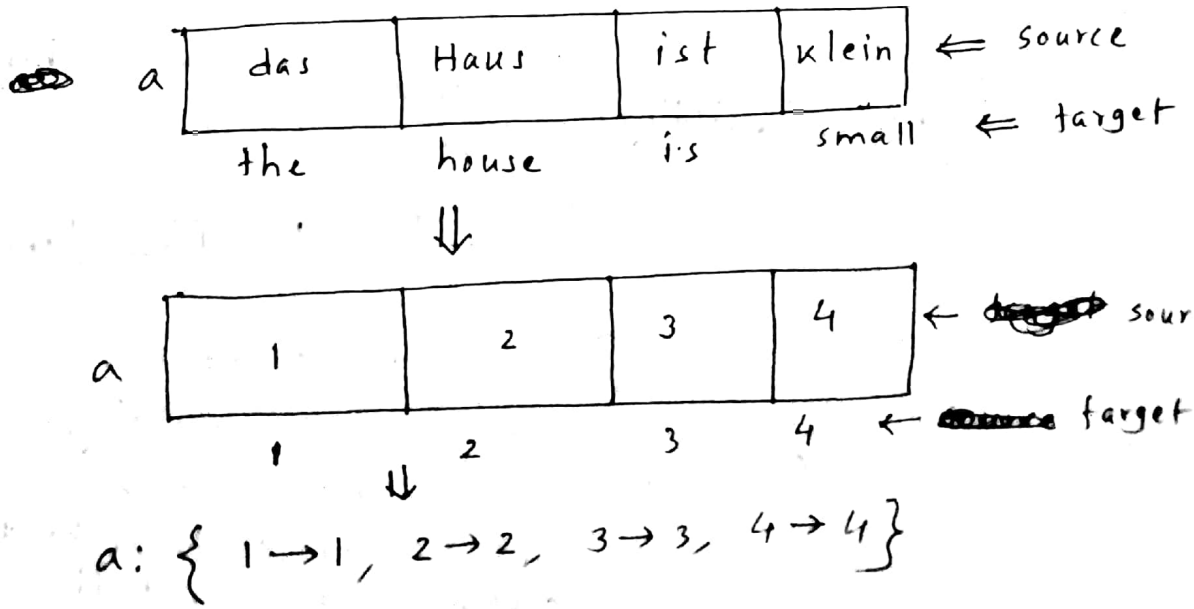
## Alignment function

Mapping an target word (English) at position i to a source word (German) at position j with a function

a : i → j

Example:



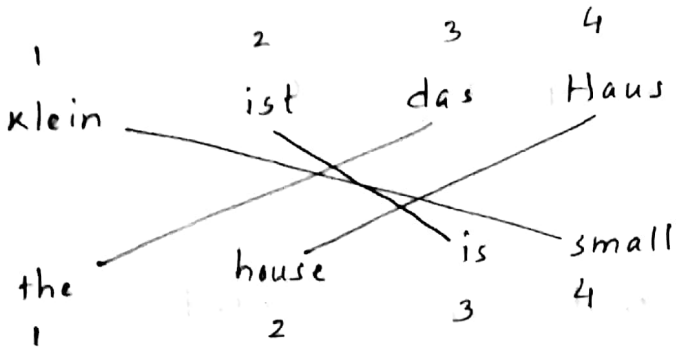



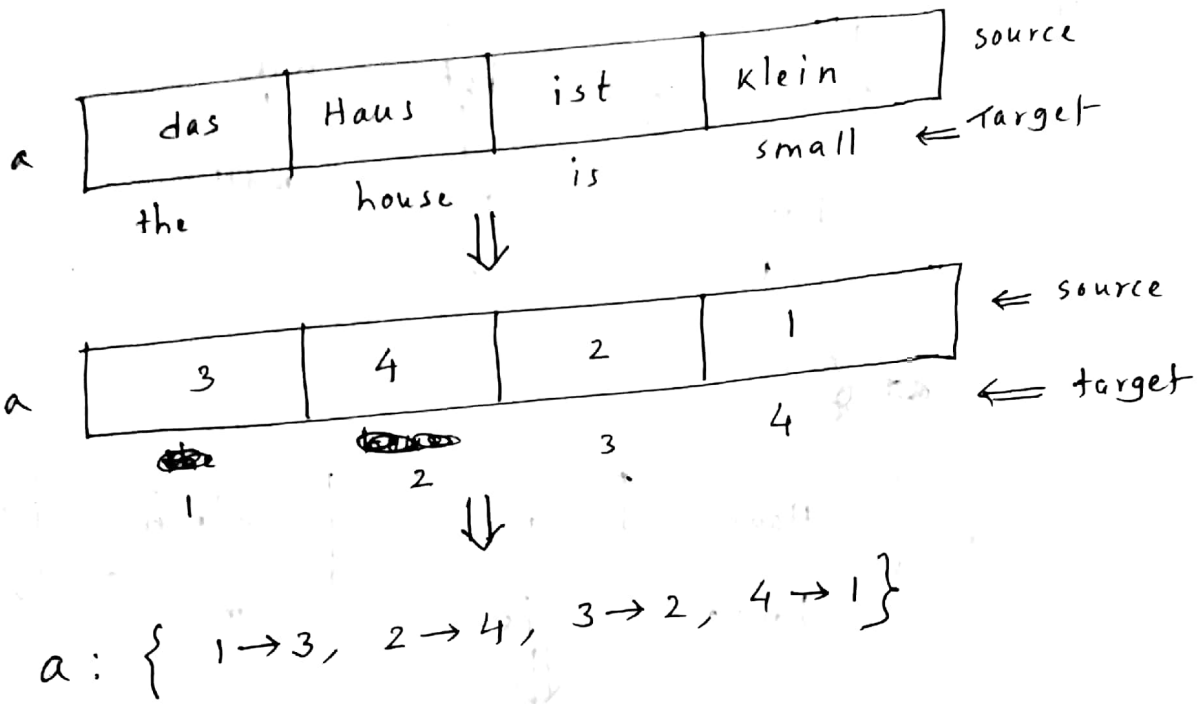

Scanned with CamScanner

ordering:

Words may be reordered during translation

Example:



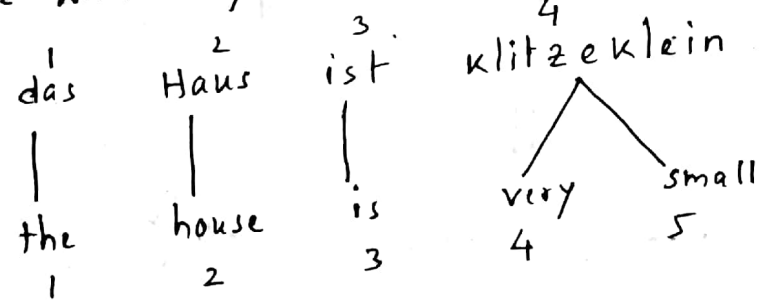



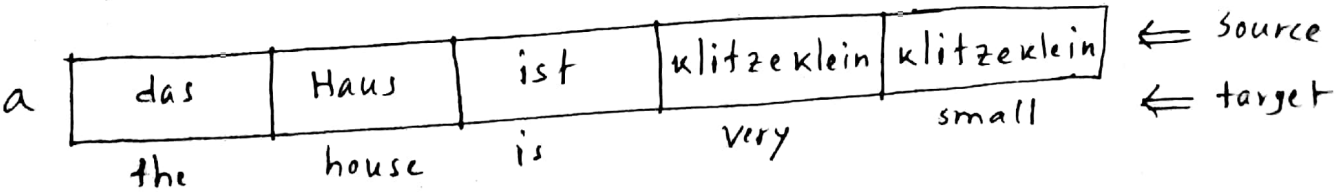

$a: \{ 1 \to 3,\ 2 \to 4,\ 3 \to 2,\ 4 \to 1 \}$

One-to-many translation:

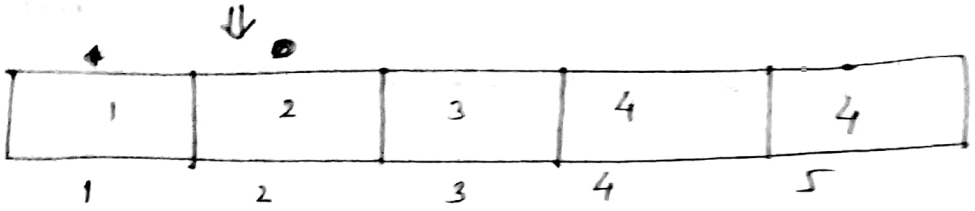
A source word may translate into multiple target words

Example:




Scanned with CamScanner

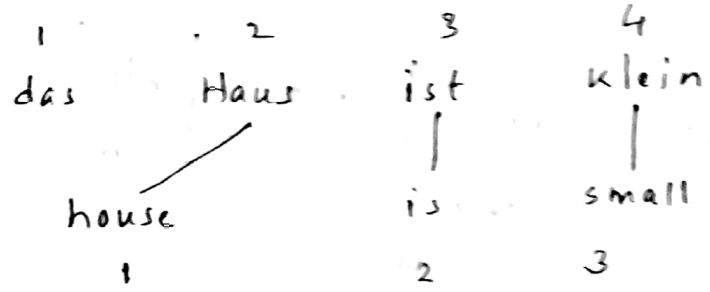


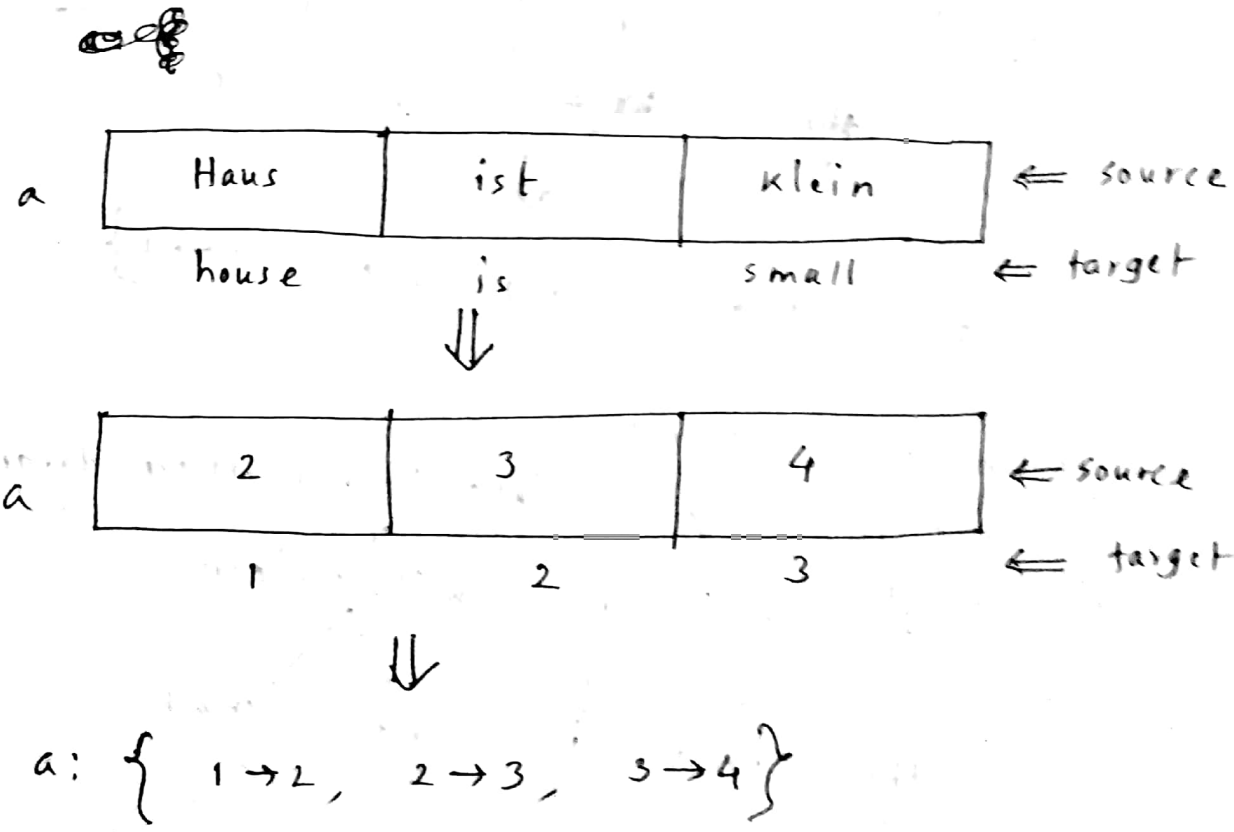

a : { 1→1, 2→2, 3→3, 4→4, 5→4 }

## Dropping Words :

Words may be dropped when translated

Example :




a : { 1→2, 2→3, 3→4 }

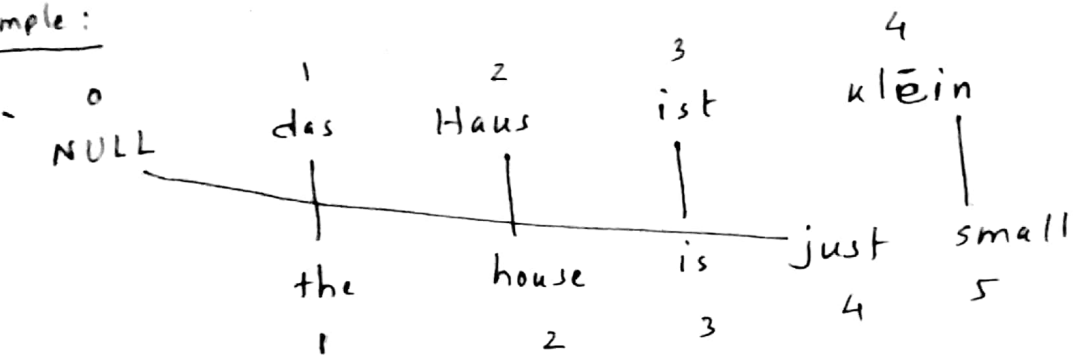
Scanned with CamScanner

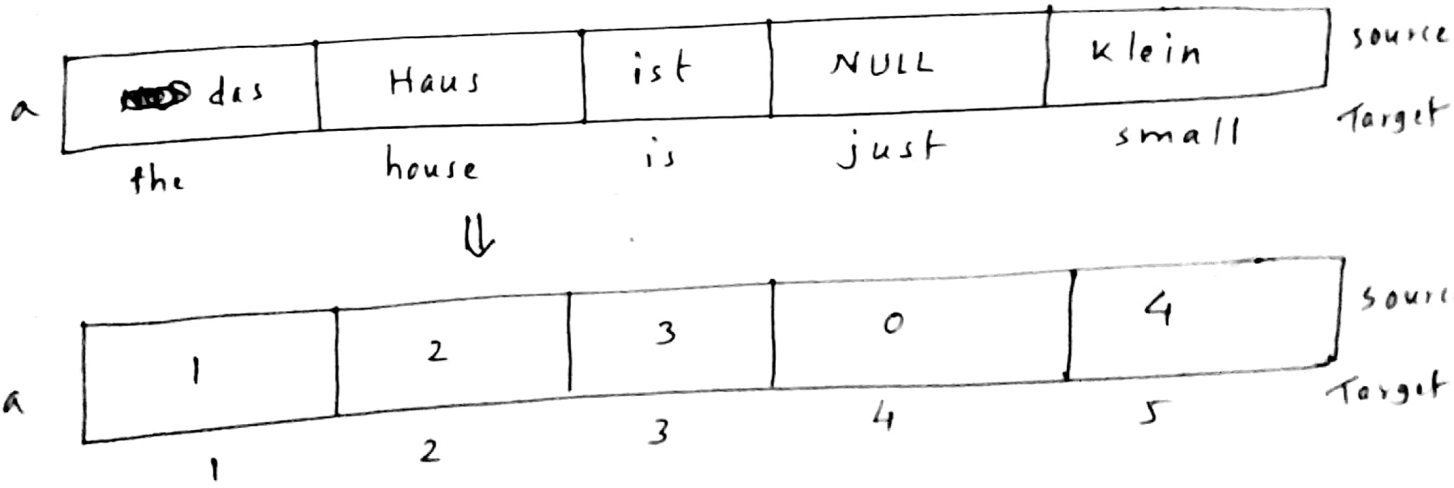
## Inserting Words:

Words may be added during translation

- The English just does not have an equivalent in German
- We still need to map it to something: special NULL token

**Example:**




$a: \{1 \to 1,\ 2 \to 2,\ 3 \to 3,\ 4 \to 0,\ 5 \to 4\}$

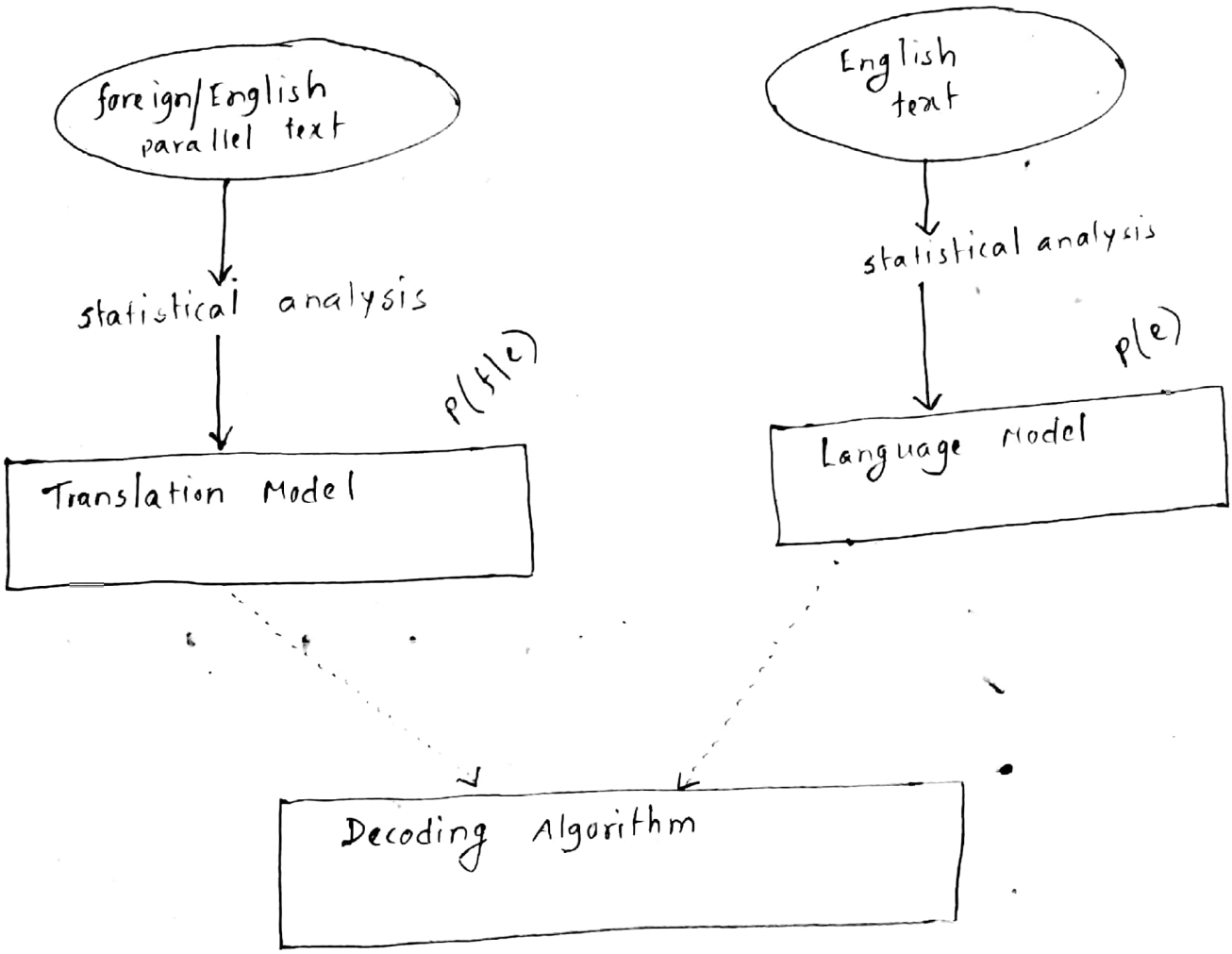
Scanned with CamScanner

## Statistical Machine Translation:

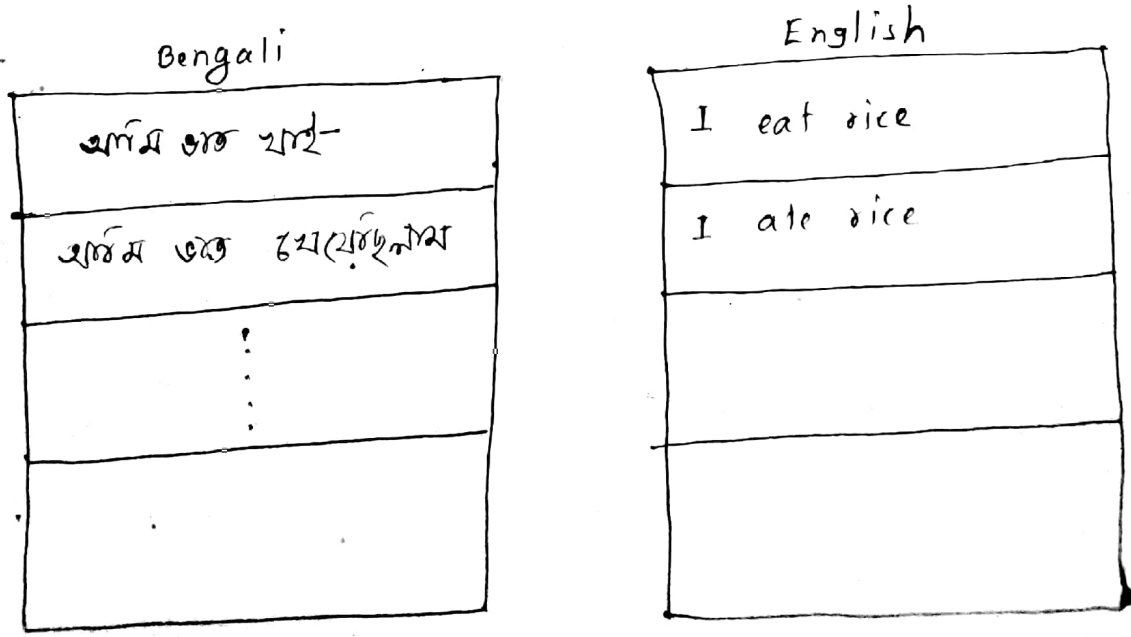
$$P(e|f) = \max\left( P(f|e) * P(e) \right)$$

$P(e)$ → Language model

$P(f|e)$ → Translation model




## Parallel Corpus:




Scanned with CamScanner

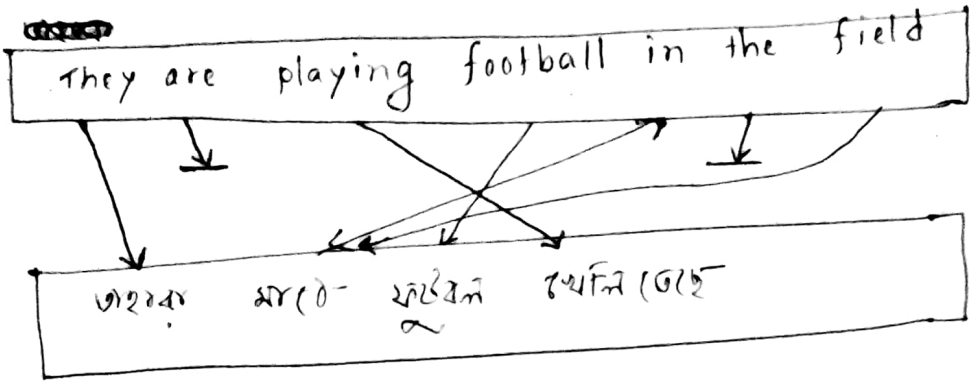
Different Models in statistical Machine Translation

- Word based models.
- Phrase based models.
- Sytax based models.

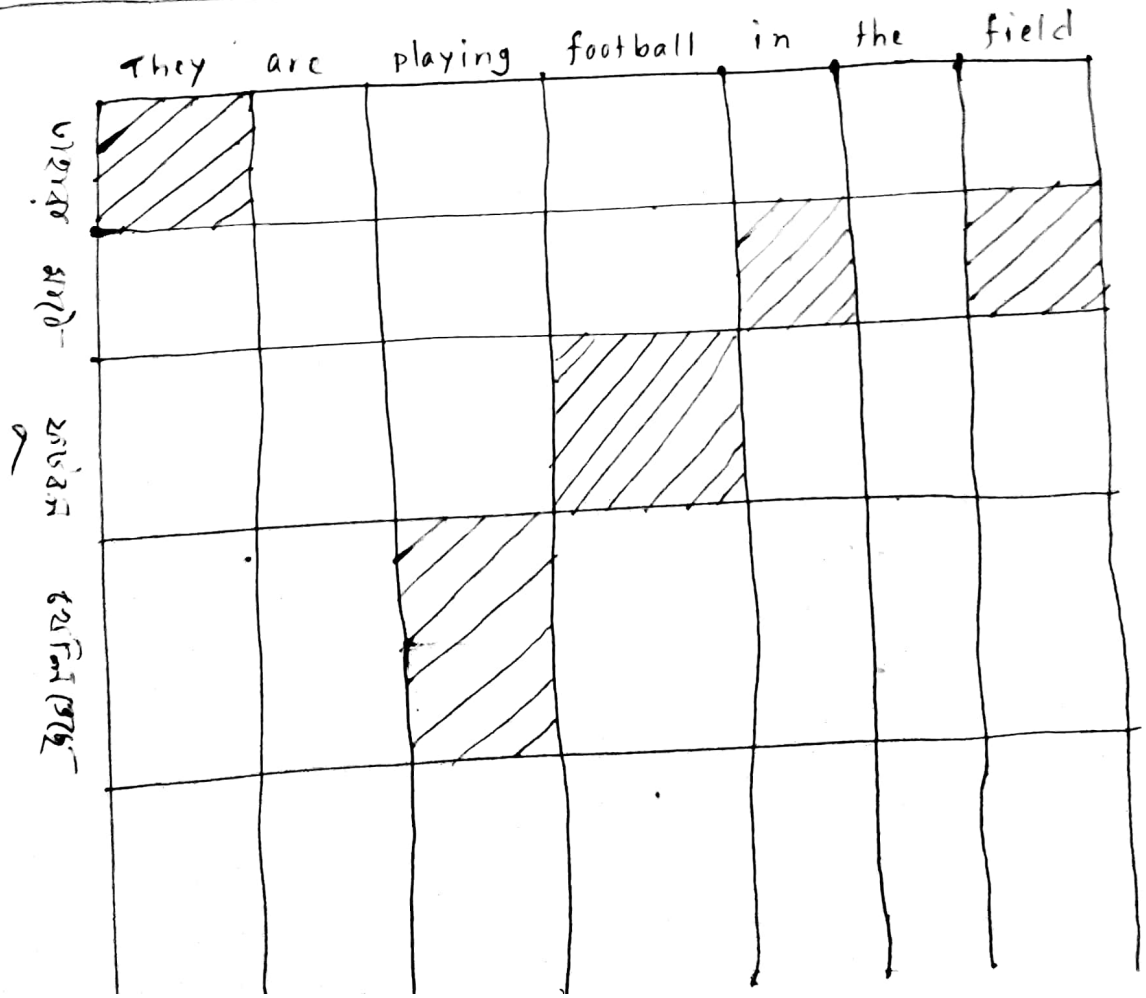
## Word based models:

Translation process is decomposed into smaller steps, each is tied words.

Example:



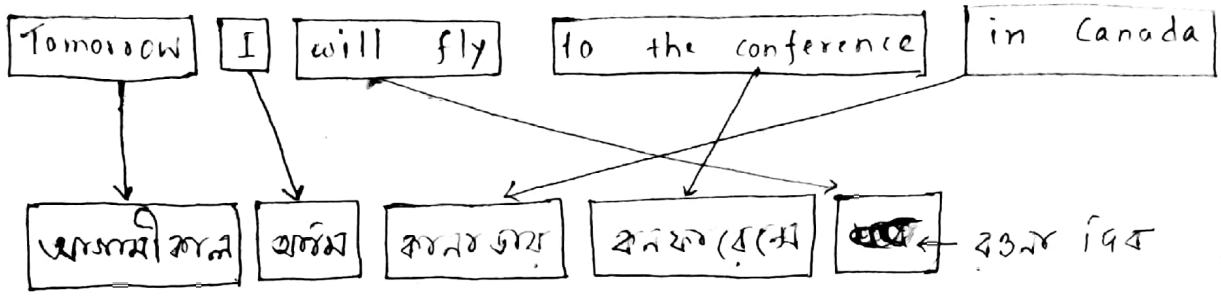

## Word alignment:




Scanned with CamScanner

## phrase-based models:

Ex:



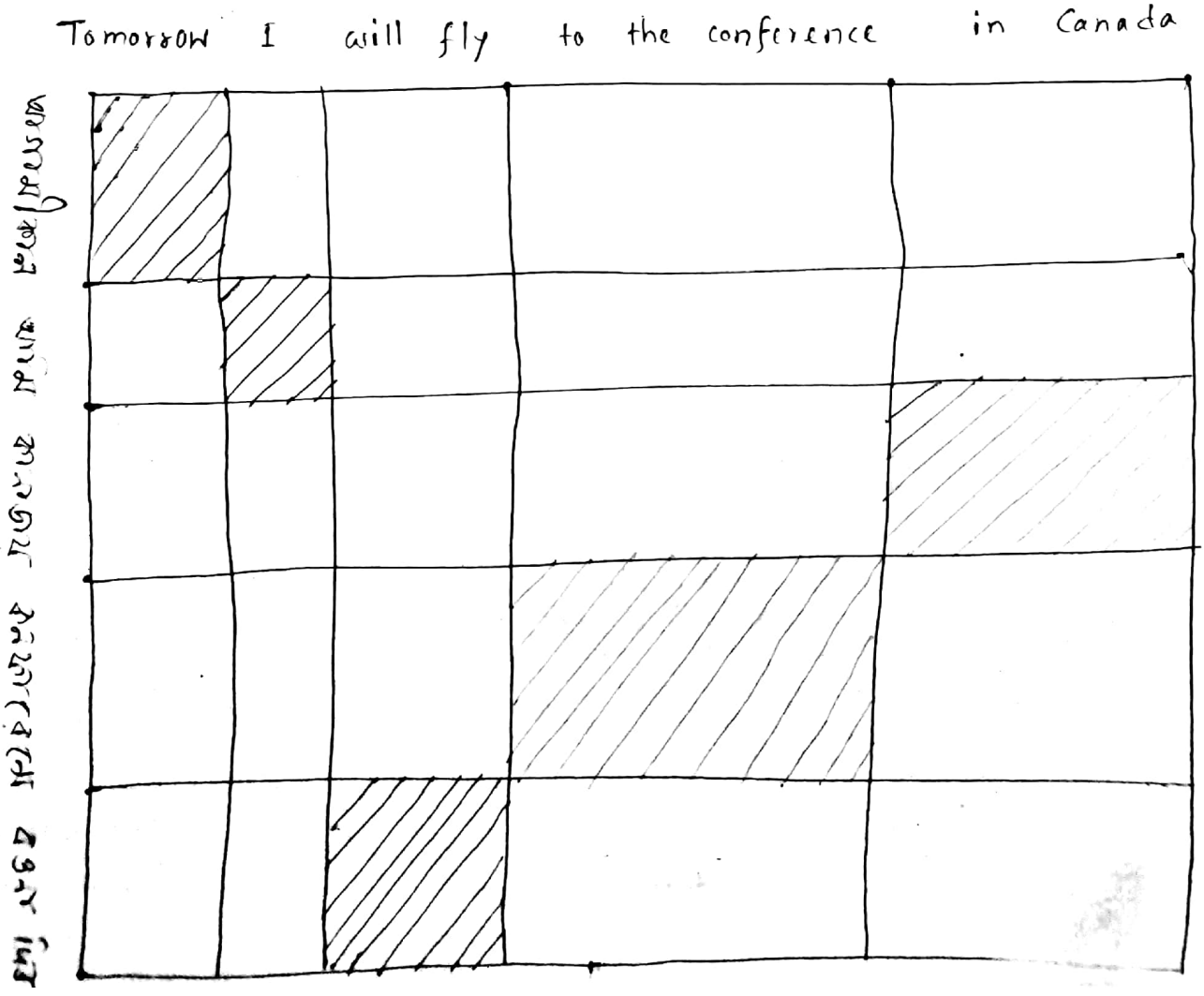

## Algorithm:

step1: Foreign input is segmented into phrases
- Any sequence of words

step2: Each phrase is translated into English

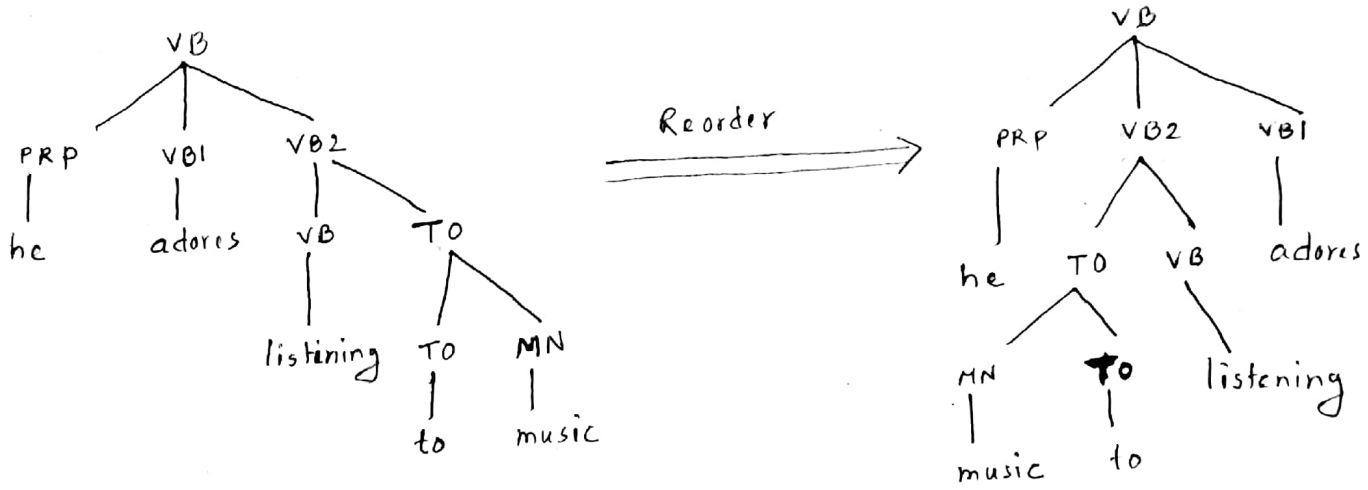
step3: phrases are reordered.

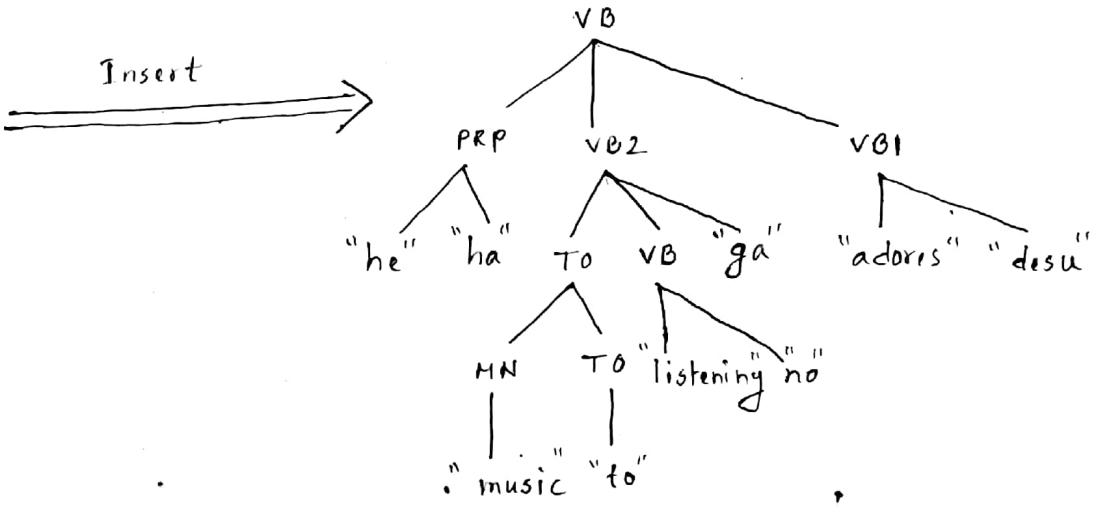
## phrase alignment:

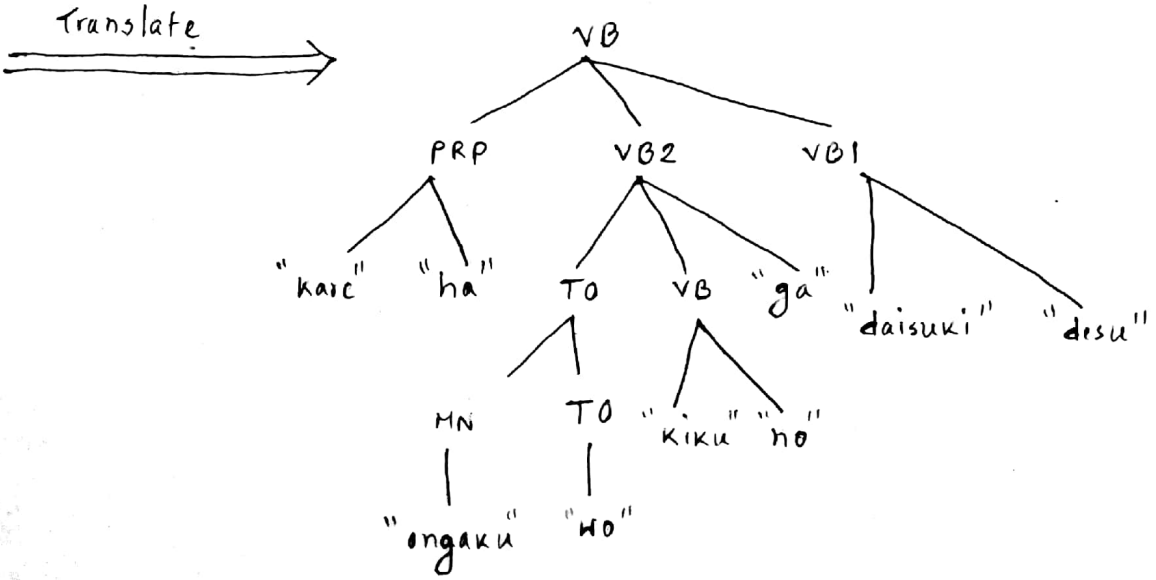



Scanned with CamScanner

## Syntax-based models:




Take leaves

Kare ha ongaku wo kiku no ga daisuki desu

Scanned with CamScanner

Performance :

- Word Error Rate
- Word Inaccuracy

Example:

Bangali: তারা মাঠে ফুটবল খেলছে

English: They are playing football in the field

তারা মাঠে ফুটবল খেলছে

Input: তারা | | খেলছে | ফুটবল | | | মাঠে

Output:

| They | are | playing | football | | the | in field |
|---|---|---|---|---|---|---|

Total words, $N = 4$
Substitution, $S = 0$
Deletion, $D = 0$
Insertion, $I = 2$

$$\text{Word Error Rate} = 100\% - \text{Word Correct Rate}$$

$$= 100\% - \frac{N - S - D}{N} * 100\%$$

$$= 100\% - \frac{4 - 0 - 0}{4} * 100\%$$

$$= 100\% - 100\%$$

$$= 0\%$$

Scanned with CamScanner

# IBM Model 1

1. **Generative model:**

Break up translation process into smaller steps.
- IBM Model 1 only uses lexical translation.

2. **Translation probability:**

For a foreign sentence, $f = (f_1, \ldots, f_{l_f})$ of length $l_f$

To a English sentence, $e = (e_1, \ldots, e_{l_e})$ of length $l_e$

With an alignment of each english word $e_j$ to a foreign word $f_i$ according to the alignment function $a: j \rightarrow$

$$p(e, a|f) = \frac{\epsilon}{(l_f + 1)^{l_e}} \prod_{j=1}^{l_e} t\left(e_j \mid f_{a(j)}\right)$$

where parameter $\epsilon$ is a normalization constant.

**Example:**

Source sentence, f : das Haus ist klein

Target sentence, e : the house is small

das

| e | t(e\|f) |
|---|---|
| the | 0.7 |
| that | 0.15 |
| which | 0.075 |
| who | 0.05 |
| this | 0.025 |

Haus

| e | t(e\|f) |
|---|---|
| house | 0.8 |
| building | 0.16 |
| home | 0.02 |
| household | 0.015 |
| shell | 0.005 |

Scanned with CamScanner

ist

| e | t(e\|f) |
|---|---|
| is | 0.8 |
| 's | 0.16 |
| exists | 0.02 |
| has | 0.015 |
| are | 0.005 |

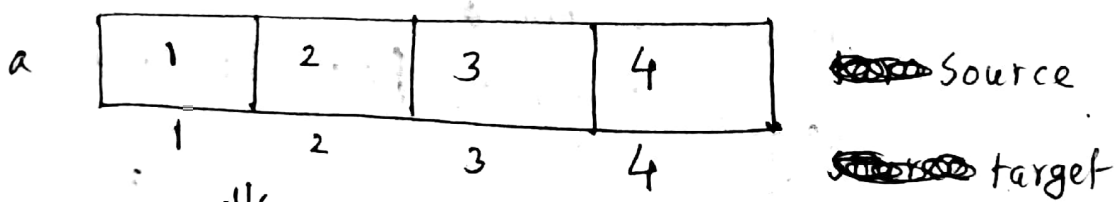
| e | t(e\|f) |
|---|---|
| small | 0.4 |
| little | 0.4 |
| short | 0.1 |
| minor | 0.06 |
| petty | 0.04 |

Find lexical Translation probability for the given source and target language sentences.

Ans:

$$P(e, a|f) = \frac{\epsilon}{(l_f+1)^{l_e}} \prod_{j=1}^{l_e} t(e_j | f_{a(j)}) \quad \text{..... eqn (1)}$$

| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Source language sentence, f: | das | Haus | ist | klein |
| Target language sentence, e: | the | house | is | small |
| | 1 | 2 | 3 | 4 |




a: {1→1, 2→2, 3→3, 4→4}

Here, $l_f = 4$

$l_e = 4$

Scanned with CamScanner

From eq (1)

$$P(e, a|f) = \frac{\epsilon}{(4+1)^4} \prod_{j=1}^{4} t(e_j | f_{a(j)})$$

$$= \frac{\epsilon}{5^4} \cdot \{ t(e_1 | f_{a(1)}) * t(e_2 | f_{a(2)}) * t(e_3 | f_{a(3)}) * t(e_4 | f_{a(4)}) \}$$

$$= \frac{\epsilon}{5^4} \{ t(e_1 | f_1) * t(e_2 | f_2) * t(e_3 | f_3) * t(e_4 | f_4) \}$$

$$= \frac{\epsilon}{5^4} \{ t(\text{the} | \text{das}) * t(\text{house} | \text{Haus}) * t(\text{is} | \text{ist}) * t(\text{small} | \text{klein}) \}$$

$$= \frac{\epsilon}{5^4} (0.7 * 0.8 * 0.8 * 0.4)$$

$$= \frac{0.7 * 0.8 * 0.8 * 0.4}{625} \quad \text{where } \epsilon = 1$$

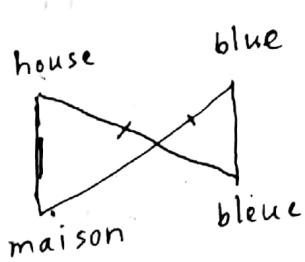
$$= \frac{0.1792}{625}$$

$$= 0.00028672$$

Scanned with CamScanner

## Expection Maximization algorithm :

To estimate translation values from two sentences.



**Example:**

maison bleue ⟷ blue house

maison ⟷ house

From the example, maison ⟷ blue
maison ⟷ house
bleue ⟷ blue
bleue ⟷ house

From another, maison → house

step1: set parameter values uniformly

$t(\text{bleue}|\text{house}) = \frac{1}{2}$

$t(\text{maison}|\text{house}) = \frac{1}{2}$

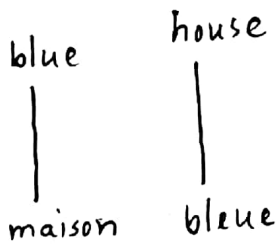
$t(\text{bleue}|\text{blue}) = \frac{1}{2}$

$t(\text{maison}|\text{blue}) = \frac{1}{2}$

**Iteration 1:**

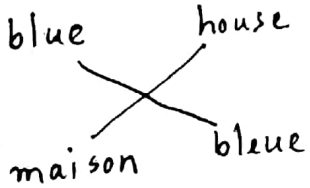
step2: Compute $P(a, f|e)$ for all alignments

Alignment 1:




$P(a,f|e) = t(\text{maison}|\text{blue}) * t(\text{bleue}|\text{ho}$
$= \frac{1}{2} * \frac{1}{2} = \frac{1}{4}$

Alignment 2:



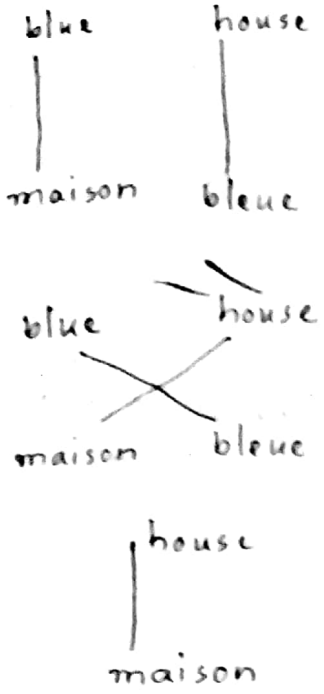

$P(a,f|e) = t(\text{bleue}|\text{blue}) * t(\text{maison}|\text{house})$
$= \frac{1}{2} * \frac{1}{2} = \frac{1}{4}$

Alignment 3:

$P(a,f|e) = t(\text{maison}|\text{house}) = \frac{1}{2}$

Scanned with CamScanner

step3: Normalize $P(a, f|e)$ values to yeild $P(a|e,f)$ values




$$P(a|e,f) = \frac{P(a,f|e)}{\sum_a P(a,f|e)} = \frac{\frac{1}{4}}{\frac{1}{4}+\frac{1}{4}} = \frac{\frac{1}{4}}{\frac{2}{4}} = \frac{1}{2}$$

$$P(a|e,f) = \frac{P(a,f|e)}{\sum_a P(a,f|e)} = \frac{\frac{1}{4}}{\frac{1}{4}+\frac{1}{4}} = \frac{1}{2}$$

$$P(a|e,f) = \frac{P(a,f|e)}{\sum_a P(a,f|e)} = \frac{\frac{1}{2}}{\frac{1}{2}} = 1$$

[There is only one alignment, so $P(a|e,f)$ will be 1 always]

step4: Collect fractional counts

$tc(\text{maison}|\text{blue}) = \frac{1}{2}$

$tc(\text{bleue}|\text{house}) = \frac{1}{2}$

$tc(\text{bleue}|\text{blue}) = \frac{1}{2}$

$tc(\text{maison}|\text{house}) = \frac{1}{2} + 1 = \frac{3}{2}$

step5: Normalize fractional counts to get revised parameter values

$$t(\text{maison}|\text{blue}) = \frac{\frac{1}{2}}{\frac{1}{2}+\frac{1}{2}} = \frac{1}{2}$$

$$t(\text{bleue}|\text{house}) = \frac{\frac{1}{2}}{\frac{1}{2}+\frac{3}{2}} = \frac{1}{2} * \frac{2}{4} = \frac{1}{4}$$

$$t(\text{bleue}|\text{blue}) = \frac{\frac{1}{2}}{\frac{1}{2}+\frac{1}{2}} = \frac{1}{2}$$

$$t(\text{maison}|\text{house}) = \frac{\frac{3}{2}}{\frac{1}{2}+\frac{3}{2}} = \frac{3}{2} * \frac{2}{4}$$

Scanned with CamScanner

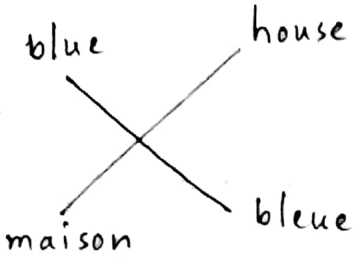
**Iteration2 :**

step2 : Compute $P(a, f|e)$ for all alignments

Alignment-I

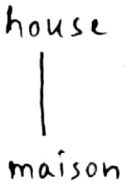
$$P(a, f|e) = t(\text{maison}|\text{blue}) * t(\text{bleue}|\text{house}) = \frac{1}{2} * \frac{1}{4} = \frac{1}{8}$$

Alignment-II




$$P(a, f|e) = t(\text{bleue}|\text{blue}) * t(\text{maison}|\text{house}) = \frac{1}{2} * \frac{3}{4} = \frac{3}{8}$$

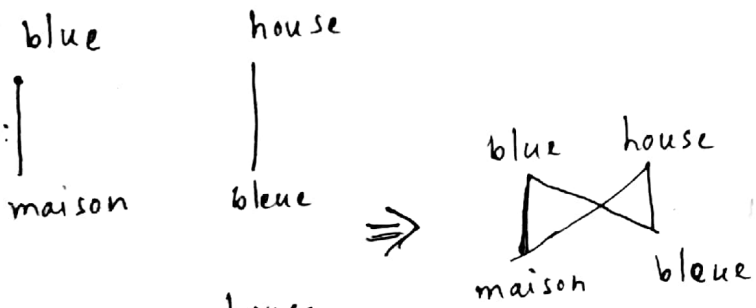
Alignment-III




$$P(a, f|e) = t(\text{maison}|\text{house}) = \frac{3}{4}$$

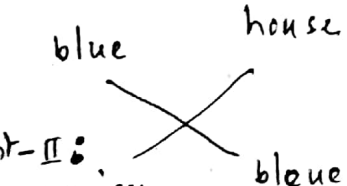
step3 : Normalize $P(a, f|e)$ values to yeild $P(a|e, f)$ values

Alignment-I:



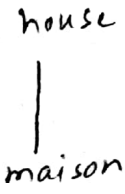

$$P(a|e, f) = \frac{P(a, f|e)}{\sum P(a, f|e)} = \frac{\frac{1}{8}}{\frac{1}{8} + \frac{3}{8}} = \frac{1}{4}$$

Alignment-II :




$$P(a|e, f) = \frac{P(a, f|e)}{\sum P(a, f|e)} = \frac{\frac{3}{8}}{\frac{1}{8} + \frac{3}{8}} = \frac{3}{4}$$

Alignment-III :




$$P(a|e, f) = \frac{P(a, f|e)}{\sum P(a, f|e)} = \frac{\frac{3}{4}}{\frac{3}{4}} = 1$$

Scanned with CamScanner

house)

house)

step4: Collect fractional counts

$tc(\text{maison} \mid \text{blue}) = \frac{1}{4}$

$tc(\text{bleue} \mid \text{house}) = \frac{1}{4}$

$tc(\text{bleue} \mid \text{blue}) = \frac{3}{4}$

$tc(\text{maison} \mid \text{house}) = \frac{3}{4} + 1 = \frac{7}{4}$

step5: Normalize fractional counts to get revised parameter values

$$t(\text{maison} \mid \text{blue}) = \frac{\frac{1}{4}}{\sum tc(\ldots \mid \text{blue})}$$

$$= \frac{\frac{1}{4}}{\frac{1}{4} + \frac{3}{4}} = \frac{1}{4}$$

$$t(\text{bleue} \mid \text{house}) = \frac{\frac{1}{4}}{\sum tc(\ldots \mid \text{house})}$$

$$= \frac{\frac{1}{4}}{\frac{1}{4} + \frac{7}{4}}$$

$$= \frac{1}{8}$$

$$t(\text{bleue} \mid \text{blue}) = \frac{\frac{3}{4}}{\sum tc(\ldots \mid \text{blue})} = \frac{\frac{3}{4}}{\frac{1}{4} + \frac{3}{4}} = \frac{3}{4}$$

$$t(\text{maison} \mid \text{house}) = \frac{\frac{7}{4}}{\frac{1}{4} + \frac{7}{4}} = \frac{\frac{7}{4}}{\frac{8}{4}} = \frac{7}{8}$$

Scanned with CamScanner

Repeating steps 2-5 many times yields

$t(\text{bleue} \mid \text{house}) = 0.0001$

$t(\text{maison} \mid \text{house}) = 0.9999$ ✓

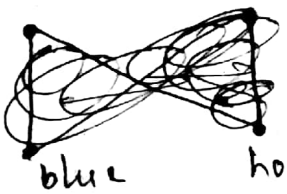
$t(\text{bleue} \mid \text{blue}) = 0.9999$ ✓

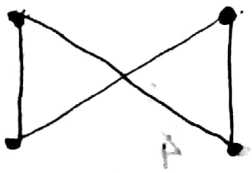
$t(\text{maison} \mid \text{blue}) = 0.0001$

maison bleue ⟷ blue house

maison ⟷ house

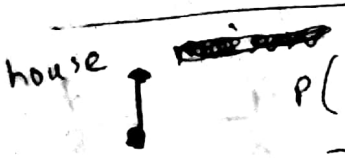
So we can calculate translation probabilites by the way of these alignment probabilites




$$P(\underline{f} \mid \underline{e}) = \sum_{a} P(a, f \mid e)$$

$$= \sum_{a} \prod_{j=1}^{m} t(f_j \mid e_i)$$




$P(f \mid e)$

$= \sum_{a} P(a, f \mid e)$

$= t(\text{maison} \mid \text{blue}) * t(\text{blue} \mid \text{house})$

$+ t(\text{blue} \mid \text{blue}) * t(\text{maison}$

Scanned with CamScanner

Dr. Mohammad Nurul Huda

CSE 6015, Statistical Machine Translation

Lecture sheet-7

## Computing Model 1 parameters:

How to compute translation probabilities for model 1 from a parallel corpus?

**Step1:** Determine candidates. For each English word e collect all foreign words f that co-occur at least once with e.

**Step2:** Initialize $P(f|e)$ uniformly

$$P(f|e) = \frac{1}{\text{No of co-occuring foreign words}}$$

**Step3:** Iteratively refine translation probabilities

```
for n iterations
    set tc to zero
    for each sentence pair (e, f) of lengths (l, m)
        for j = 1 to m
            total = 0
            for i = 1 to l
                total = total + P(f_j | e_i)
            end for
            for i = 1 to l
                tc(f_j | e_i) = tc(f_j | e_i) + P(f_j | e_i) / total
            end for
        end for
    end for
    for each word e
        total = 0
        for each word f
            total = total + tc(f|e)
        end for
        for each word f
            P(f|e) = tc(f|e) / total
        end for
    end for
end for
```

Scanned with CamScanner

Example: (parallel corpus)

the dog :: le chien

the cat :: le chat

step 1+2 (collect candidates and initialize uniformly)

the → le, chien, chat

dog → le, chien, chat

cat → le, chat, chien

NULL → le, chien, chat

$$P(le|the) = P(chien|the) = P(chat|the) = \frac{1}{3}$$

$$P(le|dog) = P(chien|dog) = P(chat|dog) = \frac{1}{3}$$

$$P(le|cat) = P(chat|cat) = P(chien|cat) = \frac{1}{3}$$

$$P(le|NULL) = P(chien|NULL) = P(chat|NULL) = \frac{1}{3}$$

step 3: Iterate

$e_1$ NULL $e_2$ the $e_3$ dog :: $f_1$ le $f_2$ chien

e: $l = 3$ f: $m = 2$

$j = 1$ total = 0 $i = 1$ total = $0 + P(f_1|e_1)$

$i = 2$ total = $P(f_1|e_1) + P(f_1|e_2)$

$i = 3$ total = $P(f_1|e_1) + P(f_1|e_2) + P(f_1|e_3)$

$= P(le|NULL) + P(le|the) + P(le|dog)$

$= \frac{1}{3} + \frac{1}{3} + \frac{1}{3} = 1$

$i = 1$ $tc(f_1|e_1) = 0 + \frac{P(f_1|e_1)}{total} =$

$\Rightarrow tc(le|NULL) = P(le|NULL)/1 = \frac{1/3}{1} = 0.33$

$i = 2$ $tc(le|the) = P(le|the)/1 = \frac{1/3}{1} = 0.333$

$i = 3$ $tc(le|dog) = P(le|dog)/1 = \frac{1/3}{1} = 0.333$

Scanned with CamScanner

$j = 2$ total = 0 $i = 1$ total = $0 + P(f_2|e_1) = P(f_1|e_1)$

$i = 2$ total = $P(f_2|e_1) + P(f_2|e_2)$

$i = 3$ total = $P(f_2|e_1) + P(f_2|e_2) + P(f_2|e_3)$

$= P(\text{chien}|\text{NULL}) + P(\text{chien}|\text{the}) + P(\text{chien}|\text{dog})$

$= 1$

$i = 1$ $tc(f_2|e_1) = 0 + \frac{P(f_2|e_1)}{\text{Total}} = 0 + \frac{\frac{1}{3}}{1} = 0.33$

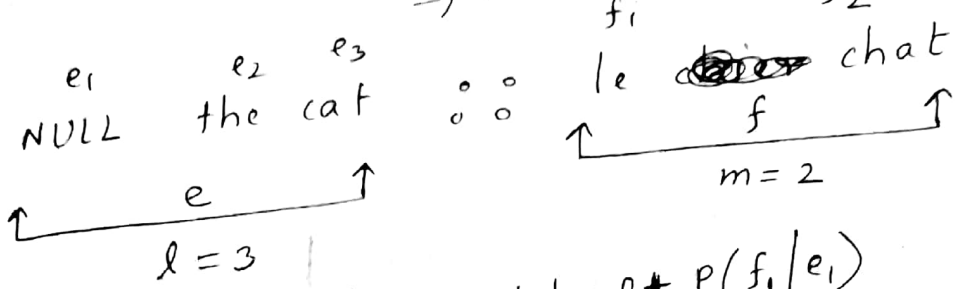
$\Rightarrow tc(\text{chien}|\text{NULL})$

$i = 2$ $tc(f_2|e_2) = 0 + \frac{P(f_2|e_2)}{\text{Total}} = 0 + \frac{\frac{1}{3}}{1} = 0.33$

$\Rightarrow tc(\text{chien}|\text{the})$

$i = 3$ $tc(f_2|e_3) = 0 + \frac{P(f_2|e_3)}{\text{Total}} = 0 + \frac{\frac{1}{3}}{1} = 0.33$

$\Rightarrow tc(\text{chien}|\text{dog})$




$j = 1$ total = 0 $i = 1$ total = $0 + P(f_1|e_1)$

$i = 2$ total = $P(f_1|e_1) + P(f_1|e_2)$

$i = 3$ total = $P(f_1|e_1) + P(f_1|e_2) + P(f_1|e_3)$

$= P(\text{le}|\text{NULL}) + P(\text{le}|\text{the}) + P(\text{le}|\text{cat})$

$= \frac{1}{3} + \frac{1}{3} + \frac{1}{3} = 1$

$i = 1$ $tc(\text{le}|\text{NULL}) = 0.333 + \frac{P(\text{le}|\text{NULL})}{\text{total}} = 0.333 + 0.333 = 0.666$

$i = 2$ $tc(\text{le}|\text{the}) = 0.333 + \frac{P(\text{le}|\text{the})}{\text{total}} = 0.333 + 0.333 = 0.666$

$i = 3$ $tc(\text{le}|\text{cat}) = 0 + \frac{P(\text{le}|\text{cat})}{\text{total}} = 0 + 0.333 = 0.333$

Scanned with CamScanner

$j=2$ total $=0$ $i=1$

$i=2$ total $= P(f_2|e_1) + P(f_2|e_2)$

$i=3$ total $= P(f_2|e_1) + P(f_2|e_2) + P(f_2|e_3)$

$= P(\text{chat}|\text{NULL}) + P(\text{chat}|\text{the}) + P(\text{chat}|\text{cat})$

$= 1$

$i=1$ $tc(\text{chat}|\text{NULL}) = 0 + \frac{P(\text{chat}|\text{NULL})}{\text{total}} = 0 + \frac{1/3}{1} = 0.333$

$i=2$ $tc(\text{chat}|\text{the}) = 0 + \frac{P(\text{chat}|\text{the})}{\text{total}} = 0 + \frac{1/3}{1} = 0.333$

$i=3$ $tc(\text{chat}|\text{cat}) = 0 + \frac{P(\text{chat}|\text{cat})}{\text{total}} = 0 + \frac{1/3}{1} = 0.333$

(Recompute translation probabilities)

$\text{total}(\text{the}) = 0 + tc(\text{le}|\text{the}) + tc(\text{chien}|\text{the}) + tc(\text{chat}|\text{the})$

$= 0.666 + 0.333 + 0.333$

$= 1.333$

$P(\text{le}|\text{the}) = \frac{tc(\text{le}|\text{the})}{\text{total}(\text{the})} = \frac{0.666}{1.333} = 0.5$

$P(\text{chien}|\text{the}) = \frac{tc(\text{chien}|\text{the})}{\text{total}(\text{the})} = \frac{0.333}{1.333} = 0.25$

$P(\text{chat}|\text{the}) = \frac{tc(\text{chat}|\text{the})}{\text{total}(\text{the})} = \frac{0.333}{1.333} = 0.25$

Scanned with CamScanner

**total (dog):**

$$total(dog) = 0 + tc(le|dog) + tc(chien|dog)$$
$$= 0 + \frac{1}{3} + \frac{1}{3}$$
$$= 0.666$$

$$P(le|dog) = tc(le|dog) / total(dog)$$
$$= \frac{0.333}{0.666}$$
$$= 0.5$$

$$P(chien|dog) = tc(chien|dog) / total(dog)$$
$$= \frac{0.333}{0.666}$$
$$= 0.5$$

**total (cat):**

$$total(cat) = 0 + tc(le|cat) + tc(chien|cat)$$
$$= 0.333 + 0.333$$
$$= 0.666$$

$$P(le|cat) = \frac{tc(le|cat)}{total(cat)}$$
$$= \frac{0.333}{0.666}$$
$$= 0.5$$

$$P(chat|cat) = \frac{tc(chat|cat)}{total(cat)}$$
$$= \frac{0.333}{0.666}$$
$$= 0.5$$

**total (NULL):**

$$total(NULL) = 0 + tc(le|NULL) + tc(chien|NULL) + tc(chat|NULL)$$
$$= 0.666 + 0.333 + 0.333 = 1.332$$

$$P(le|NULL) = \frac{tc(le|NULL)}{total(NULL)} = \frac{0.666}{1.332}$$

$$P(chien|NULL) = \frac{tc(chien|NULL)}{total(NULL)} = \frac{0.333}{1.332}$$

$$P(chat|NULL) = \frac{tc(chat|NULL)}{total(NULL)} = \frac{0.333}{1.332}$$

**iteration 2:**

Go to step 3

**iteration 3:**

Go to step 3

Scanned with CamScanner

# K- Nearest Neighbor Algorithm

Find a phoneme for test frame using the Euclidean distance measure.

| | | | | |
|---|---|---|---|---|
| /a/ : | 2 | 4 | 5 | Training Data |
| /a/ : | 3 | 5 | 6 | |
| /i/ : | 4 | 5 | 3 | |
| /i/ : | 3 | 4 | 5 | |
| /i/ : | 2 | 3 | 4 | |

Test Data : 2 5 4

**Answer:**

From Training Data :

$dist_a^2 = (2-2)^2 + (4-5)^2 + (5-4)^2 = 2$ *

$dist_a^2 = (3-2)^2 + (5-5)^2 + (6-4)^2 = 5$

$dist_i^2 = (4-2)^2 + (5-5)^2 + (3-4)^2 = 5$

$dist_i^2 = (3-2)^2 + (4-5)^2 + (5-4)^2 = 3$ *

$dist_i^2 = (2-2)^2 + (3-5)^2 + (4-4)^2 = 4$ *

if $K = 3$,

Minimum are $dist_a^2$, $dist_i^2$, $dist_i^2$

If we take majority voting,

One vote for a

Two vote for i

Winner is i

Output phoneme = /i/

Scanned with CamScanner

# Nearest Neighbor Algorithm

Find a phoneme for test frame using the Euclidean distance measure.

| | | | | |
|---|---|---|---|---|
| /a/ : | 2 | 4 | 5 | Training data |
| /a/ : | 3 | 5 | 6 | |
| /i/ : | 4 | 5 | 3 | |
| /i/ : | 3 | 4 | 5 | |
| /i/ : | 2 | 3 | 4 | |
| Test data : | 2 | 5 | 4 | |

**Answer:**

## Training Data:

**For /a/ : Mean calculation**

$$\mu_{1a} = \frac{2+3}{2} = 2.5, \quad \mu_{2a} = \frac{4+5}{2} = 4.5, \quad \mu_{3a} = \frac{5+6}{2} = 5.5$$

**For /i/ : Mean calculation**

$$\mu_{1i} = \frac{4+3+2}{3} = 3, \quad \mu_{2i} = \frac{5+4+3}{3} = 4, \quad \mu_{3i} = \frac{3+5+4}{3} = 4$$

## Testing Data:

Using Euclidean distance measure

$$d_a^2 = (\mu_{1a} - 2)^2 + (\mu_{2a} - 5)^2 + (\mu_{3a} - 4)^2$$
$$= (2.5-2)^2 + (4.5-5)^2 + (5.5-4)^2 = 2.75$$

$$d_i^2 = (\mu_{1i} - 2)^2 + (\mu_{2i} - 5)^2 + (\mu_{3i} - 4)^2$$
$$= (3-2)^2 + (4-5)^2 + (4-4)^2 = 2$$

Since $d_i^2 < d_a^2$, output phoneme = /i/

Scanned with CamScanner

# Nearest Neighbor Algorithm

Find a phoneme for test frame using the ~~Euclidean distance~~ Mahalanobis distance measure.

Training Data:

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| /a/: | 2 | 1 | /i/: | 3 | 2 | /u/: | 5 | 3 |
| /a/: | 1 | 2 | /i/: | 1 | 3 | /u/: | 4 | 2 |

Test Data: 2 3

Ans:

For /a/ phoneme:

Mean calculation:

$$\mu_{1a} = \frac{2+1}{2} = 1.5$$

$$\mu_{2a} = \frac{1+2}{2} = 1.5$$

Covariance calculation:

$$\Sigma_{1,a} = \frac{(1.5-2)^2 + (1.5-1)^2}{2} = 0.25$$

$$\Sigma_{2,a} = \frac{(1.5-1)^2 + (1.5-2)^2}{2} = 0.25$$

~~Inverse~~ Covariance Matrix:

$$\Sigma_a = \begin{bmatrix} \Sigma_{1,a} & 0 \\ 0 & \Sigma_{2,a} \end{bmatrix}$$

$$= \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$$

Inverse covariance matrix:

$$\Sigma_a^{-1} = \begin{bmatrix} \frac{1}{0.25} & 0 \\ 0 & \frac{1}{0.25} \end{bmatrix} = \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix}$$

Scanned with CamScanner

For /i/ phoneme:

Mean calculation:

$$\mu_{1,i} = \frac{3+1}{2} = 2$$

$$\mu_{2,i} = \frac{2+3}{2} = 2.5$$

Covariance calculation:

$$\Sigma_{1,i} = \frac{(2-3)^2 + (2-1)^2}{2} = 1$$

$$\Sigma_{2,i} = \frac{(2.5-2)^2 + (2.5-3)^2}{2} = 0.25$$

Diagonal covariance Matrix:

$$\Sigma_i = \begin{bmatrix} 1 & 0 \\ 0 & 0.25 \end{bmatrix}$$

Inverse covariance Matrix:

$$\Sigma_i^{-1} = \begin{bmatrix} \frac{1}{1} & 0 \\ 0 & \frac{1}{0.25} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}$$

For /u/ Phoneme

Mean calculation:

$$\mu_{1,u} = \frac{5+4}{2} = 4.5$$

$$\mu_{2,u} = \frac{3+2}{2} = 2.5$$

Covariance calculation:

$$\Sigma_{1,u} = \frac{(4.5-5)^2 + (4.5-4)^2}{2} = 0.25$$

$$\Sigma_{2,u} = \frac{(2.5-3)^2 + (2.5-2)^2}{2} = 0.25$$

Scanned with CamScanner

Diagonal covariance Matrix:

$$\Sigma_u = \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$$

Inverse covariance Matrix:

$$\Sigma_u^{-1} = \begin{bmatrix} \frac{1}{0.25} & 0 \\ 0 & \frac{1}{0.25} \end{bmatrix} = \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix}$$

Mahalanobis Distance Calculation:

$$x = \begin{bmatrix} 2 & 3 \end{bmatrix}$$

$$d_a^2 = (x - \mu_a) \Sigma_a^{-1} (x - \mu_a)^T$$

$$= \left( \begin{bmatrix} 2 & 3 \end{bmatrix} - \begin{bmatrix} 1.5 & 1.5 \end{bmatrix} \right) \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix} \left( \begin{bmatrix} 2 & 3 \end{bmatrix} - \begin{bmatrix} 1.5 & 1.5 \end{bmatrix} \right)^T$$

$$= \begin{bmatrix} 0.5 & 1.5 \end{bmatrix} \times \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix} \times \begin{bmatrix} 0.5 & 1.5 \end{bmatrix}^T$$

$$= \begin{bmatrix} 0.5 & 1.5 \end{bmatrix} \times \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix} \times \begin{bmatrix} 0.5 \\ 1.5 \end{bmatrix}$$

$$= \begin{bmatrix} 0.5 & 1.5 \end{bmatrix} \times \begin{bmatrix} 2 \\ 6 \end{bmatrix}$$

$$= 1 + 9$$

$$= 10$$

Scanned with CamScanner

(6)

$$d_i^r = \left( \begin{bmatrix} 2 & 3 \end{bmatrix} - \begin{bmatrix} 2 & 2.5 \end{bmatrix} \right) \times \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix} \times \left( \begin{bmatrix} 2 & 3 \end{bmatrix} - \begin{bmatrix} 2 & 2.5 \end{bmatrix} \right)^T$$

$$= \begin{bmatrix} 0 & 0.5 \end{bmatrix} \times \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0.5 \end{bmatrix} \times \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

$$= 0 + 1$$

$$= 1$$

$$d_u^r = \left( \begin{bmatrix} 2 & 3 \end{bmatrix} - \begin{bmatrix} 4.5 & 2.5 \end{bmatrix} \right) \times \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix} \times \left( \begin{bmatrix} 2 & 3 \end{bmatrix} - \begin{bmatrix} 4.5 & 2.5 \end{bmatrix} \right)^T$$

$$= \begin{bmatrix} -2.5 & 0.5 \end{bmatrix} \times \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix} \times \begin{bmatrix} -2.5 \\ 0.5 \end{bmatrix}$$

$$= \begin{bmatrix} -2.5 & 0.5 \end{bmatrix} \times \begin{bmatrix} -10 \\ 2 \end{bmatrix}$$

$$= 25 + 1$$

$$= 26$$

Here, $d_a^r = 10$

$d_i^r = 1$

$d_u^r = 26$

phoneme = $\text{argmin}(d_a^r, d_i^r, d_u^r)$

= /i/

Output phone = /i/

Scanned with CamScanner

# Morphology



Lecture-04
Dr. Md. Nurul Huda
Computational Linguistics

## Morphology:

- Scientific study of word formation
- study of patterns of word formation
- How words are orginated, grammatical forms, on what basis the word class is formed.

## Word:

A word is a minimal free form of a language that has a given sound, meaning and syntactic function.

## Language:

- An arbitrary vocal system used for human communication
- Elements in a language are arranged according to certain rules, but not at random.
- Elements at a lower level are combined according to certain rules to form elements at a higher level.

## Lexicology:

- A branch of linguistics concerned with the study of the vocabulary of a given language.
- Deals with words, their origin, development, structure, formation, meaning and usage

## Morpheme:

- Minimal, meaningful unit of a word or in the grammar of a language.
- Not further divisible or analyzable into smaller forms.
- Breaking up morpheme loses its identity and leaves its meaningless noises

Scanned with CamScanner

- May be a word or not a word
- A synonym for morpheme is glosseme

Example:

Unladylike ⟶ Un + lady + like

Encouragement ⟶ En + courage + ment

Disillusionment ⟶ Dis + illusion + ment

## Morph:

- A morph is the phonetic realization of a morpheme.
- The actual utterance of the morpheme.

## Allomorph:

- Allomorph is phonologically distinct variants of the same morpheme
- Different realization, manifestation of the same morpheme.

Examples of Allomorph:

[s] ⟶ English plural morpheme

[-s]

| | |
|---|---|
| cat | thank |
| cats | thanks |

Here [-s] with morphs ending in /p, t, k/

[-z]

| | |
|---|---|
| dog | rim |
| dogs | rims |
| ↓ | ↓ |
| dogz | rimz |

Here [-z] with morphs ending in /b, d, g, v, l/

[-iz]

| | |
|---|---|
| bus | horse |
| buses | horses |

Here [-iz] with morphs ending in /z, t/

Scanned with CamScanner

## Morpheme types

- Free morpheme

(A free morpheme is a word that can occur by itself)

(It can stand alone)

(We can add other morphemes in it)

Example: boy, man, establish are free morphemes

- Bound morpheme

(A grammatical unit that never occurs by itself, but is always attached to some other morphemes)

Examples:

| Bound Morpheme | Words |
|---|---|
| [s] | cats |
| [ness] | kindness |
| [ize] | visualize |

## Affixation

An affix is a bound morpheme that is joined before, after or within a root or stem

Types of affixes

- Prefix

(Added before the root or stem of the word)

Example: Unpredictable, Unresponsive, reschedule, etc

- Suffix

(Attached to the end of a root or stem)

Example: cats, eaten, eating, higher, highest

- Infix

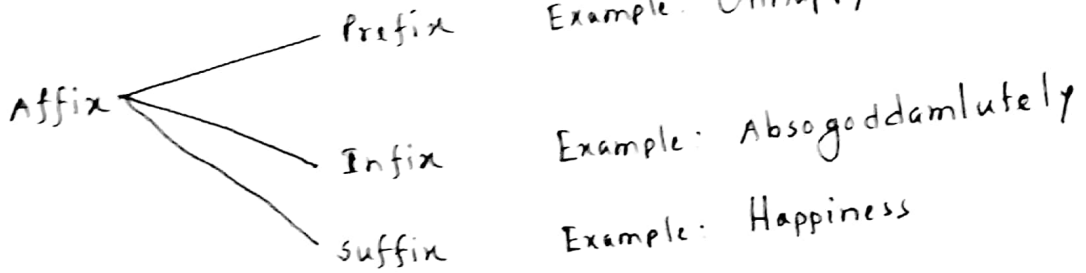
(Inserted within a root or stem)

(Added in the base form of a word to create a new word or intensify meaning)

Scanned with CamScanner

Example:

Absogoddam lutely




- Circumfixes:

Consists of two parts

- A prefix and a suffix that together create a new word

Example:

In Dutch 'berg' means 'mountain'

ge-berg-te means mountain chain

- Simulfix:

A simulfix is a change or replacement of vowels or consonants (usually vowels) which changes the meaning of a word

Example:

eat ⟶ ate

tooth ⟶ teeth

## Morphemes and classes

- Root

(A root is the irreducible core of a word, with absolutely nothing else attached to it)

Example:

jump – jumps
jump – jumping
jump – jumped

} Here, jump is the root

Scanned with CamScanner

stem

A stem deals with inflectional morphology
(Inflectional only, but not derivational affixes added to it)

Example:

friend ⟶ friends
friendship ⟶ friendships
greenhouse ⟶ greenhouses

worker ⟶ workers
shift ⟶ shifted

The stem is that part of a word that exists before the addition of any inflectional morpheme.

- Base

A base is any form to which affixes of any kind can be added.
It can be both inflectional or derivational

boy ⟶ boys
boy ⟶ boyish
boy ⟶ boyhood

[All roots are bases, bases are called stem in context of inflectional morphology]

Bound Morpheme

Definition: must be attached to another morpheme

Bound Morpheme

Derivational morpheme
a. may change syntactic class
b. to form new words
c. examples: -able, un-, re-, etc

Inflectional morpheme
a. Different forms of the same word
b. Not change syntactic class
c. Only 8 kinds in English:
-'s, -s (plural nouns), -ing, -ed/-e
-est, -er, -s (SV agreement)

Scanned with CamScanner

## Formation of new words:

1. **Derivational Morpheme**

- Adding an affix to the root or stem
- changing the meaning of the base to which they are attached

a) changing the meaning of the base to which they are attached

kind ⟶ unkind (Adjectives)

obey ⟶ disobey (Verbs)

b) by changing the word-class that a base belongs

kind ⟶ kindly

simple ⟶ simply

### Derivation of Nouns:

Verb ⟶ Noun

speak ⟶ Speaker

swim ⟶ Swimming

read ⟶ read / reading

Adjective ⟶ Noun

kind ⟶ kindness

good ⟶ goodness

black ⟶ blackness

Noun ⟶ Noun

mother ⟶ motherhood

sense ⟶ non-sense

### Derivation of Adjective:

Noun ⟶ Adjective

master ⟶ masterly

brother ⟶ brotherly

Verb ⟶ adjective

read ⟶ readable

love ⟶ loveable

Scanned with CamScanner

jective ⟶ Adjective

common ⟶ uncommon

possible ⟶ impossible

Derivation of Verbs

Noun ⟶ Verb

analysis ⟶ analyze

slave ⟶ enslave

Adjective ⟶ Verb

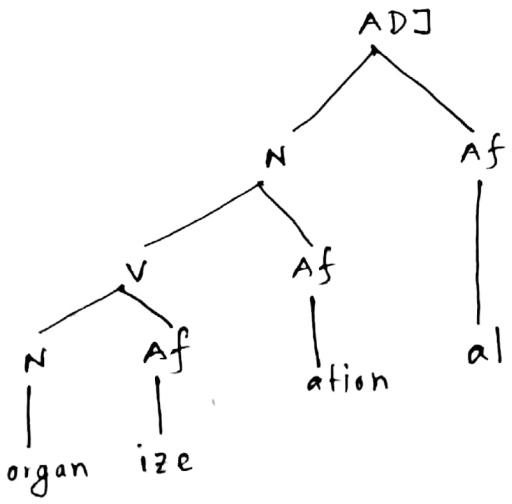
rich ⟶ enrich

legal ⟶ legalize

Verb ⟶ verb

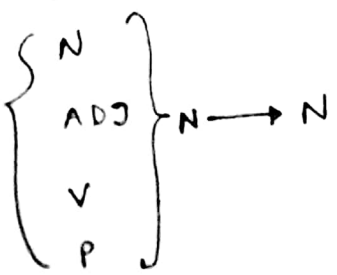
continue ⟶ discontinue

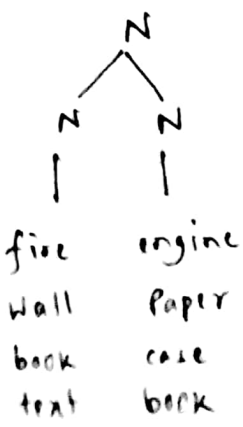
pack ⟶ Unpack

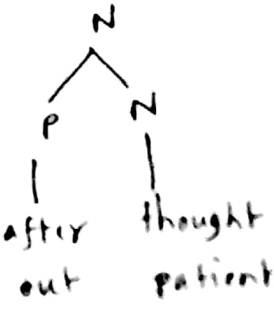



## Compounding

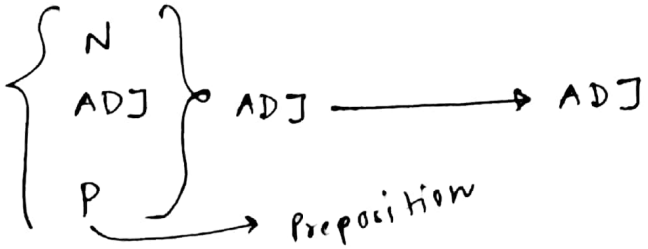
When two words or at least two root morphemes are joined together, the derived word results a compounding.

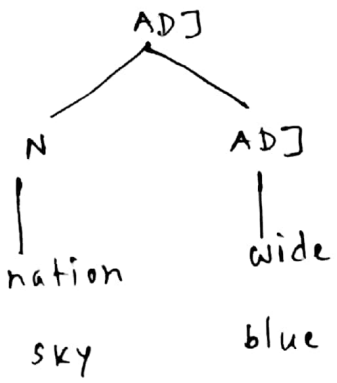


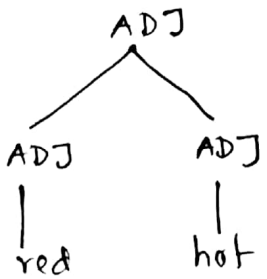



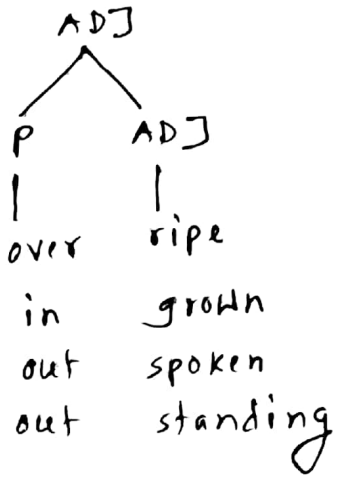


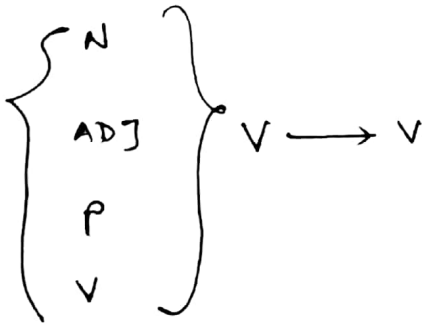

Scanned with CamScanner

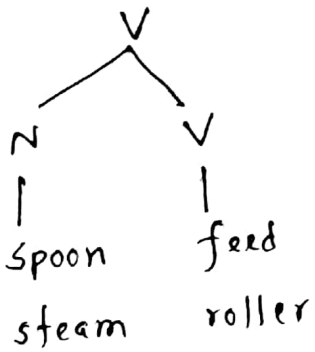


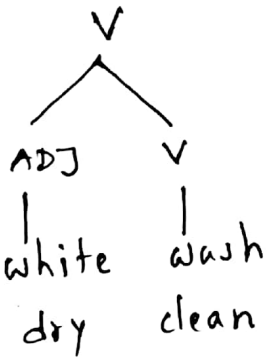



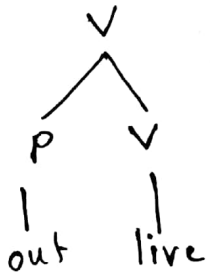



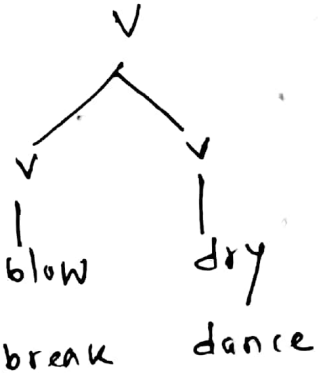




Scanned with CamScanner

## Duplication

- In this process all or part of the base is repeated as prefix or suffix
- Full or partial repetition of a free morpheme; sometimes with variation

| Full | With variation |
|---|---|
| so-so | zigzag |
| bye-bye | dilly-dally |
| | hotch potch |
| | hodge podge |
| | mishmash |
| | willy-nilly |
| | hanky-panky |

## Blending:

Similar to compounding, but parts of the free morphemes involved are lost
(Usually 1st part of 1st word + end of 2nd word)

| Word 1 | Word 2 | Word |
|---|---|---|
| breakfast | lunch | brunch |
| smoke | fog | smog |
| motor | hotel | motel |
| news | broadcast | newscast |

## Clipping (clipped forms)

Part of a free morpheme is cut off (i.e., shortening a polysyllabic word); often in casual speech.

Professor ⟶ Prof.
advertisement ⟶ ad
document ⟶ doc
Laboratory ⟶ lab
Automatic ⟶ auto

Scanned with CamScanner

## Acronyms:

Abbreviate a longer term by taking the initial letters

Example:

NATO (North Atlantic Treaty Organization)
TOEFL (Test of English as a Foreign Language)
AIDS (Acquired Immune Deficiency Syndrome)
NASA (National Aeronautics & Space Administration)

If unpronounceable ⟶ each letter is sounded out separately

ATM (Automatic teller machine)
VCR (Video cassette Recorder)

## Back formation

A word (usually a noun) is reduced to form another word of a different type (usually a verb).

editor ⟶ edit
donation ⟶ donate
babysitter ⟶ babysit
terminator ⟶ terminate

[The direction of derivation is inverted]

먹기 ⟶ 먹

## Coining

Creating a completely new free morpheme, which is unrelated to any existing morphemes; a rare thing

Example: googol
Nylon

Scanned with CamScanner

## Conversion

In some languages it is possible to form new lexeme merely by shifting the word category without adding any affixes.

Example:

table ⟶ to table
chair ⟶ to chair
water ⟶ to water

## Onomatopoeia:

Words imitate sounds in nature

Example:

A cat ⟶ meow
A cow ⟶ moo
Ring of a bell → ding-dong

## Extension of word formation rules

hamburger
cheeseburger
fishburger

marathon
telethon

alcoholic
workaholic

[part of a word is treated as a morpheme though it's not]

## Synthetic Compound

The head lexeme is derived from a verb

Example:

hand-washing
User-driven
home-made

Scanned with CamScanner

## Portmanteau

When a single morpheme represents simultaneously two or more grammatical functions is called portmanteau morpheme.

Example:

[-s] is used to denote 3rd person singular number
[-s] is used to denote plural number

* Run ⟶ Run ⟶ Run
  cut ⟶ cut ⟶ cut

## A Single Word with different morphemes

- One morpheme – nation
- Two morphemes – national
- Three morphemes – national -ize
- Four morphemes – de nation al ize
- More than four morphemes – de nation al iz ation

Scanned with CamScanner